

UNIVERSITÀ
DI TORINO

Università degli Studi di Torino

Corso di Laurea in Informatica

Riprogettazione e automazione dell'infrastruttura di CI/CD aziendale tramite Bitbucket Pipelines

Analisi critica del debito tecnico, implementazione di versioning con CI-gate per il testing automatizzato e ambienti di sviluppo con Dev Container

Tesi di Laurea

Relatore
Prof. Gianluca Pozzato

Candidato
Salvatore Pierri
Matricola 1043293

Anno Accademico 2025/2026

"Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata."

"I learned a lot, by the end of everything. The past is past, now, but that's... you know, that's okay! It's never really gone completely. The future is always built on the past, even if we won't get to see it. Still, it's... um, time for something new, now."

— *Riebeck*

Indice

1 Introduzione **1**

1.1 Contesto . . . 1
1.2 Il problema . . . 1
1.3 Obiettivi del lavoro . . . 2
1.4 Struttura della tesi . . . 2

2 Analisi dello stato iniziale **5**

2.1 L'ecosistema di repository . . . 5
2.2 Criticità rilevate . . . 6
2.2.1 Duplicazione e divergenza della logica . . . 6
2.2.2 Copertura disomogenea . . . 7
2.2.3 Versionamento inaffidabile . . . 8
2.2.4 Processi manuali e rilascio frammentato . . . 9
2.2.5 Ambienti di sviluppo non uniformi . . . 9
2.3 Requisiti . . . 9

3 Background tecnologico **11**

3.1 Integrazione e rilascio continui . . . 11
3.1.1 Bitbucket Pipelines . . . 12
3.1.2 Bash . . . 13
3.2 Git: tag, branch e merge . . . 13
3.2.1 Il modello a branch . . . 13
3.3 Versionamento semantico . . . 14
3.4 Containerizzazione . . . 14
3.4.1 Dev Container . . . 15
3.5 L'ecosistema Rust . . . 15
3.6 Pacchetti Debian e repository apt . . . 16

4 Una libreria condivisa per le pipeline **17**

4.1 Dal modello a silos al repository condiviso . . . 17

4.1.1 Alternative considerate . . . 18
4.2 Adozione graduale . . . 19
4.3 Struttura della libreria . . . 20
4.3.1 Punto d'ingresso unico . . . 21
4.3.2 Aggiornamenti sicuri: l'aggancio alla versione stabile . . . 21
4.3.3 Compatibilità all'indietro . . . 22
4.4 Passaggio di stato fra step . . . 23
4.5 Pipeline in due tempi . . . 23
4.6 Produzione degli artefatti . . . 25
4.7 Un output leggibile . . . 25
4.8 Esito . . . 26

5 Versionamento automatico 29
5.1 Il tag come fonte di verità . . . 29
5.2 Evoluzione del modello . . . 30
5.2.1 V1: un contatore unico . . . 30
5.2.2 V2: il ciclo di `develop` . . . 31
5.2.3 V3: un contatore per branch . . . 32
5.2.4 V4: branch paralleli e `develop` di appoggio . . . 33
5.3 Alternative considerate . . . 34
5.4 Le tre famiglie di tag . . . 35
5.5 Regole di calcolo . . . 35
5.5.1 I branch di lavoro . . . 36
5.5.2 `develop` decide il numero . . . 36
5.5.3 `master` pubblica . . . 37
5.5.4 L'origine del merge . . . 38
5.6 Quando decide lo sviluppatore . . . 39
5.7 Versioni e pacchetti Debian . . . 39
5.8 Ordinamento dei tag . . . 41
5.9 Esito . . . 41

6 Verifica automatica e rilascio sicuro 43
6.1 Il CI gate . . . 43
6.2 Eseguire la CI solo quando serve . . . 44
6.2.1 Il problema . . . 44
6.2.2 La decisione . . . 45
6.2.3 In caso di dubbio, verificare . . . 45
6.3 Controlli per linguaggio . . . 46

6.3.1 Rust . . . 47
6.3.2 Il limite della memoria . . . 47
6.3.3 Python e TypeScript . . . 48
6.4 Controlli che segnalano senza bloccare . . . 48
6.5 Guardie sul rilascio . . . 49
6.5.1 Il problema . . . 49
6.5.2 La guardia . . . 50
6.6 Quanto si risparmia . . . 51
6.7 Esito . . . 52

7 Ambienti di sviluppo riproducibili 53
7.1 Il problema . . . 53
7.2 Alternative considerate . . . 53
7.3 Template per Rust . . . 54
7.3.1 Immagine di base e strumenti . . . 54
7.3.2 Dipendenze precompilate con cargo-chef . . . 55
7.4 Template per Python . . . 56
7.5 Esperienza dello sviluppatore . . . 56
7.6 Evoluzione successiva . . . 57
7.7 Esito . . . 57

8 Risultati e valutazione 59
8.1 Verifica dei requisiti . . . 59
8.2 Copertura dei repository . . . 60
8.3 Semplicità delle pipeline . . . 60
8.4 Tempi e costi della CI . . . 62
8.5 Manutenibilità . . . 62
8.6 Limiti del lavoro svolto . . . 63

9 Conclusioni 65
9.1 Sintesi del lavoro svolto . . . 65
9.2 Sviluppi futuri . . . 66
9.3 Considerazioni personali . . . 66

A Codice sorgente 69
A.1 Libreria condivisa . . . 69
A.2 Esempio di pipeline di un repository . . . 84
A.3 Template dei Dev Container . . . 90

Bibliografia **95**

Ringraziamenti **97**

Elenco delle figure

2.1 Architettura delle *pipeline* prima dell'intervento: ogni repository possiede una propria copia della logica di versionamento, *build* e *deployment*. 6

4.1 Architettura centralizzata: le *pipeline* dei repository applicativi caricano la logica comune dalla libreria condivisa. 18
4.2 Flusso di una *pipeline*: la *pipeline* del branch calcola la versione e crea il tag, la *pipeline* del tag costruisce e pubblica gli artefatti. 24
4.3 Lo stesso tipo di informazione prima e dopo l'introduzione di `log-utils.sh`. 27

5.1 Tag prodotti da V1. I branch di lavoro condividono un solo contatore RC, `develop` avanza di una PATCH a ogni *commit* e il rilascio su `master` non corrisponde ad alcuna versione provata in staging. 31
5.2 Tag prodotti da V2. `develop` apre il ciclo a `1.1.0` e decide il numero di rilascio, ma i branch di lavoro condividono ancora lo stesso contatore RC. 32
5.3 Tag prodotti da V3. Ogni branch di lavoro ha un proprio contatore, riconoscibile dal suffisso, ma parte dalla base di `develop` invece che dal numero con cui verrà rilasciato. 33
5.4 Tag prodotti da V4. I branch di lavoro si numerano dall'ultima release ignorando gli altri branch aperti; `develop` adotta il numero del primo branch integrato e poi aggiunge una MINOR per ogni *feature* e una PATCH per ogni *bugfix*. 34

Elenco dei listati

2.1 Estratto della prima versione della pipeline del *gateway* MQTT (gennaio 2025): creazione del tag su `develop`. . . . 6
4.1 Il blocco con cui ogni pipeline carica la libreria. . . . 21
4.2 Il vecchio nome della variabile resta disponibile per i repository che lo usano. . . . 22
4.3 Contenuto di `.tag_ready` per un *commit* che porta già un tag. . . . 23
4.4 `log_step` ricava da sola file e funzione chiamante. . . . 25
5.1 Estrazione del branch di origine dal messaggio del merge commit (estratto). . . . 38
5.2 Risposta a una richiesta di tag manuale. . . . 39
5.3 Conversione di un tag Git in versione Debian. . . . 40
6.1 Lo step di verifica nella pipeline di un repository. . . . 43
6.2 Comportamento in caso di errore nella lettura dello stato (estratto). . . . 46
6.3 I due *pattern* delle pipeline per tag, uguali in tutti i repository. . . . 49
6.4 Il primo controllo di `deployment_env_guard` (estratto). . . . 50
7.1 Strumenti obbligatori e opzionali nel Dockerfile del template Rust (estratto; il file completo è nel listato A.6). . . . 54
7.2 Preparazione delle dipendenze nel Dockerfile del template Rust. . . . 55
A.1 `bootstrap.sh`. . . . 69
A.2 `pipeline-guards.sh`. . . . 70
A.3 `package-utils.sh`. . . . 74
A.4 `semver-utils.sh` (estratto). . . . 76
A.5 `bitbucket-pipelines.yml` del servizio di *scheduling*. . . . 84
A.6 `Dockerfile` del template Rust. . . . 90
A.7 `devcontainer.json` del template Rust. . . . 91

Capitolo 1

Introduzione

Questa tesi descrive il lavoro svolto durante il tirocinio presso Drivesec, sotto la supervisione del tutor aziendale Luca Ferrua. Il lavoro ha riguardato l'infrastruttura con cui il codice dell'azienda viene verificato, versionato e rilasciato: le *pipeline* di Continuous Integration e Continuous Deployment (CI/CD) dei repository applicativi e gli ambienti in cui gli sviluppatori scrivono il codice.

1.1 Contesto

Drivesec è un'azienda torinese che si occupa di sicurezza informatica dei prodotti connessi, nata nel settore *automotive* ed estesa agli elettrodomestici e ai dispositivi medicali. Il prodotto principale è una piattaforma per eseguire test di sicurezza in modo remoto e automatizzato.

Il codice è distribuito in 18 repository applicativi, scritti in Rust, Go, Python e TypeScript, ospitati su Bitbucket e automatizzati con Bitbucket Pipelines. Il team è piccolo, e ciascuno sviluppatore lavora spesso su più progetti e linguaggi. Per un'azienda che vende verifiche di sicurezza, poi, sapere con certezza quale versione di un componente è installata presso un cliente, da quale codice è stata costruita e quali controlli ha superato non è un dettaglio.

1.2 Il problema

All'inizio del tirocinio le *pipeline* dei 18 repository erano state scritte in momenti diversi e da persone diverse, ciascuna a modo suo. Non esisteva alcuna parte comune, e la stessa logica, dal calcolo della versione alla pubblicazione degli artefatti, era copiata da un repository all'altro, e nel tempo le copie si erano allontanate fra loro.

Una correzione andava ripetuta a mano in ogni file, e lo stesso compito veniva svolto in modi diversi da repository diversi.

Le conseguenze erano concrete. In 10 repository su 18 non esisteva una *pipeline* che partisse da sola a ogni modifica. Il codice veniva verificato con test automatici in 2 repository su 18; negli altri la *pipeline* trasformava il codice in un artefatto, qualunque fosse il suo stato. La versione degli artefatti veniva calcolata dalla *pipeline* solo in 4 repository; altrove era decisa a mano o letta da un file aggiornato a mano, con il rischio di pubblicare due artefatti diversi con lo stesso numero. Nessun repository impediva che una *release candidate* venisse pubblicata in produzione. Infine, gli ambienti di sviluppo venivano configurati a mano da ciascuno, diversi fra loro e diversi da quello in cui girava la *pipeline*.

Nessuno di questi problemi bloccava il lavoro quotidiano, perché le *pipeline* producevano gli artefatti che servivano. Ciascuno però rendeva il processo più costoso da mantenere e più dipendente dalla memoria di chi lo eseguiva. Il capitolo 2 documenta questa situazione con i dati raccolti dai repository e ne ricava i requisiti del lavoro.

1.3 Obiettivi del lavoro

O1. Centralizzare in una libreria condivisa la logica comune delle *pipeline* di tutti i repository, rendendone sicuro l'aggiornamento.

O2. Automatizzare il calcolo delle versioni e la creazione dei tag, con regole uguali per tutti i repository.

O3. Portare *format*, lint e test automatici su ogni repository, senza sprecare minuti di CI.

O4. Impedire che una *release candidate* arrivi in produzione.

O5. Rendere riproducibili gli ambienti di sviluppo.

1.4 Struttura della tesi

Il capitolo 2 analizza lo stato iniziale delle *pipeline* e ne ricava i requisiti. Il capitolo 3 introduce le tecnologie impiegate. Il capitolo 4 descrive la libreria condivisa e il modo in cui viene distribuita ai repository. Il capitolo 5 presenta il modello di versionamento automatico. Il capitolo 6 tratta la verifica automatica del codice e le

guardie sul rilascio. Il capitolo 7 descrive i template per gli ambienti di sviluppo. Il capitolo 8 valuta i risultati e il capitolo 9 traccia le conclusioni.

Capitolo 2

Analisi dello stato iniziale

Questo capitolo descrive le *pipeline* dell'azienda come si presentavano all'inizio del tirocinio, nel marzo 2026. Le criticità individuate sono il punto di partenza di tutto il lavoro successivo: da ciascuna nasce un requisito (sezione 2.3), e ogni requisito viene verificato nel capitolo 8.

2.1 L'ecosistema di repository

Il prodotto dell'azienda è composto da due parti. La prima è un insieme di servizi *backend* eseguiti come container, distribuiti sotto forma di immagini Docker pubblicate su un *registry*. La seconda è costituita da componenti installati direttamente sui dispositivi presso i clienti, distribuiti come pacchetti `.deb` attraverso un repository apt. Alcuni componenti producono entrambi gli artefatti.

Il lavoro ha riguardato 18 repository applicativi, ospitati su Bitbucket e scritti in quattro linguaggi. La tabella 2.1 li raggruppa per linguaggio e per tipo di artefatto prodotto. La varietà di linguaggi è rilevante, perché qualsiasi soluzione comune doveva adattarsi a strumenti di *build* e di test diversi per ciascuno di essi.

Tabella 2.1: I repository coinvolti, raggruppati per linguaggio.

Linguaggio	Repository	Artefatti prodotti
Rust	8	immagini Docker, pacchetti `.deb`
Go	3	immagini Docker, pacchetti `.deb`
Python	3	immagini Docker
TypeScript	2	immagini Docker
solo Docker	2	immagini Docker
Totale	**18**	

Ogni repository conteneva un proprio file `bitbucket-pipelines.yml`, scritto e mantenuto in modo indipendente dagli altri. Le *pipeline* erano nate in momenti diversi, le più vecchie già nel 2021, e da autori diversi; la maggior parte era stata scritta ed estesa nel tempo da un collega, aggiungendo di volta in volta ciò che serviva al repository di turno. Uno dei 18 repository, il servizio di *scheduling*, non aveva ancora alcuna *pipeline*. La figura 2.1 riassume la situazione, in cui ogni repository possedeva la propria copia della logica di versionamento, *build* e *deployment*, senza alcun punto in comune.

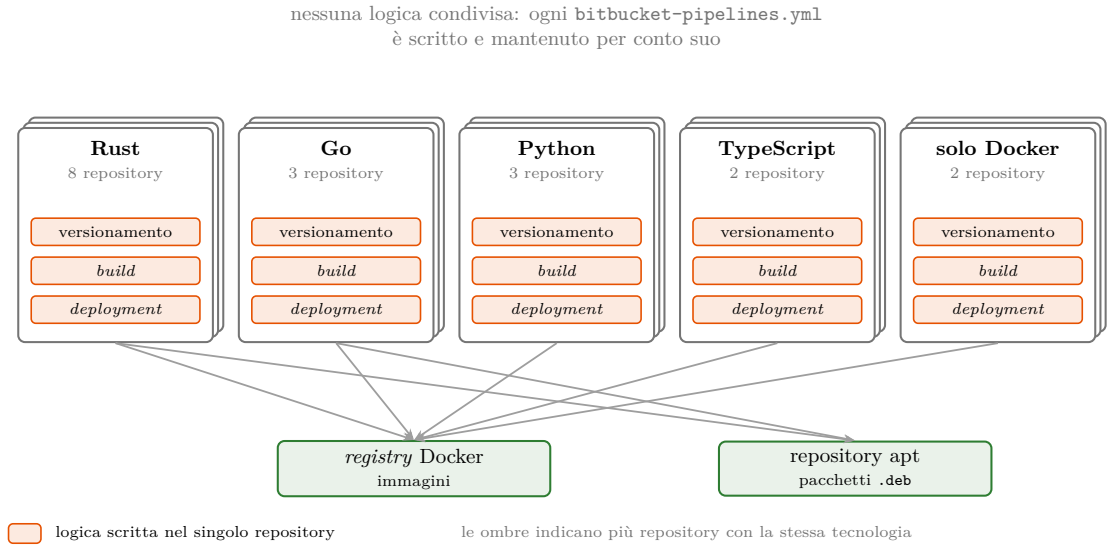


Figura 2.1: Architettura delle *pipeline* prima dell'intervento: ogni repository possiede una propria copia della logica di versionamento, *build* e *deployment*.

2.2 Criticità rilevate

2.2.1 Duplicazione e divergenza della logica

Tutta la logica delle *pipeline* era scritta in Bash direttamente dentro il file YAML. Fra repository diversi non esisteva alcun meccanismo di riuso, e anche all'interno dello stesso file il riuso era limitato a qualche ancora YAML: la *pipeline* del *gateway* MQTT, per esempio, conteneva due blocchi quasi identici per `develop` e per `master`, che differivano solo per pochi parametri. Il file era nato con 172 righe ed era arrivato a 675 al momento della migrazione.

```
1 - step:
```

```
    name: Tag Patch, Build, Package, and Upload
    trigger: manual
    script:
      - export VERSION=$(grep '^version' Cargo.toml | head -n 1 | cut -d '"' -f 2)
      # Tagging
      - git tag -a "v${VERSION}-develop" -m "Release version ${VERSION} (develop)"
          || echo "Tag already exists or another error occurred. Continuing ..."
      - git push origin "v${VERSION}-develop"
          || echo "Failed to push tag. Continuing..."
```

Listato 2.1: Estratto della prima versione della pipeline del *gateway* MQTT (gennaio 2025): creazione del tag su `develop`.

Il listato 2.1 mostra in poche righe tre problemi della prima generazione di *pipeline*: la versione letta da `Cargo.toml`, gli errori ignorati e l'esecuzione manuale. Il primo e il secondo erano stati in parte superati nelle *pipeline* più recenti (sezione 2.2.3). Il problema principale, però, è che righe come queste esistevano, con piccole varianti, in ogni repository. Una modifica alla logica comune, anche minima, andava ripetuta a mano in ciascuna *pipeline*; nel tempo le copie si erano allontanate l'una dall'altra, e lo stesso compito veniva svolto in modi diversi da repository diversi.

2.2.2 Copertura disomogenea

Il dato più significativo riguarda ciò che le *pipeline* facevano davvero. La tabella 2.2 riporta, per ciascuna funzionalità, quanti dei 18 repository la implementavano prima dell'intervento.

Tabella 2.2: Funzionalità presenti nelle *pipeline* prima dell'intervento, su 18 repository.

Funzionalità	Repository
una *pipeline* qualsiasi	17
una *pipeline* avviata da un *push* o da un tag	8
build e pubblicazione di un'immagine Docker	13
build di un pacchetto `.deb`	5
calcolo della versione e creazione del tag	4
test o analisi statica del codice in CI	2
controllo che impedisca di rilasciare una *release candidate*	0

Il primo dato è che in 10 repository su 18 non esisteva una *pipeline* automatica. In 9 il file conteneva soltanto *pipeline* di tipo *custom*, che vanno avviate a mano

dall'interfaccia di Bitbucket: nessun *push* e nessun tag le faceva partire. Il decimo, il servizio di *scheduling*, non aveva alcuna *pipeline*.

Il secondo dato riguarda il contenuto. Quasi tutte le *pipeline* esistenti producevano un artefatto, ma la maggior parte si fermava lì. In 10 repository costruivano un'immagine Docker e nient'altro, senza test né tag. Solo 2 repository verificavano il codice in modo automatico, con test e analisi statica; un terzo controllava soltanto lo stile del `Dockerfile`, e il suo step "Build and Test" costruiva l'immagine senza eseguire alcun test. Nessuno aveva un controllo sul rilascio. In questi repository la *pipeline* si limitava a trasformare il codice in un artefatto, qualunque fosse lo stato del codice.

2.2.3 Versionamento inaffidabile

Il versionamento è il punto in cui la storia delle *pipeline* si vede meglio. Nella prima generazione, rappresentata dal listato 2.1, la versione veniva letta da `Cargo.toml`. Il file, però, era aggiornato a mano e non sempre, quindi due *build* diverse potevano uscire con lo stesso numero, e il tag creato poteva non corrispondere al contenuto dell'artefatto. Il comando che crea il tag era inoltre seguito da `|| echo "...Continuing..."`: se la creazione falliva, per esempio perché il tag esisteva già, la *pipeline* stampava un messaggio e proseguiva, restando verde. Un errore di versionamento non lasciava quindi traccia nell'esito della *build*.

Prima dell'inizio del tirocinio un collega aveva riprogettato le *pipeline* di quattro repository Rust: il *gateway* MQTT, il catalogo dei test, il gestore dei flussi di test e il servizio di identificazione. In queste quattro la versione non veniva più letta da `Cargo.toml` ma calcolata a partire dall'ultimo tag Git; `Cargo.toml` veniva riallineato dalla *pipeline* stessa, lo stato passava da uno step all'altro attraverso file artefatto, e la creazione del tag avviava una seconda *pipeline* che costruiva gli artefatti. È il progetto da cui la libreria descritta nel capitolo 4 è partita. Restavano però tre limiti:

- il progetto esisteva in 4 repository su 18, sotto forma di quattro copie che già differivano fra loro. Nei tre repository Go il tag veniva creato a mano e la *pipeline* si limitava a usarlo; negli altri non c'era alcun tag;
- l'ultima versione veniva individuata con `git describe`, cioè in base alla data del *commit* e non al numero di versione, con le conseguenze descritte nella sezione 5.8;
- quasi ogni passaggio andava avviato a mano (sezione 2.2.4).

2.2.4 Processi manuali e rilascio frammentato

In 13 delle 17 *pipeline* esistenti almeno uno step era marcato `trigger: manual`, e doveva quindi essere avviato a mano dall'interfaccia di Bitbucket. Nelle quattro *pipeline* più complete gli step manuali andavano da 4 a 21: la creazione del tag, la *build* e la pubblicazione erano passaggi separati, ciascuno da avviare a mano. Il processo dipendeva quindi dalla memoria di chi lo eseguiva, e l'ordine dei passaggi non era garantito.

La *build* e la pubblicazione degli artefatti erano inoltre gestite in modo diverso da ogni repository: comandi diversi per costruire i pacchetti `.deb` e le immagini, modi diversi di pubblicarli sul repository apt e sui *registry*. Chi doveva modificare una *pipeline* che non conosceva doveva prima capire le convenzioni di quel particolare repository.

Anche i *log* rispecchiavano questa situazione. L'output delle *pipeline* non seguiva alcuna struttura. I messaggi dei comandi si mescolavano senza indicare a quale fase appartenessero, e capire dove e perché una *build* fosse fallita richiedeva di leggere l'intero *log*, cosa difficile per chi non aveva scritto la *pipeline*.

2.2.5 Ambienti di sviluppo non uniformi

L'ultima criticità non riguarda le *pipeline* ma il lavoro quotidiano degli sviluppatori. Nel team era frequente passare da un progetto all'altro, e quindi da un linguaggio e da un insieme di dipendenze all'altro. Ogni volta l'ambiente di sviluppo andava configurato a mano: versioni del compilatore, strumenti di formattazione e analisi, librerie di sistema. Oltre al tempo perso, ne risultavano ambienti diversi da sviluppatore a sviluppatore, e diversi da quello in cui girava la *pipeline*. A questo problema è dedicato il capitolo 7.

2.3 Requisiti

Dalle criticità descritte derivano i requisiti che hanno guidato il lavoro. Ciascuno di essi viene ripreso nel capitolo 8, dove se ne verifica il soddisfacimento.

R1 *Unica fonte di verità.* La logica comune delle *pipeline* risiede in un solo punto ed è riutilizzata da tutti i repository. (sezione 2.2.1)

R2 *Aggiornamenti sicuri.* Una modifica alla logica comune non deve poter rompere contemporaneamente tutte le *pipeline*. (sezione 2.2.1)

R3 *Versionamento automatico e coerente.* La versione di ogni artefatto è calcolata dalla *pipeline* con regole uguali in tutti i repository; nei casi ambigui la decisione torna allo sviluppatore. (sezione 2.2.3)

R4 *Adattabilità.* La *pipeline* riconosce da sola il linguaggio e la struttura del progetto. (sezione 2.1)

R5 *Verifica automatica ed efficiente. Format*, lint e test girano in CI su ogni repository, ma solo quando servono. (sezione 2.2.2)

R6 *Rilascio sicuro.* Una *release candidate* non può raggiungere la produzione. (sezione 2.2.2, sezione 2.2.4)

R7 *Riproducibilità degli ambienti di sviluppo.* (sezione 2.2.5)

Capitolo 3

Background tecnologico

Questo capitolo introduce le tecnologie su cui si basa il lavoro. Non ne dà una descrizione completa, per la quale si rimanda ai riferimenti, ma si ferma agli aspetti che i capitoli successivi danno per noti, e per ciascuno indica perché è rilevante.

3.1 Integrazione e rilascio continui

L'*integrazione continua* (*Continuous Integration*, CI) è la pratica per cui gli sviluppatori integrano spesso il proprio lavoro nel codice comune, e ogni integrazione viene verificata da una *build* automatica che comprende i test [10]. Lo scopo è scoprire gli errori quando sono ancora piccoli e se ne conosce la causa, invece che al momento del rilascio, quando si sono sommati.

Humble e Farley estendono l'idea al rilascio [12]. Con la *Continuous Delivery* ogni modifica che supera i controlli automatici produce un artefatto pronto per essere rilasciato, e il rilascio diventa una decisione e non un'operazione tecnica. Con il *Continuous Deployment* anche quella decisione è automatica: ogni modifica che supera i controlli arriva in produzione senza intervento umano. Lo strumento che rende possibili entrambe è la *pipeline* di *deployment*, una sequenza di fasi automatiche (*build*, test, pubblicazione) che ogni modifica attraversa. Forsgren, Humble e Kim hanno mostrato, su dati raccolti da decine di migliaia di professionisti, che queste pratiche sono associate sia a una maggiore frequenza dei rilasci sia a una minore probabilità che un rilascio causi un guasto [9].

Con questa distinzione, il flusso descritto nei capitoli successivi è di *Continuous Delivery*: verifica, versionamento e costruzione degli artefatti sono automatici, mentre la creazione del tag e la pubblicazione in produzione restano un passaggio avviato da una persona (sezione 4.5).

3.1.1 Bitbucket Pipelines

Bitbucket Pipelines è il servizio di CI integrato in Bitbucket, la piattaforma su cui l'azienda ospitava già i propri repository; la scelta del servizio era quindi un vincolo di partenza. Una *pipeline* è descritta da un file `bitbucket-pipelines.yml` nella radice del repository [2]. Gli aspetti rilevanti per la tesi sono sei.

Step e container

Una *pipeline* è una sequenza di *step*, e ogni step viene eseguito in un container Docker nuovo, a partire da un'immagine scelta per lo step. Nulla di ciò che uno step crea sopravvive al suo termine, salvo ciò che viene dichiarato esplicitamente: gli *artefatti*, file copiati negli step successivi, e le *cache*, cartelle conservate fra un'esecuzione e l'altra. Su questo si basa il passaggio di stato descritto nella sezione 4.4.

Avvio

Le *pipeline* si dichiarano per branch, per tag o come *pipeline* personalizzate avviate a mano. Branch e tag si indicano con *pattern* (*glob*): se un nome corrisponde a più *pattern* viene eseguita una sola *pipeline*, scelta in base alla specificità del *pattern*. Questa regola è all'origine della guardia descritta nella sezione 6.5.

Step manuali

Uno step marcato `trigger: manual` non parte da solo, ma attende che qualcuno lo avvii dall'interfaccia; finché non viene avviato, la *pipeline* resta in pausa.

Variabili e segreti

Le credenziali si definiscono come variabili a livello di *workspace*, di repository o di ambiente, e quelle protette non compaiono nei *log*.

Deployment

Uno step può dichiarare l'ambiente in cui pubblica (*deployment*), per esempio *staging* o produzione; Bitbucket tiene traccia di che cosa è stato pubblicato dove.

Servizi e taglia

Uno step può affiancare al proprio container dei servizi, fra cui un *daemon* Docker, e dichiarare una taglia: `1x` corrisponde a 4 GB di memoria, `2x` a 8 GB e così via, e la memoria è condivisa fra lo step e i suoi servizi. I minuti di esecuzione sono a pagamento e vengono conteggiati moltiplicando la durata

per la taglia [1], ed è per questo che eseguire la verifica solo quando serve ha un valore economico (sezione 6.6).

3.1.2 Bash

La libreria descritta nel capitolo 4 è scritta in Bash, la *shell* predefinita delle immagini Linux usate dalle *pipeline* [11]. Tre sue caratteristiche sono rilevanti. Il comando `source` esegue un file nella *shell* corrente invece che in un processo separato, quindi le funzioni e le variabili che il file definisce restano disponibili a chi lo ha caricato, ed è così che una *pipeline* acquisisce le funzioni della libreria. Ogni comando restituisce un codice d'uscita, zero in caso di successo, e uno step di Bitbucket fallisce quando un comando termina con un codice diverso da zero. Infine Bash espone a ogni funzione la propria pila di chiamate (le variabili `BASH_SOURCE` e `FUNCNAME`), che la libreria usa per indicare nel *log* da dove proviene ogni messaggio (sezione 4.7).

3.2 Git: tag, branch e merge

Git registra la storia di un progetto come una sequenza di *commit*, ciascuno dei quali punta al proprio genitore [4]. Un *branch* è un riferimento mobile a un *commit*, che avanza a ogni nuovo *commit*; un *tag* è invece un riferimento fisso, che per convenzione non viene più spostato dopo la pubblicazione e si usa per marcare le versioni rilasciate.

Unire un branch in un altro può produrre storie diverse, e la differenza è centrale nel capitolo 5. Un *merge commit* è un *commit* con due genitori: il primo è l'ultimo *commit* del branch di destinazione, il secondo l'ultimo del branch unito. Dalla sua struttura, e dal messaggio che Bitbucket compone in modo prevedibile (*Merged in feature/x (pull request #12)*), si può risalire al branch di provenienza. Un merge *fast-forward* non crea alcun *commit*: se il branch di destinazione non è avanzato, il suo riferimento viene semplicemente spostato in avanti. Uno *squash merge* riassume tutte le modifiche del branch in un unico *commit* con un solo genitore. In entrambi gli ultimi casi l'informazione su quale branch sia stato unito non resta nella storia.

3.2.1 Il modello a branch

L'azienda seguiva una variante del modello noto come *Git flow* [8]. Il branch `master` contiene il codice di produzione e riceve solo merge. Il branch `develop` raccoglie il lavoro destinato al prossimo rilascio e viene provato in *staging*. Ogni modifica nasce su un branch di lavoro staccato da `develop`, con un prefisso che ne indica il tipo, `feature/` per una funzionalità e `bugfix/` per una correzione, e vi rientra con una *pull*

request. Le correzioni urgenti nascono invece da `master` su un branch `hotfix/` e vi rientrano direttamente. Rispetto al modello originale manca il branch di *release*: il passaggio da `develop` a `master` è un merge diretto. Il modello di versionamento del capitolo 5 sfrutta proprio questi prefissi per decidere come incrementare il numero di versione.

3.3 Versionamento semantico

Il *Semantic Versioning* [16] assegna a ogni versione un numero nella forma `MAJOR.MINOR.PATCH`. Si incrementa la MAJOR per una modifica incompatibile con le versioni precedenti, la MINOR per una funzionalità compatibile, la PATCH per una correzione compatibile; incrementando una componente, quelle alla sua destra tornano a zero. Una versione di pre-rilascio si indica aggiungendo un trattino e uno o più identificatori, come in `1.4.0-rc2`, dove `rc` sta per *release candidate*: una versione candidata a diventare `1.4.0`, che la precede nell'ordinamento.

La specifica fissa anche come ordinare due versioni, ma non tutti gli strumenti la seguono alla lettera. In SemVer gli identificatori di pre-rilascio che contengono lettere si confrontano carattere per carattere, quindi `rc10` verrebbe prima di `rc9`; Git, con l'ordinamento "per versione", confronta invece numericamente le parti numeriche e mette `rc9` prima di `rc10`, ma per impostazione predefinita pone `1.2.1-rc3` *dopo* `1.2.1`, a meno di indicargli che `-rc` è un suffisso di pre-rilascio (sezione 5.8). Ogni ecosistema, poi, ha la propria grammatica delle versioni: Python scrive `1.4.0rc2`, Debian usa una sintassi ancora diversa (sezione 3.6). Un tag Git che diventa anche versione di un pacchetto e tag di un'immagine Docker deve quindi essere valido, e ordinato correttamente, in tutti questi contesti; il capitolo 5 mostra dove questo ha richiesto una conversione esplicita.

3.4 Containerizzazione

Docker esegue le applicazioni in container, ambienti isolati che condividono il *kernel* della macchina ospite ma hanno un proprio *filesystem*, una propria rete e propri processi [7]. Un container nasce da un'immagine, costruita a partire da un `Dockerfile`: ogni istruzione del file produce uno strato (*layer*) dell'immagine. Gli strati vengono conservati in una cache, e una nuova costruzione riusa uno strato finché né l'istruzione né i file che essa copia sono cambiati; appena uno strato cambia, vengono ricostruiti quello e tutti i successivi. L'ordine delle istruzioni decide quindi quanto della costruzione si può riusare, aspetto sfruttato nel capitolo 7. Le immagini si pubblicano

su un *registry*, identificate da un nome e da un tag; a differenza dei tag Git, i tag Docker ammettono solo lettere, cifre, punti, trattini e trattini bassi, e possono essere spostati, cosa che permette di avere un tag mobile come `staging-latest`.

3.4.1 Dev Container

La specifica *Development Containers* [6] descrive come usare un container come ambiente di sviluppo completo. Un file `devcontainer.json` indica quale immagine usare, o quale `Dockerfile` costruire, quali componenti aggiuntivi installare (*features*), quali estensioni attivare nell'editor e quali comandi eseguire dopo la creazione del container (`postCreateCommand`). Visual Studio Code legge il file, avvia il container, vi monta la cartella del progetto e si collega ad esso. Lo sviluppatore lavora nell'editor come al solito, ma compilatore, strumenti e librerie sono quelli del container, uguali per tutti.

3.5 L'ecosistema Rust

Rust è un linguaggio compilato che garantisce la sicurezza della memoria senza un *garbage collector*, verificando a tempo di compilazione come i dati vengono condivisi e modificati [13]. In azienda era il linguaggio di 8 dei 18 repository, e quello per cui la *pipeline* esegue i controlli più completi. Ai fini della tesi contano soprattutto gli strumenti, riassunti nella tabella 3.1.

Tabella 3.1: Gli strumenti dell'ecosistema Rust citati nella tesi.

Strumento	Funzione	Dove
CARGO	compila, esegue i test, gestisce le dipendenze [17]	ovunque
`Cargo.toml`	manifesto del progetto: nome, versione, dipendenze	sezione 5.1
`Cargo.lock`	versioni esatte di tutte le dipendenze, per *build* riproducibili	sezione 5.1
RUSTFMT	formattazione del codice	sezione 6.3
CLIPPY	analisi statica: errori probabili e codice non idiomatico	sezione 6.3
CARGO-NEXTEST	esecuzione dei test, ciascuno in un processo separato [14]	sezione 6.3
CARGO-AUDIT	vulnerabilità note nelle dipendenze	sezione 6.4
CARGO-DEB	pacchetto Debian da un progetto CARGO [3]	sezione 4.6
CARGO-CHEF	compilazione delle sole dipendenze, per la cache Docker [15]	sezione 7.3.2

Due concetti ricorrono nei capitoli successivi. Un *workspace* CARGO raccoglie più pacchetti in un unico repository, con un solo `Cargo.lock`. Un test marcato con

l'attributo `#[ignore]` viene saltato, a meno di chiederne esplicitamente l'esecuzione; il team lo usava per i test che richiedono servizi esterni (sezione 6.3).

3.6 Pacchetti Debian e repository apt

I componenti installati sui dispositivi presso i clienti sono distribuiti come pacchetti Debian (`.deb`): un archivio che contiene i file da installare e un file di controllo con nome, versione e dipendenze del pacchetto [5]. I dispositivi li ricevono da un repository apt, una struttura di file statici (i pacchetti e gli indici che li elencano) che può essere ospitata su qualsiasi servizio in grado di servire file, come Amazon S3. Il gestore APT confronta la versione installata con quella disponibile e aggiorna il pacchetto se la seconda è più recente.

La versione di un pacchetto Debian ha la forma `[epoca:]versione-revisione`. La *versione* è quella del software, la *revisione* quella del pacchetto: `1.3.4-2` è la seconda pacchettizzazione della versione `1.3.4`. La revisione è tutto ciò che segue l'*ultimo* trattino, e una versione senza trattino equivale a una con revisione `0`. L'ordinamento, definito dalla *Debian Policy* e applicato da DPKG, confronta alternativamente parti non numeriche, carattere per carattere, e parti numeriche, come numeri. Nelle parti non numeriche il carattere `~` precede qualsiasi altro carattere e anche la fine della stringa: `1.3.4~rc5` precede quindi `1.3.4`. È il carattere previsto per le versioni di pre-rilascio, ed è la chiave della correzione descritta nella sezione 5.7.

Capitolo 4

Una libreria condivisa per le pipeline

Il capitolo 2 ha mostrato che il problema di fondo delle *pipeline* era che la stessa logica esistesse in molte copie, ciascuna con le proprie varianti. Questo capitolo descrive la risposta a quel problema: una libreria Bash, ospitata in un repository dedicato, che tutte le *pipeline* caricano all'avvio. I capitoli successivi descrivono le due funzioni più articolate della libreria, il calcolo delle versioni (capitolo 5) e la verifica automatica del codice (capitolo 6); qui si tratta di come la libreria è fatta, di come arriva ai repository e di come viene aggiornata senza rompere nulla.

4.1 Dal modello a silos al repository condiviso

Nel modello di partenza (figura 2.1) ogni repository possedeva tutta la logica della propria *pipeline*: calcolo della versione, creazione del tag, *build* e pubblicazione degli artefatti. Era un modello a silos, senza parti condivise, in cui ogni correzione andava ripetuta in ciascun file. Il caso più evidente erano le quattro *pipeline* pilota descritte nella sezione 2.2.3: nate dallo stesso progetto, a pochi mesi di distanza avevano già quattro versioni diverse della stessa logica.

L'idea della libreria è spostare questa logica fuori dai repository applicativi. Il file `bitbucket-pipelines.yml` di ogni repository conserva la struttura della *pipeline*: quali step esistono, in che ordine, con quali immagini e su quali branch. Il *contenuto* degli step, cioè i comandi che calcolano una versione o decidono se un artefatto può essere pubblicato, diventa una chiamata a una funzione della libreria. La figura 4.1 mostra il risultato. I repository applicativi restano indipendenti, mentre la logica comune risiede in un solo punto.

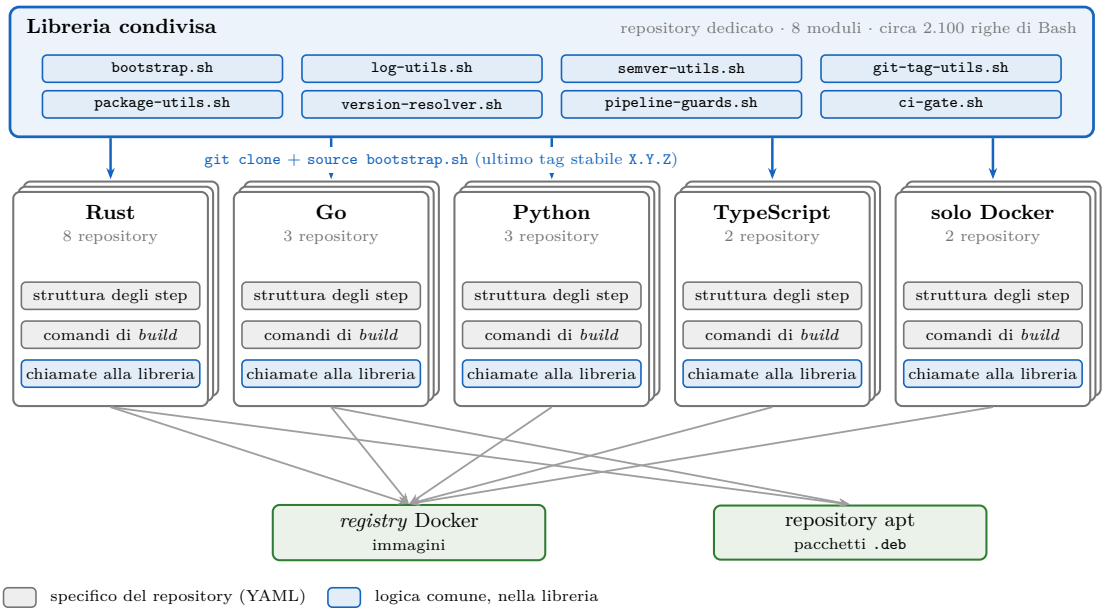


Figura 4.1: Architettura centralizzata: le *pipeline* dei repository applicativi caricano la logica comune dalla libreria condivisa.

4.1.1 Alternative considerate

Per condividere codice fra le *pipeline* di Bitbucket esistono diverse strade; prima di scegliere ne sono state valutate tre.

Sottomoduli Git

Ogni repository includerebbe la libreria come sottomodulo, fissato a un *commit* preciso. Il controllo della versione sarebbe totale, ma aggiornare la libreria richiederebbe un *commit* in ciascuno dei 18 repository, cioè proprio il lavoro ripetuto che si voleva eliminare.

Bitbucket Pipes

Sono il meccanismo di riuso offerto da Bitbucket: uno step richiama un'immagine Docker che esegue un compito. Ogni funzione richiederebbe però un'immagine da costruire, pubblicare e versionare, e una *pipe* gira in un container separato, quindi non può modificare le variabili dello step che la chiama. Per funzioni piccole e numerose, che devono passarsi stato a vicenda, il costo sarebbe stato sproporzionato.

Ancore YAML

Il formato YAML permette di definire un blocco una volta e richiamarlo altrove. Le *pipeline* le usavano già, ma un'ancora vale solo dentro il file in cui è definita,

quindi risolve la duplicazione all'interno di un repository ma non fra repository diversi.

La scelta è caduta su un repository Git che contiene script Bash e che ogni *pipeline* clona e carica con `source` all'avvio. Le ragioni sono quattro. Le funzioni girano nella stessa *shell* dello step, quindi leggono e impostano le sue variabili senza meccanismi intermedi. Ogni repository può scegliere quale versione della libreria usare (sezione 4.3.2). La libreria si può provare in locale, prima di pubblicarla, perché è fatta degli stessi comandi che girano nella *pipeline*. Infine non introduce dipendenze: richiede soltanto Bash e gli strumenti già presenti nelle immagini Linux dei *runner* (GIT, CURL, JQ), e per accedervi basta un unico *token* Bitbucket, lo stesso per tutti i repository.

4.2 Adozione graduale

La libreria non è stata scritta da zero. È nata estraendo le funzioni dalle quattro *pipeline* pilota scritte da un collega, le più complete fra quelle esistenti. Molte funzioni portano ancora nel codice il commento che ne indica l'origine (*shared in mqtt-bridge, test-catalog, test-flow-engine, test-id-service*): le quattro copie sono state confrontate, le differenze sono state risolte in un'unica versione, e il risultato è diventato una funzione della libreria.

In questa prima fase non era possibile provare le modifiche direttamente sui repository aziendali. Le *pipeline* sono state quindi copiate nel repository della libreria e fatte girare in locale con BITBUCKET-PIPELINE-RUNNER, uno strumento che riproduce l'esecuzione di Bitbucket su una macchina di sviluppo. Le variabili e i segreti aziendali (credenziali, *token*, indirizzi dei *registry*) sono stati sostituiti da valori fittizi, e i servizi esterni simulati. Solo quando le *pipeline* pilota producevano in locale gli stessi risultati di quelle originali, la libreria è stata adottata nei primi repository reali.

Validata sui primi repository, la libreria è stata estesa agli altri, fino a coprire tutti i 18 repository applicativi. L'estensione ha richiesto più che una semplice sostituzione. In 10 repository non esisteva una *pipeline* automatica, e negli altri non pilota la *pipeline* svolgeva una sola parte del lavoro, la *build* di un'immagine o di un pacchetto (sezione 2.2.2). Per questi la migrazione ha significato scrivere la *pipeline* quasi da zero, con gli step di verifica, versionamento e controllo sul rilascio che prima non c'erano. Non tutte le migrazioni sono state eseguite da chi ha scritto la libreria: quattro repository sono stati migrati da colleghi del team, che hanno adottato la libreria in autonomia.

L'intervento sulle *pipeline*, del resto, non era previsto dal piano di tirocinio. In origine il tirocinio doveva servire a conoscere le architetture usate ogni giorno dall'azienda e ad approfondirne alcune, senza un compito di sviluppo. Le *pipeline* sono state il primo argomento presentato, con il loro funzionamento e le ragioni delle scelte fatte, a partire da due dei quattro repository pilota, quelli in cui erano più complete. Già in quella fase è emerso il problema descritto nel capitolo 2: quasi ogni modifica a una *pipeline* andava ripetuta identica in tutte le altre. Il problema è stato segnalato al collega che nelle prime settimane seguiva il tirocinio. La sua risposta è stata la richiesta di proporre una soluzione senza toccare i repository aziendali, e da qui nasce l'ambiente locale con BITBUCKET-PIPELINE-RUNNER e i servizi simulati descritto sopra. Dopo una dimostrazione su quell'ambiente sono stati concessi i permessi di modifica sulle *pipeline* dei due repository. Risolti i primi problemi su questi, l'accesso è stato esteso gradualmente ad altri repository, fino a comprendere *pipeline* di altri progetti dell'azienda. L'adozione graduale ha quindi limitato il rischio e, allo stesso tempo, ha permesso al team di vedere la soluzione funzionare prima di estenderla.

4.3 Struttura della libreria

La libreria è composta da otto moduli, per un totale di circa 2.100 righe di Bash. La tabella 4.1 ne riporta le responsabilità.

Tabella 4.1: I moduli della libreria.

Modulo	Righe	Responsabilità
`bootstrap.sh`	29	punto d'ingresso: carica tutti gli altri moduli
`log-utils.sh`	91	output strutturato (sezione 4.7)
`semver-utils.sh`	672	calcolo della prossima versione (capitolo 5)
`git-tag-utils.sh`	259	elenco ordinato dei tag, creazione e pubblicazione del tag, allineamento di `Cargo.toml`
`package-utils.sh`	102	conversione del tag in versione Debian e PEP 440, ambiente di destinazione
`version-resolver.sh`	258	orchestrazione: dal tag o dal branch alla versione, ai file di stato
`pipeline-guards.sh`	146	guardie che decidono se uno step può proseguire
`ci-gate.sh`	539	verifica automatica del codice (capitolo 6)

I moduli hanno una gerarchia semplice. Alla base c'è `log-utils.sh`, usato da tutti gli altri. Le funzioni della libreria stampano attraverso i suoi comandi, salvo pochi casi come il *banner* del *gate*, così che il formato dell'output si modifica in un solo punto. Sopra di esso stanno i moduli che lavorano su versioni e tag (`semver-`

`utils.sh`, `git-tag-utils.sh`, `package-utils.sh`), e `version-resolver.sh` li combina nelle funzioni che le *pipeline* chiamano davvero. Le guardie e il *gate* di CI sono indipendenti dal resto.

4.3.1 Punto d'ingresso unico

Nelle prime versioni ogni *pipeline* caricava i moduli uno per uno, con una riga `source` per ciascuno. Aggiungere un modulo alla libreria significava quindi modificare tutti i repository, cioè reintrodurre in piccolo il problema che la libreria doveva risolvere. Il file `bootstrap.sh` elimina questo passaggio: carica tutti i moduli nell'ordine corretto, e i repository caricano soltanto lui (listato A.1). Il listato 4.1 mostra il blocco, identico in tutti i repository, con cui ogni step prepara la libreria.

```
1 git clone --filter=blob:none --no-checkout \
2   https://x-token-auth:${PIPE_SCRIPT_TOKEN}@bitbucket.org/esempio/pipeline-
    ↪ shared-lib.git \
3   /tmp/pipeline-utils
4 PIPE_REF="$(git -C /tmp/pipeline-utils tag --list --sort=-v:refname \
5   | grep -E '^[0-9]+\.[0-9]+\.[0-9]+$' | head -1)"
6 git -C /tmp/pipeline-utils checkout --quiet "${PIPE_REF:-master}"
7 source /tmp/pipeline-utils/reusable-bash-code/lib/bootstrap.sh
```

Listato 4.1: Il blocco con cui ogni pipeline carica la libreria.

Il clone non può essere spostato dentro la libreria, perché non si può chiamare una funzione di un repository che non è ancora stato clonato. Per questo le prime quattro righe restano nel file YAML, definite una volta come ancora e richiamate da ogni step. L'opzione `--filter=blob:none` scarica la storia del repository senza il contenuto dei file, che viene recuperato soltanto per la versione effettivamente usata, così il clone resta rapido anche quando la storia della libreria cresce.

4.3.2 Aggiornamenti sicuri: l'aggancio alla versione stabile

In una libreria condivisa una modifica raggiunge subito tutti i repository, anche quando è sbagliata. Finché le *pipeline* caricavano la libreria dal branch `master`, ogni *commit* sulla libreria entrava in produzione nello stesso istante in tutti i 18 repository.

Il rischio si è concretizzato durante lo sviluppo del *gate* di CI (capitolo 6). Una versione intermedia del *gate* usava la cache di Docker, che riempiva la memoria dello step su Bitbucket: la *pipeline* entrava in un ciclo e restava in esecuzione per ore, finché l'uso della cache non veniva interrotto. Il difetto è stato poi corretto, ma ha

mostrato un limite del caricamento da `master`. Senza un modo per indicare una versione della libreria come stabile, ogni prova di una nuova funzione finiva anche nelle *pipeline* già funzionanti degli altri repository.

La soluzione è nella seconda e terza riga del listato 4.1, dove la *pipeline* usa l'ultimo tag stabile della libreria, nella forma `X.Y.Z`, al posto di `master`. I tag di *release candidate* e quelli dei branch di lavoro vengono esclusi dall'espressione regolare, e l'ordinamento è per versione e non alfabetico (`--sort=-v:refname`), così che `1.10.0` segua correttamente `1.9.0`. Il valore `master` resta solo come ripiego, nel caso in cui la libreria non abbia ancora alcun tag stabile.

L'effetto è che unire una modifica su `master` della libreria non cambia nulla per i repository, perché a distribuire la modifica è la pubblicazione di un nuovo tag stabile. Fra le due operazioni c'è lo spazio per provare la modifica. La libreria, del resto, si versiona con le proprie regole: la sua *pipeline* usa le stesse funzioni di calcolo della versione che usano i repository applicativi, ed è quindi il primo consumatore di sé stessa. La stessa *pipeline* esegue un *self-test*: costruisce un repository Git usa e getta con un piccolo progetto Rust ed esegue in sequenza tutte le funzioni della libreria, in 44 scenari che riproducono i casi d'uso reali, dalla prima versione di un repository al merge di due *feature* parallele. Il *self-test* mostra l'output che un repository vedrebbe in ciascun caso, ma non confronta automaticamente il risultato con quello atteso; la verifica resta affidata a chi legge il *log*, un limite ripreso nella sezione 8.6.

Per accedere alla libreria le *pipeline* usano HTTPS con un *token* e non SSH. Una chiave SSH andrebbe configurata in ciascuno dei 18 repository; il *token* è invece una variabile definita una volta a livello di *workspace* e disponibile ovunque, e dà accesso in sola lettura al solo repository della libreria.

4.3.3 Compatibilità all'indietro

Una libreria usata da 18 repository va trattata come un'interfaccia pubblica, perché i nomi delle sue funzioni e delle variabili che esporta sono un contratto con chi la usa. Il caso più concreto si è presentato quando la variabile `CARGO_VER`, che contiene la versione del pacchetto, è stata rinominata in `PKG_VER`. Il vecchio nome era legato a Rust, mentre la stessa variabile serve anche ai pacchetti dei repository Go. La rinomina era corretta, ma tre repository usavano ancora il vecchio nome nei propri file YAML, e le loro *pipeline* si sarebbero interrotte al primo aggiornamento della libreria. La libreria esporta quindi entrambi i nomi, con lo stesso valore (listato 4.2):

```
export PKG_VER DEB_REVISION ENVIRONMENT DEB_VERSION RAW_VERSION

```

```
# BACKWARD COMPATIBILITY for downstream repos
export CARGO_VER="${PKG_VER}"
```

Listato 4.2: Il vecchio nome della variabile resta disponibile per i repository che lo usano.

Una riga di codice evita una modifica coordinata a più repository, e permette di aggiornarli uno alla volta, quando capita di lavorarci.

4.4 Passaggio di stato fra step

In Bitbucket ogni step di una *pipeline* viene eseguito in un container nuovo, quindi le variabili impostate in uno step non esistono nello step successivo. Lo step che calcola la versione deve però comunicarla a quelli che creano il tag e costruiscono gli artefatti. Le *pipeline* pilota risolvevano il problema scrivendo lo stato su file e dichiarando questi file come artefatti, che Bitbucket copia da uno step ai successivi. La libreria ha mantenuto il meccanismo, che ha un vantaggio ulteriore: un file si legge allo stesso modo in qualsiasi immagine, mentre all'inizio del tirocinio le immagini usate dai repository erano diverse fra loro e gestivano le variabili in modo non uniforme.

I file sono due. `.version` contiene soltanto la versione calcolata. `.tag_ready` contiene invece lo stato del tag, in forma di assegnazioni Bash, così che lo step successivo possa caricarlo con un semplice `source` (listato 4.3):

```
TAG_READY=1
TAG_VERSION=1.4.0-rc2
MANUAL_TAG_REQUIRED=0
```

Listato 4.3: Contenuto di `.tag_ready` per un *commit* che porta già un tag.

Rispetto alle *pipeline* pilota la libreria ha aggiunto la terza variabile. Quando il calcolo della versione non può decidere e chiede un tag manuale (sezione 5.6), è questo file a comunicarlo allo step che crea il tag, che altrimenti ne inventerebbe uno.

4.5 Pipeline in due tempi

Un tag Git, una volta pubblicato, avvia in Bitbucket una *pipeline* propria, distinta da quella del branch. Le *pipeline* sono quindi organizzate in due tempi, come mostra la figura 4.2. La *pipeline* del branch verifica il codice (capitolo 6), calcola la versione e crea il tag; la *pipeline* del tag, avviata dalla sua pubblicazione, determina l'ambiente di destinazione, costruisce gli artefatti e li pubblica.

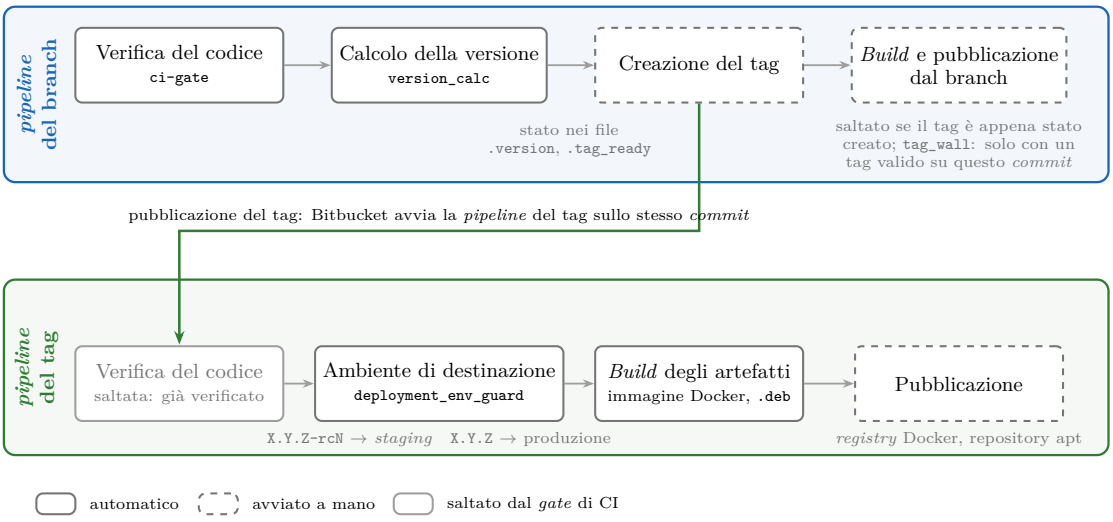


Figura 4.2: Flusso di una *pipeline*: la *pipeline* del branch calcola la versione e crea il tag, la *pipeline* del tag costruisce e pubblica gli artefatti.

Il vantaggio della separazione è che ciò che viene pubblicato è sempre legato a un tag, e dato un artefatto si risale con certezza al codice da cui è stato costruito. La conseguenza è che lo stesso *commit* attraversa due *pipeline*, e alcune operazioni rischiano di essere eseguite due volte. Tre funzioni di `pipeline-guards.sh`, anche queste ereditate dalle *pipeline* pilota e rese comuni, evitano il problema:

- `init_tag_guard` carica lo stato da `.tag_ready`;
- `skip_if_tag_just_created` interrompe con successo uno step del branch se il tag è appena stato creato: la *build* la farà la *pipeline* del tag;
- `tag_wall` protegge gli step di pubblicazione avviati a mano dalla *pipeline* del branch: prosegue solo se il *commit* porta un tag valido, se quel tag punta proprio a questo *commit* e se non è appena stato creato. Un artefatto non può così essere pubblicato con la versione di un *commit* diverso.

Il codice delle tre funzioni è nel listato A.2.

Le guardie servono anche perché alcuni passaggi restano manuali. In alcuni casi la creazione del tag e la pubblicazione vengono avviate da una persona. Automatizzarle sarebbe stato possibile, ma il team ha preferito che uno sviluppatore potesse dare un'ultima occhiata quando serve, e decidere quando una versione è pronta. Nelle *pipeline* pilota l'ordine di questi passaggi era affidato a chi li eseguiva (sezione 2.2.4); con le guardie, avviarli nell'ordine sbagliato produce al massimo un'uscita senza effetti.

Lo step che calcola la versione si riduce invece a una sola chiamata, `version_calc`, che legge l'eventuale tag già presente sul *commit*, calcola altrimenti la versione successiva (capitolo 5), riallinea `Cargo.toml` se esiste e scrive i due file di stato. Nella *pipeline* pilota del *gateway* MQTT lo stesso step occupava 108 righe di YAML; dopo la migrazione ne occupa meno di 15, ed è identico in tutti i repository. Una *pipeline* completa dopo la migrazione è riportata nel listato A.5.

4.6 Produzione degli artefatti

All'inizio del tirocinio ogni repository costruiva e pubblicava i propri artefatti a modo suo (sezione 2.2.4). La standardizzazione ha seguito una divisione dei compiti precisa. Alla libreria spettano le decisioni: quale versione porta l'artefatto, in quale ambiente va pubblicato, come si traduce il tag nella versione di un pacchetto Debian (sezione 5.7). Al file YAML restano i comandi di *build*, che dipendono dal progetto (un `Dockerfile`, il nome dell'immagine, il nome del pacchetto), ma seguono in tutti i repository lo stesso schema.

L'ambiente si ricava dalla forma del tag: una versione stabile `X.Y.Z` va in produzione, una *release candidate* o un tag di lavoro va in *staging*. Le immagini Docker vengono pubblicate su un *registry* privato con due tag, uno fisso per versione (`staging-1.4.0-rc2`) e uno mobile che indica l'ultima immagine dell'ambiente (`staging-latest`). I pacchetti `.deb` vengono pubblicati in un repository apt ospitato su Amazon S3, da cui i dispositivi presso i clienti li installano, e conservati anche fra i *download* di Bitbucket.

4.7 Un output leggibile

Quando una *pipeline* fallisce, il *log* è l'unico strumento per capire perché. Nelle *pipeline* di partenza l'output dei comandi si mescolava senza struttura (sezione 2.2.4). Il modulo `log-utils.sh` impone un formato comune a tutte le funzioni della libreria, con lo scopo dichiarato di rendere il *log* comprensibile anche a chi non ha mai modificato una *pipeline*.

Ogni funzione si apre con `log_step`, che stampa una descrizione dell'operazione seguita dal file e dalla funzione da cui è stata chiamata. Questi ultimi non vengono passati a mano, ma ricavati dalle variabili con cui Bash descrive la pila delle chiamate (listato 4.4):

```
1 log_step() {
```

```
    local desc="$1"
    local caller_file caller_fn
    caller_file="$(basename "${BASH_SOURCE[1]:-unknown}")"
    caller_fn="${FUNCNAME[1]:-unknown}"
    printf '\n%s▷ %s%s %s(%s::%s)%s\n' \
        "${_LOG_BOLD}${_LOG_BLUE}" "$desc" "${_LOG_RESET}" \
        "${_LOG_DIM}" "$caller_file" "$caller_fn" "${_LOG_RESET}"
}
```

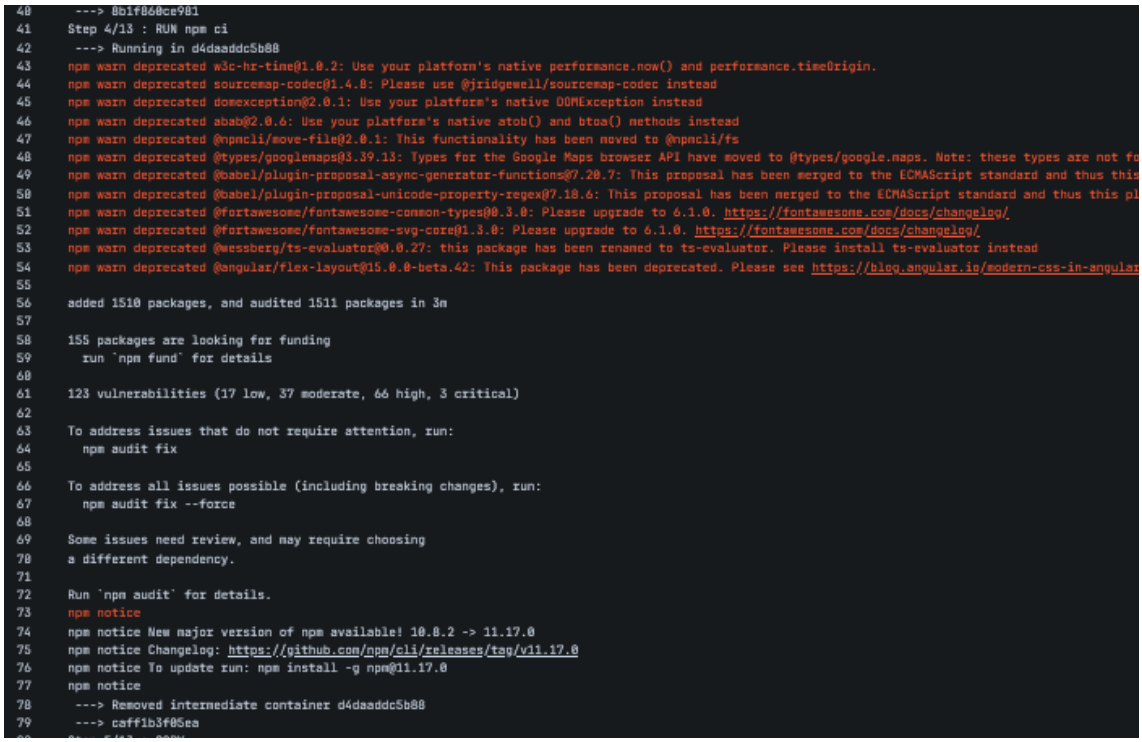
Listato 4.4: `log_step` ricava da sola file e funzione chiamante.

Gli esiti hanno un simbolo e un colore fissi: riuscito, avviso, errore, operazione saltata. I colori vengono attivati soltanto quando l'output è un terminale oppure quando lo script gira in Bitbucket, il cui visualizzatore li interpreta; in tutti gli altri casi, per esempio se l'output viene salvato su file, i codici di colore sporcherebbero il testo e vengono omessi. Due funzioni completano il modulo. `run_quiet` esegue i comandi più verbosi, come un'installazione di pacchetti, e ne mostra l'output soltanto se falliscono; in caso di successo stampa una sola riga. `log_manual_tag` stampa il riquadro con cui la *pipeline* chiede un tag manuale (sezione 5.6). Poiché in quel caso la *build* resta verde, il messaggio deve essere abbastanza evidente da non passare inosservato.

La figura 4.3 mette a confronto i due formati. Nel *log* di partenza (figura 4.3a), la *build* dell'immagine di un'applicazione web, l'output di Docker e di NPM scorre senza interruzioni. Gli avvisi sulle dipendenze, in rosso, hanno lo stesso aspetto di un errore, e nulla indica a quale fase della *pipeline* appartenga una riga. Nel *log* prodotto dalla libreria (figura 4.3b) ogni operazione si apre con una riga che ne descrive lo scopo e ne indica l'origine, per esempio `ci-gate.sh::check_if_ci_needed`; le installazioni degli strumenti occupano una riga ciascuna; le decisioni sono scritte in chiaro (la *build* precedente è fallita, quindi la verifica va eseguita), e un errore come il controllo di formattazione fallito mostra la differenza da correggere seguita da un esito che non si può confondere con un avviso.

4.8 Esito

Al termine del tirocinio tutti i 18 repository applicativi usano la libreria; 16 la caricano attraverso `bootstrap.sh`, agganciati all'ultima versione stabile, mentre 2 caricano ancora i moduli uno per uno da `master`, secondo il meccanismo delle prime versioni. La logica comune risiede in un solo repository, e una correzione si scrive una volta e raggiunge tutti i repository con la pubblicazione di un tag, invece di



(a) Prima: la *build* dell'immagine Docker di un'applicazione web, con l'output di Docker e NPM non filtrato.



(b) Dopo: l'output della libreria durante il suo *self-test*, su un progetto Rust di prova.

Figura 4.3: Lo stesso tipo di informazione prima e dopo l'introduzione di `log-utils.sh`.

richiedere 18 modifiche separate. È quanto chiedevano i requisiti **R1** e, per i 16 repository agganciati alla versione stabile, **R2**; la verifica è nel capitolo 8.

Capitolo 5

Versionamento automatico

Fra le funzioni della libreria, il calcolo della versione è quella che ha richiesto più iterazioni. La prima implementazione si limitava a leggere un numero e a incrementarlo; quella attuale deduce la versione dalla posizione del *commit* nel flusso dei branch e dalla provenienza dei merge. In mezzo ci sono tre riscritture, ognuna motivata da un caso che la precedente gestiva male. Questo capitolo descrive il modello finale, indicato nel codice come V4, e le ragioni che hanno portato a ciascuna delle sue regole.

5.1 Il tag come fonte di verità

Nella prima generazione di *pipeline* la versione di un artefatto veniva letta da `Cargo.toml`, aggiornato a mano e non sempre (sezione 2.2.3). Nelle quattro *pipeline* pilota un collega aveva già spostato l'autorità sul tag Git, e la libreria ha adottato questo principio per tutti i repository. Un tag è disponibile in qualsiasi repository indipendentemente dal linguaggio, per convenzione non viene più spostato una volta pubblicato e Bitbucket può avviare una *pipeline* alla sua creazione, cosa che rende possibile il flusso in due tempi descritto nella sezione 4.5. Con il tag come fonte di verità, lo stesso calcolo vale anche per i repository Go, Python e TypeScript, che non hanno un `Cargo.toml`.

I file di manifesto diventano una conseguenza del tag. Nei progetti Rust la *pipeline* allinea `Cargo.toml` e `Cargo.lock` alla versione calcolata e registra la modifica con un *commit* marcato `[skip ci]`, così da non innescare un'altra esecuzione. Anche questo meccanismo era presente nelle *pipeline* pilota, dove però `Cargo.lock` veniva modificato con SED, cercando a mano la riga della versione. Nella libreria si usa invece `cargo fetch`, che lascia a CARGO il compito di aggiornare il *lockfile* e lo tocca solo quanto basta a registrare la nuova versione del pacchetto locale. Non si usa

`cargo update`, che aggiornerebbe anche le dipendenze di terze parti, rendendo la *build* diversa da quella provata.

Sono previste due eccezioni. La prima, ereditata dalle *pipeline* pilota: se `Cargo.toml` riporta una versione più alta di quella calcolata, prevale il file, perché è il segnale che qualcuno l'ha modificato di proposito, e la *pipeline* si adegua invece di tornare indietro. La seconda è stata introdotta con la libreria: su `master` non viene fatto alcun *commit* automatico. Come concordato con il team di sviluppo, `master` contiene soltanto il codice di produzione e riceve esclusivamente merge, e non vi si scrive direttamente, nemmeno da parte della *pipeline*. Il suo compito è solo rendere stabile una versione già provata, togliendo il suffisso `-rc`, e la versione nei file di manifesto è già stata scritta nei branch da cui il codice arriva.

Resta il caso di un repository che non ha ancora alcun tag. Le *pipeline* pilota si fermavano con un errore; la prima versione della libreria partiva invece da `1.0.0-rc0`, una *release candidate* e non una versione stabile, perché nessun branch di lavoro produce direttamente una versione di produzione. Il modello finale (sezione 5.2.4) chiede invece di creare a mano il primo tag stabile su `master`. I branch di lavoro si numerano a partire da quello, e il numero da cui parte un prodotto lo sceglie il team.

5.2 Evoluzione del modello

Il modello di versionamento è stato riscritto tre volte nel corso del tirocinio. Ripercorrere le versioni precedenti serve a capire perché quella finale ha le regole che ha.

Le figure di questa sezione applicano le quattro versioni allo stesso scenario. `master` parte dalla release `1.0.0`; da `develop` nascono `feature/auth`, che riceve due *commit*, `bugfix/typo` e `feature/login`; i tre branch vengono integrati in `develop` in quest'ordine e `develop` viene poi rilasciato su `master`. Lo scenario è identico in tutte le figure e cambiano soltanto i tag prodotti, e quindi le differenze fra una versione e l'altra si leggono direttamente dal confronto.

5.2.1 V1: un contatore unico

La prima versione è quella rimasta al termine della standardizzazione delle *pipeline*. Il calcolatore partiva dall'ultimo tag creato nel repository, qualunque fosse il branch che l'aveva prodotto, e applicava una regola per tipo di branch: su `develop` PATCH+1 con `-rc0`, su `master` MINOR+1 con PATCH azzerata, su ogni altro branch RC+1. Nella prima stesura anche `master` produceva una *release candidate*, che la *pipeline*

instradava in staging; dalla versione `1.2.0` della libreria `master` produce una versione stabile.

Il limite principale è visibile nella figura 5.1. I quattro *commit* sui tre branch di lavoro producono `1.0.1-rc1`, `rc2`, `rc3` e `rc4`, e dal tag non si capisce quale branch abbia generato quale *release candidate*, e il numero ottenuto da un branch dipende da quanti *commit* hanno fatto gli altri nel frattempo. Inoltre ogni *commit* su `develop` consuma una PATCH: tre integrazioni portano `develop` a `1.0.4-rc0`, dopodiché `master` pubblica `1.1.0`, un numero che in staging non è mai comparso.

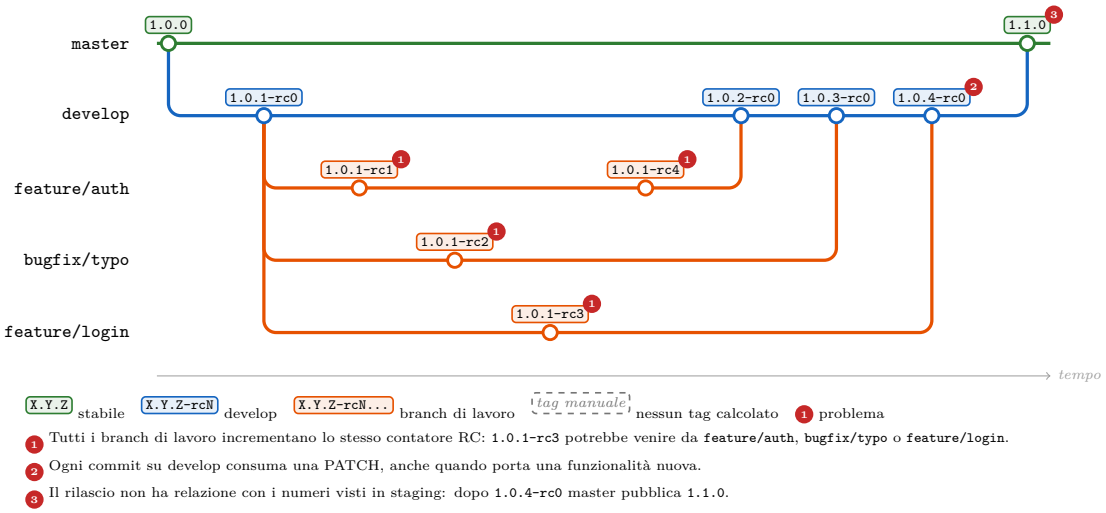


Figura 5.1: Tag prodotti da V1. I branch di lavoro condividono un solo contatore RC, `develop` avanza di una PATCH a ogni *commit* e il rilascio su `master` non corrisponde ad alcuna versione provata in staging.

5.2.2 V2: il ciclo di `develop`

La seconda versione ha legato il numero di rilascio a `develop`. Al primo *commit* `develop` apre un ciclo a MINOR+1 dall'ultima versione stabile (`1.1.0-rc0`); l'integrazione di una *feature* incrementa la PATCH e riporta l'RC a zero, l'integrazione di un *bugfix* e i *commit* diretti incrementano soltanto l'RC. `master` non calcola più nulla: pubblica il candidato di `develop` togliendo il suffisso `-rcN`. Nella figura 5.2 il rilascio è `1.1.2`, lo stesso numero provato in staging come `1.1.2-rc0`.

I branch di lavoro, però, continuavano a condividere un solo contatore RC sulla base di `develop`: i quattro *commit* producono `1.1.0-rc1` … `1.1.0-rc4`, e con più branch aperti insieme non era possibile stabilire quale branch stesse provando una determinata *release candidate*. A questo si aggiungevano due incongruenze rispetto al Semantic Versioning [16]: una *feature* integrata incrementava la PATCH anziché

la MINOR, e un *bugfix* non lasciava traccia nel numero. Per questi motivi V2 non è mai stata pubblicata. È stata provata soltanto in locale, e nessuna versione rilasciata della libreria la contiene. Ha comunque fissato il principio, ripreso da tutte le versioni successive, che il numero di rilascio si decide su `develop`.

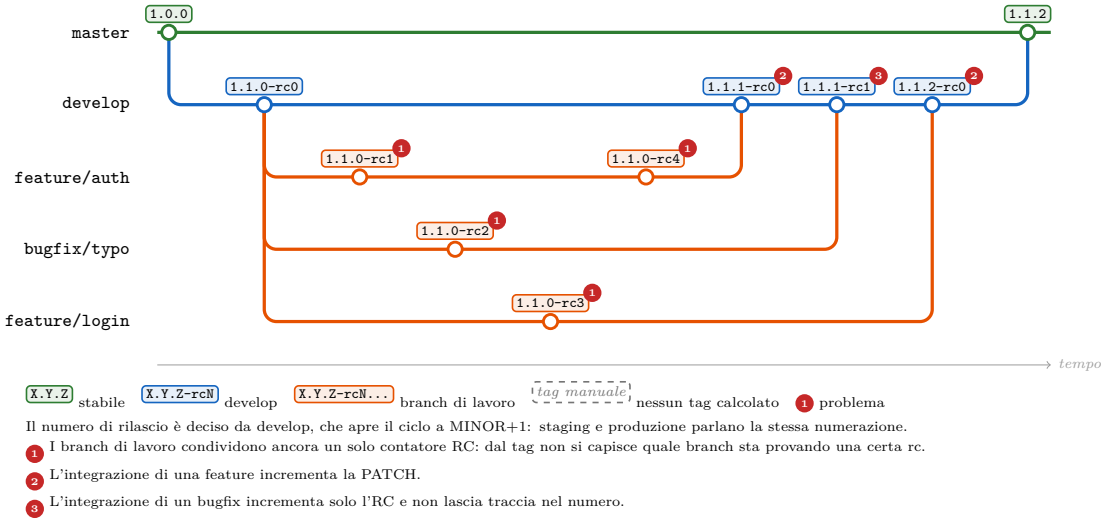


Figura 5.2: Tag prodotti da V2. `develop` apre il ciclo a `1.1.0` e decide il numero di rilascio, ma i branch di lavoro condividono ancora lo stesso contatore RC.

5.2.3 V3: un contatore per branch

La terza versione, pubblicata con la versione `1.14.0` della libreria, ha separato i tag dei branch di lavoro da quelli di `develop`. Sono nate le tre famiglie di tag descritte nella sezione 5.4: i branch di lavoro producono tag nella forma `X.Y.Z-rcN.nome`, con un contatore RC proprio e un suffisso ricavato dal nome del branch. Le regole di `develop` e di `master` sono rimaste quelle di V2.

Nella figura 5.3 ogni tag identifica il branch che lo ha prodotto: `1.1.0-rc1.auth` può venire solo da `feature/auth`. Restavano due limiti. Il primo è che tutti i branch di lavoro prendevano la base di `develop` (`1.1.0`), che non era il numero con cui sarebbero stati rilasciati; `feature/login`, per esempio, esce in `1.1.2`, ma dal numero del branch non si poteva saperlo. Il secondo è che gli incrementi erano quelli di V2, con la *feature* che incrementa la PATCH e il *bugfix* che incrementa solo l'RC. Infine, un branch di lavoro non poteva essere numerato se `develop` non aveva ancora un tag: in quel caso il primo *commit* falliva.

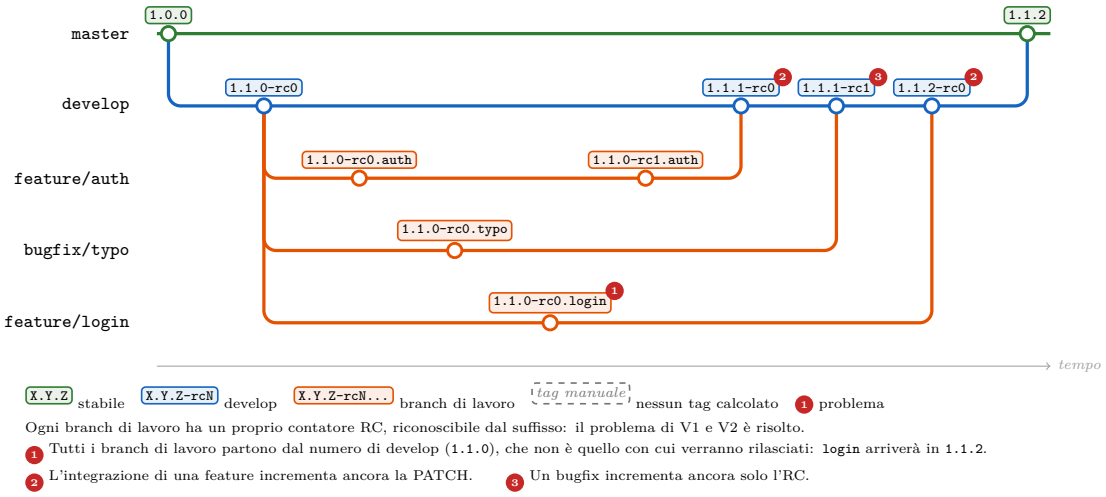


Figura 5.3: Tag prodotti da V3. Ogni branch di lavoro ha un proprio contatore, riconoscibile dal suffisso, ma parte dalla base di `develop` invece che dal numero con cui verrà rilasciato.

5.2.4 V4: branch paralleli e `develop` di appoggio

La versione attuale, pubblicata con la versione `1.15.2` della libreria, ha spostato il calcolo sui branch di lavoro, che si numerano partendo soltanto dall'ultima versione stabile: MINOR+1 per una *feature*, PATCH+1 per un *bugfix* o un *hotfix*. Gli altri branch aperti vengono ignorati, e il separatore del suffisso è diventato un trattino (`X.Y.Z-rcN-nome`). `develop` non apre più un ciclo proprio: adotta il numero del primo branch integrato e da lì sale di una MINOR per ogni *feature* e di una PATCH per ogni *bugfix*. `master` continua a pubblicare il candidato di `develop`.

Nella figura 5.4 `feature/auth` e `feature/login`, aperti in parallelo, producono entrambi `1.1.0` e si distinguono per il suffisso; `bugfix/typo` produce `1.0.1`. `develop` adotta `1.1.0` da `feature/auth`, passa a `1.1.1` con il *bugfix* e a `1.2.0` con la seconda *feature*, e `master` pubblica `1.2.0`. Un *commit* diretto su un `develop` appena creato non rientra in nessuna regola, quindi il calcolatore chiede un tag manuale (sezione 5.6) e la *build* resta verde. Le regole complete sono descritte nella sezione 5.5.

La tabella 5.1 riassume le differenze fra le quattro versioni.

Tabella 5.1: Confronto fra le quattro versioni del calcolatore.

	V1	**V2**	**V3**	**V4**
Suffisso di lavoro	`-rcN`	`-rcN`	`-rcN.nome`	`-rcN-nome`
Contatore RC	unico	unico	per branch	per branch
Base del branch	ultimo tag	ciclo di develop	ciclo di develop	ultima release
Feature	RC+1	PATCH+1	PATCH+1	MINOR+1
Bugfix	RC+1	RC+1	RC+1	PATCH+1
Numero deciso da	ogni branch	`develop`	`develop`	`develop`

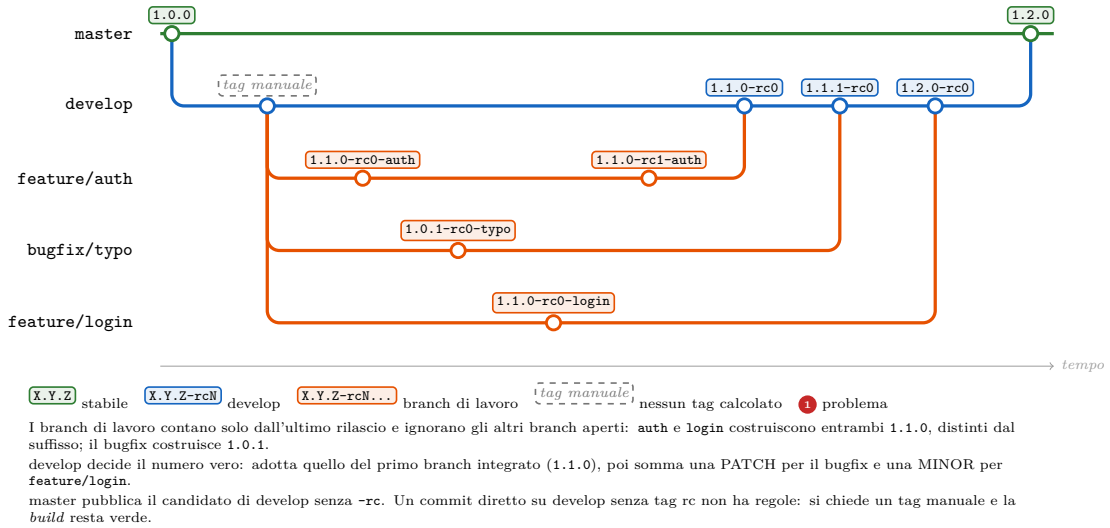


Figura 5.4: Tag prodotti da V4. I branch di lavoro si numerano dall'ultima release ignorando gli altri branch aperti; `develop` adotta il numero del primo branch integrato e poi aggiunge una MINOR per ogni *feature* e una PATCH per ogni *bugfix*.

5.3 Alternative considerate

Prima di scrivere un calcolatore dedicato esistevano strade già battute. La più diffusa ricava la versione dai messaggi dei *commit* secondo la convenzione *Conventional Commits*: un *commit* `feat:` incrementa la MINOR, un `fix:` la PATCH. Strumenti come SEMANTIC-RELEASE automatizzano l'intero processo. Un'altra è GITVERSION, che calcola la versione a partire dal modello a branch del Git flow, cioè da una struttura molto simile a quella adottata in azienda.

Nessuna delle due strade è stata adottata, per due ragioni. La prima riguarda il punto di partenza, perché i messaggi di *commit* del team non seguivano alcuna convenzione, e un calcolo basato sui messaggi avrebbe richiesto di imporne una a tutti gli sviluppatori di 18 repository prima di produrre un solo numero corretto. Il modello a branch, invece, era già in uso, e basarsi su di esso non chiedeva al team di cambiare abitudini. La seconda riguarda ciò che serviva oltre al numero: la richiesta di un tag manuale nei casi non previsti, l'allineamento di `Cargo.toml` e `Cargo.lock`, la conversione in versione Debian (sezione 5.7). Anche adottando uno strumento esterno, questi comportamenti avrebbero richiesto codice scritto attorno a esso, nella stessa libreria. Tanto valeva che anche il calcolo stesso vi fosse scritto, con regole leggibili e modificabili dal team.

5.4 Le tre famiglie di tag

Il modello V4 distingue tre famiglie di tag, riconoscibili dalla sola forma e riportate nella tabella 5.2. Le famiglie sono disgiunte, cioè un tag appartiene a una e una sola di esse, e ogni funzione del calcolatore cerca soltanto nella famiglia che le interessa.

Tabella 5.2: Le tre famiglie di tag del modello V4.

Famiglia	Forma	Prodotta su
stabile	`X.Y.Z`	`master`
develop	`X.Y.Z-rcN`	`develop`
lavoro	`X.Y.Z-rcN-nome`	`feature/*`, `bugfix/*`, `hotfix/*`

Il suffisso dei tag di lavoro deriva dal nome del branch: si prende la parte dopo la prima barra, si sostituiscono barre e trattini bassi con trattini, si eliminano gli altri caratteri, si comprimono i trattini consecutivi e si tronca a 30 caratteri. Il branch `feature/login_SSO` produce così il suffisso `login-SSO`. Il suffisso rende evidente, già dal nome dell'immagine Docker o del pacchetto, quale branch l'ha generato.

La scelta del separatore non è stata indifferente. Il Semantic Versioning [16] e la PEP 440 di Python prevedono il carattere `+` per attaccare un'etichetta a una versione, e sarebbe stata la forma più "corretta" sul piano formale. Il tag, però, non resta dentro Git: viene riusato tale e quale come tag delle immagini Docker (per esempio `staging-1.4.0-rc2-login-SSO`), e Docker non ammette il `+` nei tag. Il trattino è invece valido in tutti i contesti in cui il tag finisce: Git, Docker, i nomi dei file sul repository apt e le versioni Debian, dove viene poi convertito come descritto nella sezione 5.7.

5.5 Regole di calcolo

Il calcolatore riceve il nome del branch su cui gira la *pipeline*, lo classifica in uno dei cinque tipi previsti (`master`, `develop`, `feature`, `bugfix`, `hotfix`; `main` e `dev` sono accettati come alias) e applica le regole di quel tipo. Per `master` e `develop` conta anche se il *commit* è un merge e da quale branch proviene. Qualsiasi altro branch porta alla richiesta di un tag manuale (sezione 5.6). Il flusso completo, con due *feature* parallele e un *bugfix*, è quello già mostrato nella figura 5.4. Le funzioni principali del calcolatore sono riportate nel listato A.4.

5.5.1 I branch di lavoro

Un branch di lavoro calcola il proprio numero partendo soltanto dall'ultima versione stabile: MINOR+1 per un `feature`, PATCH+1 per un `bugfix` o un `hotfix`. Gli altri branch aperti vengono ignorati di proposito. Due *feature* nate dalla stessa release producono quindi lo stesso numero, per esempio `1.1.0`, e si distinguono soltanto per il suffisso. Questo basta a evitare collisioni, perché il suffisso fa parte del tag, e permette a ogni branch di numerarsi senza conoscere lo stato degli altri.

Per `feature` e `bugfix` il numero è provvisorio: quello definitivo verrà deciso all'ingresso in `develop`. Per un `hotfix` è invece quello reale, perché l'`hotfix` va direttamente su `master`.

La base si calcola una volta sola, al primo tag del branch, e poi resta fissa. Per ritrovarla il calcolatore cerca, fra i tag raggiungibili dal *commit* corrente, il più alto con lo stesso suffisso; se la sua base è ancora più avanti dell'ultima release, il branch continua su quella. Così un rilascio arrivato su `master` mentre il branch è aperto non ne sposta il numero. Il controllo sulla release serve anche a un secondo caso: un nome di branch già usato in passato per lavoro poi rilasciato non eredita una base ormai superata, e riparte da capo. Il contatore delle rc è legato alla coppia formata da base e suffisso.

5.5.2 `develop` decide il numero

`develop` è il punto in cui il numero provvisorio dei branch diventa definitivo. Il numero portato dal branch non viene usato: conta soltanto il fatto che il branch sia arrivato, e di che tipo sia. `develop` parte dalla base su cui si trova e sale di una MINOR per ogni *feature* e di una PATCH per ogni *bugfix*. Le regole complete sono nella tabella 5.3.

Tabella 5.3: Calcolo della versione su `develop`.

Evento	**Versione successiva**
primo branch del ciclo	la base del branch, `rc0`
merge da `feature/*`	MINOR+1 sulla base di develop, `rc0`
merge da `bugfix/*`	PATCH+1 sulla base di develop, `rc0`
merge di un branch già entrato nel ciclo	RC+1
commit diretto	RC+1
commit diretto senza alcun tag su develop	tag manuale
merge da altri branch o di origine illeggibile	tag manuale

Un esempio chiarisce il meccanismo. Con `master` a `1.0.0` e due *feature* aperte, entrambi i branch producono `1.1.0` con il proprio suffisso. Se entra per prima

`feature/export`, `develop` diventa `1.1.0-rc0`; quando entra `feature/auth`, `develop` passa a `1.2.0-rc0`, anche se il branch aveva costruito `1.1.0-rc3-auth`. L'ordine di ingresso decide chi prende quale numero, e nessuno dei due branch ha dovuto saperlo in anticipo.

Un `develop` appena staccato da `master` non ha ancora tag `rc`, e quindi nessuna base da cui contare. Non ne calcola una propria: resta senza tag finché non entra il primo branch, che apre il ciclo con il proprio numero. In questo senso `develop` è un branch di appoggio, che segue i branch di lavoro senza aprire cicli da solo.

Due controlli evitano che il conteggio vada fuori strada. Il primo riguarda un branch che rientra in `develop` una seconda volta (per esempio dopo una correzione emersa in collaudo): se fra i tag raggiungibili da `develop` prima del merge ce n'è già uno con lo stesso suffisso e con una base non ancora rilasciata, il branch fa già parte del ciclo aperto, e il merge produce soltanto una nuova rc invece di consumare un altro numero. Il secondo riguarda un rilascio avvenuto su `master` mentre `develop` lavorava, tipicamente un `hotfix`: se il numero calcolato non è più avanti dell'ultima release, il conteggio riparte dalla release stessa, perché una candidata per una versione già pubblicata non ha senso. Infine, se un branch porta un numero più alto di quello che `develop` calcolerebbe (succede quando è stato staccato da un `master` più recente), viene tenuto il numero del branch, per non mettere `develop` dietro a codice che già contiene. Questi controlli derivano da casi individuati in fase di progettazione, ripercorrendo il flusso dei branch e chiedendosi, per ogni passaggio, che cosa sarebbe successo al numero.

5.5.3 `master` pubblica

`master` non calcola nulla e si limita a pubblicare il numero che i branch di lavoro hanno già stabilito passando da `develop`. Quando riceve un merge da `develop`, la release è la *release candidate* di `develop` privata della parte `-rc`: `1.3.0-rc4` diventa `1.3.0`. Quando riceve un `hotfix`, la release è la candidata dell'`hotfix` privata di rc e suffisso: `1.1.1-rc2-crash` diventa `1.1.1`. Un *commit* diretto su `master`, o un merge da qualsiasi altro branch, porta a una richiesta di tag manuale.

La candidata va letta sull'ultimo *commit* del branch unito, e non presa come il tag più alto della sua storia. Le due cose coincidono finché il branch ha una sola linea di sviluppo, ma divergono appena ne ha più d'una.

Un branch porta con sé più linee appena vi è stato unito un altro branch con tag propri, e in quel caso il tag più alto della sua storia può essere un numero da cui il branch si è già spostato: pubblicandolo, `master` rilascerebbe una versione diversa da quella provata in *staging*. Il tag sull'ultimo *commit* è invece, per costruzione, la

candidata che è stata provata. Se l'ultimo *commit* non ha alcun tag, il calcolatore ripiega sul tag più alto e lo segnala nel *log*.

Prima di pubblicare, il calcolatore verifica che la versione sia più alta dell'ultima release. Se non lo è, significa che `develop` era rimasto indietro rispetto a `master`, e anche in questo caso la decisione passa allo sviluppatore.

5.5.4 L'origine del merge

Tutto il modello si regge sul sapere da quale branch proviene il *commit* che si sta versionando. Git non la conserva in modo esplicito, ma la lascia in due posti. Il primo è la struttura del *commit*, perché un merge ha due genitori, e il secondo è l'ultimo *commit* del branch unito. Da lì il calcolatore legge i tag che il branch portava con sé. Il secondo è il messaggio del merge, che Bitbucket compone in modo prevedibile. La funzione nel listato 5.1 riconosce la forma usata da Bitbucket e altre quattro forme meno frequenti.

```
_merge_source_branch(){
    local subject
    subject="$(git log -1 --format=%s HEAD 2>/dev/null || true)"

    # Bitbucket pull request: "Merged in feature/x (pull request #12)"
    if [[ "$subject" =~ ^Merged\ in\ ([^[:space:]]+)\ \(pull\ request\ #
    ↪ [0-9]+\)$ ]]; then
        echo "${BASH_REMATCH[1]}"
        return 0
    fi
    # git default: "Merge branch 'feature/x' into develop"
    if [[ "$subject" =~ ^Merge\ (remote-tracking\ )?branch\ \'([^\']+)\' ]];
    ↪ then
        local b="${BASH_REMATCH[2]}"
        echo "${b#origin/}"
        return 0
    fi
    # ... altre tre forme ...
    echo ""
}
```

Listato 5.1: Estrazione del branch di origine dal messaggio del merge commit (estratto).

Questa scelta impone al team un vincolo, cioè che i merge siano merge *commit* veri. Uno *squash* o un *fast-forward* producono un *commit* con un solo genitore e un

messaggio qualsiasi, cioè cancellano proprio l'informazione su cui il calcolo si basa. In quel caso il calcolatore non prova a indovinare e chiede un tag manuale.

Il vincolo è stato presentato al team a voce, spiegando che cosa avrebbe perso il calcolo con uno *squash*, ed è riportato nel README della libreria, così che chi arriva dopo lo trovi accanto alle regole che lo motivano. Il team lo ha accettato, perché era la condizione perché branch aperti in parallelo nello stesso repository non entrassero in conflitto sul numero di versione.

5.6 Quando decide lo sviluppatore

Le regole coprono i flussi che il team usa ogni giorno. Tutto il resto (un *commit* diretto su `master`, un merge da un branch con un prefisso non previsto, un messaggio di merge illeggibile, una versione calcolata che esiste già) finisce nello stesso punto. La *pipeline* non crea alcun tag, stampa nel *log* un riquadro ben visibile con il motivo e il comando da eseguire, e termina con successo.

La scelta di lasciare la *build* verde è deliberata. In questi casi i controlli di qualità sono stati eseguiti e superati; il codice è valido, manca soltanto una decisione sul numero che le regole non sono in grado di prendere. Segnare la *build* come fallita avrebbe confuso un problema di codice con una scelta che spetta a una persona. Lo sviluppatore risponde creando il tag a mano (listato 5.2):

```
git tag 1.4.0 && git push origin 1.4.0
```

Listato 5.2: Risposta a una richiesta di tag manuale.

Un tag già presente sull'ultimo *commit* prevale sempre sul calcolo, e la *pipeline* successiva lo trova e prosegue, senza ripetere la richiesta.

Una regola simile vale per la PATCH, che non passa mai da sola alla MINOR. Quando supera la soglia di 10 la *pipeline* continua a creare i tag ma avvisa che potrebbe essere il momento di chiudere la linea con una MINOR manuale; la soglia si può cambiare per repository. Un limite fisso non avrebbe senso, perché in progetti maturi è normale arrivare a decine, a volte centinaia, di rilasci di sola correzione.

5.7 Versioni e pacchetti Debian

Un tag Git e una versione Debian non seguono la stessa grammatica, e la differenza è emersa in modo concreto. Su una delle macchine che installano i pacchetti prodotti dalla *pipeline*, `apt list` mostrava una versione diversa da quella attesa. Dopo la

pubblicazione della versione stabile, la macchina restava sulla *release candidate* che l'aveva preceduta, perché APT considerava quest'ultima più recente. L'ordine delle versioni, cioè, non era quello che il tag Git lasciava intendere.

La causa sta nel modo in cui DPKG ordina le versioni (sezione 3.6). Tutto ciò che segue l'ultimo trattino viene interpretato come revisione del pacchetto, e una versione senza revisione equivale a una con revisione `0`. `1.3.4-rc5` viene quindi letta come versione `1.3.4` con revisione `rc5`, e `1.3.4` come la stessa versione con revisione `0`, che nell'ordinamento di DPKG precede `rc5`. Di conseguenza `1.3.4-rc5` risulta *più recente* di `1.3.4`, e un dispositivo che ha installato la candidata non passerà mai alla versione stabile. Il carattere `~` è l'unico che DPKG ordina prima della stringa vuota, ed è quindi quello che serve per un pre-rilascio: `1.3.4~rc5` precede correttamente `1.3.4`. La conversione dal tag Git alla versione Debian è nel listato 5.3.

```
deb_version_from_tag(){
    local tag="$1"
    if [[ "$tag" =~ ^(.+)-rc([0-9]+)[.-](.+)$ ]]; then
        # tag di lavoro: 1.3.4-rc5-auth -> 1.3.4~rc5.auth
        printf '%s~rc%s.%s' "${BASH_REMATCH[1]}" "${BASH_REMATCH[2]}" "${
    ↪ BASH_REMATCH[3]}"
    elif [[ "$tag" =~ ^(.+)-rc([0-9]+)$ ]]; then
        # candidata: 1.3.4-rc5 -> 1.3.4~rc5
        printf '%s~rc%s' "${BASH_REMATCH[1]}" "${BASH_REMATCH[2]}"
    else
        # stabile: 1.3.4 -> 1.3.4-1
        printf '%s-1' "$tag"
    fi
}
```

Listato 5.3: Conversione di un tag Git in versione Debian.

Le versioni stabili ricevono una revisione esplicita `-1`. Se servisse ricostruire un pacchetto senza cambiare il codice, per esempio per correggere un file di configurazione del pacchetto, si potrebbe pubblicare `1.3.4-2` senza consumare un numero di PATCH. Nei tag di lavoro il suffisso del branch viene separato con un punto, così che nessuna sua parte venga scambiata per una revisione. Il modulo completo, con la conversione analoga per le versioni PEP 440 dei progetti Python, è nel listato A.3.

5.8 Ordinamento dei tag

Un ultimo dettaglio, facile da sottovalutare, riguarda come si individua l'"ultima" versione. Le *pipeline* originali usavano `git describe` sul *commit* taggato più recente, cioè ordinavano per data e non per versione. Con `1.2.0` e `1.2.0-rc0` presenti nel repository veniva scelto quello sul *commit* più nuovo, e un tag posato su un *commit* vecchio poteva restituire una versione più bassa di una già rilasciata. La libreria ordina invece per versione, con l'opzione `versionsort.suffix=-rc` che fa precedere `1.2.1-rc3` a `1.2.1` come prevede il Semantic Versioning. L'elenco passa poi attraverso un'espressione regolare che lascia fuori i tag che non sono versioni, perché alcuni repository usano tag "puntatore" come `staging-europe`, che avrebbero fatto fallire il calcolo.

Il problema non è rimasto teorico. Quando un collega ha unito un branch di lavoro in `develop`, la *pipeline* gli ha proposto un tag che esisteva già. Il calcolo era partito da un "ultimo tag" individuato in modo sbagliato e, incrementandolo, aveva ottenuto un numero già usato. L'episodio ha portato a rivedere la ricerca dell'ultimo tag, che oggi avviene come descritto in questa sezione, cioè per versione invece che per data, e separatamente per ciascuna famiglia di tag (sezione 5.4). Come ulteriore protezione, prima di proporre un tag il calcolatore verifica che non esista già nel repository; se esiste, non tenta di crearlo e chiede un tag manuale (sezione 5.6).

5.9 Esito

Al termine del tirocinio il versionamento è automatico in tutti i 18 repository che usano la libreria, contro i 4 in cui la *pipeline* calcolava e creava un tag prima dell'intervento, e segue ovunque le stesse regole. I casi che le regole non coprono tornano allo sviluppatore con un'indicazione precisa di cosa fare. È quanto richiesto dal requisito **R3**; la verifica, insieme a quella degli altri requisiti, è nel capitolo 8.

Capitolo 6

Verifica automatica e rilascio sicuro

Nel capitolo 2 sono emersi due vuoti. Solo 2 repository su 18 verificavano il codice a ogni modifica (sezione 2.2.2), e nessuno aveva un controllo che impedisse a una *release candidate* di arrivare in produzione. Questo capitolo descrive come la libreria li ha colmati, con un *gate* di CI comune a tutti i repository, che decide da solo se e come verificare il codice, e con una guardia che controlla l'ambiente di destinazione prima di ogni pubblicazione. Nel primo caso il vincolo principale è il costo, perché in Bitbucket i minuti di esecuzione si pagano; nel secondo è l'affidabilità, perché un errore arriva fino ai dispositivi dei clienti.

6.1 Il CI gate

Le due *pipeline* pilota che eseguivano dei controlli, quella del *gateway* MQTT e quella del gestore dei flussi di test, avevano uno step chiamato *CI gate*, che eseguiva formattazione con RUSTFMT, analisi statica con CLIPPY e test solo se il *commit* modificava i sorgenti Rust. Lo step era scritto per Rust e copiato a mano, e non poteva essere esteso agli altri 16 repository così com'era.

La libreria ne ha conservato l'idea e il nome, trasformandolo in una funzione, `check_ci_gate`, uguale per tutti i repository. Nel file YAML lo step si riduce a una riga (listato 6.1):

```
- step: &ci-gate
    name: "CI: fmt + clippy + test"
    script:
      - *utils
      - check_ci_gate
```

Listato 6.1: Lo step di verifica nella pipeline di un repository.

La funzione fa due cose, in quest'ordine. Prima decide se la verifica serve (sezione 6.2); se serve, riconosce i linguaggi presenti nel repository ed esegue i controlli adatti a ciascuno (sezione 6.3). Un repository nuovo non ha quindi nulla da configurare, perché gli basta richiamare lo step.

6.2 Eseguire la CI solo quando serve

6.2.1 Il problema

Una verifica completa di un progetto Rust compila il codice due volte (una per CLIPPY, una per i test) e può richiedere diversi minuti. Eseguirla a ogni *pipeline* sarebbe stato lo spreco più semplice da introdurre, per due motivi.

Il primo è che molti *commit* non toccano il codice. Nella cronologia Git dei 16 repository che usano il *gate*, dalla sua introduzione in poi, 284 *commit* su 398 (il 71%) non modificano alcun file sorgente. Per lo più si trattava di modifiche al file della *pipeline* stesso, frequenti durante la migrazione, e in misura minore di documentazione e file di configurazione. Ripetere i test su un codice identico a quello già verificato non aggiunge informazione.

Il secondo è strutturale, e nasce dal flusso in due tempi (sezione 4.5). Quando la *pipeline* del branch crea un tag, Bitbucket avvia la *pipeline* del tag sullo *stesso commit*, e anche questa contiene lo step di verifica. Senza un controllo apposito, ogni rilascio verificava due volte lo stesso codice.

Le *pipeline* pilota affrontavano solo il primo problema, e con un meccanismo di Bitbucket, la condizione `changesets`, che esegue uno step soltanto se sono cambiati determinati percorsi. La soluzione è stata provata anche con la libreria, e abbandonata per tre motivi.

Il primo è pratico. La condizione decide soltanto se uno step parte; per far sapere agli step successivi che cosa era stato deciso servivano variabili da passare da uno step all'altro, combinate con le ancore YAML. Il meccanismo si è rivelato fragile, perché a seconda del percorso seguito dalla *pipeline* le variabili a volte arrivavano e a volte no, e ogni passaggio intermedio era un punto in cui la catena poteva rompersi.

Il secondo riguarda la struttura dei repository. La condizione è legata al singolo step e a un elenco fisso di percorsi, mentre un repository può contenere più servizi, scritti in linguaggi diversi, come il catalogo dei test, che ha una parte in Rust e una in Go. Ogni caso avrebbe richiesto una combinazione propria di step e condizioni.

Il terzo è che la condizione non sa nulla dell'esito delle esecuzioni precedenti, quindi non riconosce né il *commit* già verificato da un'altra *pipeline*, né quello la cui verifica precedente è fallita.

Spostare la decisione in una funzione della libreria risolve tutti e tre i problemi. La decisione e i controlli avvengono nello stesso step, senza stato da passare; i linguaggi vengono riconosciuti dal contenuto del repository (sezione 6.3); e dove serve un trattamento particolare il repository può richiamare la verifica su una sola parte del progetto, per esempio entrando nella cartella di un servizio prima di chiamare `check_ci_gate`.

6.2.2 La decisione

La funzione `check_if_ci_needed` risponde a tre domande, in quest'ordine; la prima risposta affermativa decide.

1. *Lo stesso commit è già stato verificato da un'altra pipeline?* Se esiste un'altra esecuzione sullo stesso *commit* riuscita, oppure ancora in corso, la verifica viene saltata. Il caso "in corso" serve perché la *pipeline* del tag nasce da uno step di quella del branch, che quindi è ancora in esecuzione quando la seconda arriva al *gate*. Solo un'esecuzione fallita o interrotta giustifica una nuova verifica.

2. *Il commit modifica file sorgente?* Se fra i file cambiati compare un sorgente o un file di dipendenze di uno dei quattro linguaggi (`src/` e `Cargo.*` per Rust, `*.py` e `pyproject.toml` per Python, `*.go` e `go.mod` per Go, `*.ts` e `package.json` per TypeScript), la verifica si esegue.

3. *L'esecuzione precedente è fallita?* Se il *commit* non tocca i sorgenti ma l'esecuzione precedente era fallita, la verifica si ripete, perché il *commit* potrebbe aver corretto proprio la causa del fallimento (un file di configurazione, per esempio). Altrimenti viene saltata.

Le domande 1 e 3 richiedono di conoscere lo stato delle altre esecuzioni, che il *runner* non vede; la funzione lo legge dalle API REST di Bitbucket, autenticandosi con una coppia di credenziali OAuth definite a livello di *workspace*. La prima versione usava una *app password*, cioè una credenziale legata all'account di una persona; il passaggio a OAuth ha legato l'accesso al *workspace* invece che a un singolo utente.

La domanda 2 è stata estesa un po' alla volta. La prima versione riconosceva solo i sorgenti Rust, poi un collega ha aggiunto Python e Go, e TypeScript è arrivato insieme ai relativi controlli.

6.2.3 In caso di dubbio, verificare

Interrogare un servizio esterno introduce un nuovo modo di fallire. Le credenziali possono mancare, lo scambio del *token* può non andare a buon fine, l'API può

rispondere con un errore. In tutti questi casi la funzione sceglie di *eseguire* la verifica (listato 6.2):

```
if [ -z "${BB_OAUTH_KEY:-}" ] || [ -z "${BB_OAUTH_SECRET:-}" ]; then
    log_warn "BB_OAUTH_KEY/SECRET not set - running CI by default"
    return 0
fi
# ... scambio del token ...
if [ "${http_code}" != "200" ]; then
    log_warn "Bitbucket API returned HTTP ${http_code} - running CI by
    ↪ default"
    return 0
fi
```

Listato 6.2: Comportamento in caso di errore nella lettura dello stato (estratto).

È una scelta *fail-safe*, perché i due errori possibili non hanno lo stesso costo. Una verifica eseguita senza bisogno costa qualche minuto; una verifica saltata per errore lascia passare codice mai controllato. Fra i due, il *gate* accetta sempre il primo.

6.3 Controlli per linguaggio

Quando la verifica serve, `check_ci_gate` cerca nel repository i file di ciascun linguaggio ed esegue i controlli corrispondenti, riassunti nella tabella 6.1. Un repository che contiene più linguaggi, come il catalogo dei test (Rust e Go), li riceve tutti. Il linguaggio viene riconosciuto dal contenuto del repository e non da una configurazione, che andrebbe scritta e mantenuta per ciascuno dei 18 repository e dimenticata nel diciannovesimo. È quanto richiesto dal requisito **R4**.

Tabella 6.1: Controlli eseguiti dal *gate* per ciascun linguaggio.

Linguaggio	Controlli bloccanti
Rust	formattazione (RUSTFMT), analisi statica (CLIPPY), test unitari e di integrazione (CARGO-NEXTEST), poi test dell'intero *workspace* inclusi quelli marcati `#[ignore]`
Go	formattazione (GOFMT)
Python	errori gravi (PYLINT limitato a sintassi e variabili non definite), test (PYTEST)
TypeScript	controllo dei tipi (`tsc --noEmit`), test

I controlli non hanno la stessa profondità per tutti i linguaggi, e la differenza riflette il punto di partenza. Rust è il linguaggio della maggior parte dei repository e quello in cui esistevano già test; per Go e Python si è partiti dai controlli che non

potevano fallire su codice corretto ma mai verificato prima. Un PYLINT con tutte le regole attive avrebbe prodotto centinaia di segnalazioni di stile e bloccato le *pipeline* fin dal primo giorno; limitato agli errori che fanno fallire il programma, segnala solo ciò che va corretto comunque. Per lo stesso motivo un repository Python senza test non fa fallire la verifica; un PYTEST che non trova test viene trattato come un controllo saltato e non come un errore (una correzione introdotta da un collega).

6.3.1 Rust

Per Rust i test sono eseguiti con CARGO-NEXTEST invece che con il comando standard `cargo test`. Le ragioni sono tre: ogni test gira in un processo separato, quindi un test che si blocca o termina in modo anomalo non trascina con sé gli altri; i test vengono eseguiti in parallelo con una gestione più efficiente, e l'output riporta in modo leggibile quali test sono falliti e quanto è durato ciascuno.

I test passano in due fasi. La prima esegue i test unitari e di integrazione ordinari. La seconda esegue tutti i test dell'intero *workspace*, compresi quelli marcati `#[ignore]`, che per convenzione del team sono i test che richiedono servizi esterni, come un database o Docker, e per questo lo step dispone di un servizio Docker.

6.3.2 Il limite della memoria

La compilazione di Rust è il punto in cui la verifica incontra i limiti fisici dei *runner*. Gli step di verifica dei repository Rust girano per lo più in taglia `2x`, cioè con 8 GB di memoria, che però vanno divisi fra il container dello step e i servizi collegati. Con il servizio Docker configurato a 7 GB, necessario ai test che avviano container, al compilatore resta 1 GB. Le *pipeline* raccolte mostrano le conseguenze. In 13 esecuzioni di 4 repository Rust la compilazione è stata interrotta dal sistema con il segnale `SIGKILL`, quasi sempre durante la *build* delle librerie crittografiche usate dal client AWS, le più pesanti del progetto. Il *log* non dice esplicitamente che la memoria è finita, ma un processo ucciso con quel segnale in quel punto è la firma tipica di un esaurimento della memoria.

Il *gate* interviene in due modi. Limita la compilazione a due processi paralleli invece che al numero di processori, perché ogni processo di compilazione in più è memoria in più; un repository con uno step più grande può alzare il valore con una variabile. E imposta `CARGO_TERM_QUIET`, che toglie dal *log* le centinaia di righe di avanzamento della compilazione (*Downloading*, *Compiling*) lasciando intatti gli errori del compilatore e l'output dei test, che sono la parte da leggere.

Limitare il parallelismo, però, riduce il problema senza eliminarlo, e il caso più istruttivo è quello del *tunnel* M2M. Lì lo step di verifica del server ha smesso di terminare. Le esecuzioni sono state fermate a mano dopo 61 e 82 minuti, contro i 4–5 minuti abituali, e sul branch dedicato alla correzione sono servite cinque esecuzioni prima di arrivare a una verifica completa. La causa era un'opzione globale del file YAML, `docker: true`, che collega il servizio Docker a *tutti* gli step della *pipeline*, compreso quello di verifica, lasciandogli 1 GB. Il collegamento delle eseguibili dei test, che include staticamente un client HTTP, un driver di database e la libreria per i container di test, superava quel limite. La correzione ha tolto l'opzione globale, facendo dichiarare il servizio solo agli step che lo usano, e ha disattivato le informazioni di *debug* nella compilazione dei test, che sono la parte più pesante del collegamento. Con 8 GB disponibili lo step è tornato a durare circa 8 minuti. Nel *gateway* MQTT, configurato poi allo stesso modo, le metriche di Bitbucket mostrano per lo step di verifica un picco di circa 2 GB, un quarto della memoria disponibile.

6.3.3 Python e TypeScript

Per Python la verifica crea ogni volta un ambiente virtuale (*virtualenv*) in cui installa le dipendenze del progetto. Le immagini dei *runner* contengono un Python di sistema, e installare pacchetti direttamente in quello lo modificherebbe o, nelle distribuzioni recenti, verrebbe rifiutato. L'ambiente virtuale isola le dipendenze del progetto e si crea con il modulo `venv` incluso in Python; solo se manca si ricorre al gestore di pacchetti del sistema. Il supporto ai progetti gestiti con UV è stato aggiunto in seguito da un collega.

Per TypeScript, oltre ai test, il *gate* esegue il compilatore in modalità di solo controllo (`tsc --noEmit`), che verifica i tipi senza produrre file. Il controllo è stato aggiunto su richiesta del team di sviluppo: in un linguaggio in cui i tipi sono verificati solo in compilazione, è l'equivalente più vicino al controllo che per Rust fa il compilatore stesso.

6.4 Controlli che segnalano senza bloccare

Oltre ai controlli bloccanti, il *gate* esegue tre controlli che segnalano i problemi nel *log* senza far fallire la *pipeline*:

- la ricerca di vulnerabilità note nelle dipendenze Rust, con CARGO AUDIT (e CARGO DENY, se il repository lo configura);

- la misura della copertura dei test Rust, con CARGO LLVM-COV, riportata come riepilogo e senza soglia minima;
- la ricerca di vulnerabilità nei *lockfile* di qualsiasi linguaggio, con TRIVY.

La scelta di non bloccare deriva dal modo in cui i controlli sono stati introdotti: attivati in un colpo solo su 18 repository, avrebbero fatto fallire tutte le *pipeline* con una vulnerabilità già presente in una dipendenza, anche se nessuna modifica recente l'aveva introdotta. Così, invece, il problema diventa visibile senza fermare il lavoro, e ogni repository può decidere quando affrontarlo. Ciascun controllo si può disattivare con una variabile d'ambiente nel file YAML del repository, senza modificare la libreria. Sullo stesso principio un collega ha poi aggiunto la ricerca di segreti nel codice con GITLEAKS, che a differenza degli altri viene eseguita sempre, anche quando la verifica viene saltata, ed è l'unica che un repository può rendere bloccante.

6.5 Guardie sul rilascio

6.5.1 Il problema

Nella *pipeline* del tag l'ambiente di destinazione dipende dalla forma del tag (sezione 4.6), e ogni ambiente ha la sua *pipeline*. In ogni repository il file YAML definisce una *pipeline* per i tag di *release candidate*, che pubblica in *staging*, e una per i tag stabili, che pubblica in produzione. La scelta fra le due è affidata ai *pattern* con cui le *pipeline* sono dichiarate (listato 6.3):

```yaml
tags:
  "[0-9]*.[0-9]*.[0-9]*-rc[0-9]*":    # staging
    - step: ...
  "[0-9]*.[0-9]*.[0-9]*":             # produzione
    - step: ...
```

Listato 6.3: I due *pattern* delle pipeline per tag, uguali in tutti i repository.

I due *pattern* non sono disgiunti. L'asterisco finale del secondo accetta qualsiasi sequenza di caratteri, quindi anche il tag `1.2.3-rc1` corrisponde a entrambi. Bitbucket esegue una sola *pipeline* per tag, e quando più *pattern* corrispondono la documentazione indica che prevale quello più specifico [2], senza però definire in modo completo come la specificità venga stabilita; esiste inoltre un difetto noto, segnalato pubblicamente, per cui la precedenza non viene rispettata quando un

pattern ne raggruppa più d'uno. Nei commenti dei file YAML il team assumeva una regola ancora diversa, "vince il primo *pattern* dell'elenco". La correttezza del rilascio dipendeva quindi da una regola che nessuno poteva verificare leggendo il file, e un errore non avrebbe prodotto alcun messaggio, e una *release candidate* sarebbe stata pubblicata in produzione con l'esito di una *pipeline* riuscita.

Il caso non si è mai verificato. È stato individuato leggendo il codice, ripercorrendo per ciascun tipo di tag la strada che avrebbe seguito.

6.5.2 La guardia

Invece di cercare un ordine dei *pattern* sicuro, la soluzione rende il risultato indipendente da quale *pipeline* venga scelta. La funzione `deployment_env_guard` viene chiamata in ogni step di pubblicazione, subito prima della *build*, e confronta due informazioni che hanno origini diverse: l'ambiente calcolato dalla libreria a partire dal tag e il *deployment* dichiarato nello step del file YAML (in Bitbucket, per esempio, `production-europe`). I controlli sono due:

1. un tag di *release candidate* non può mai essere pubblicato in produzione, comunque sia arrivato in quello step;
2. l'ambiente calcolato deve coincidere con quello del *deployment*: se il tag dice *staging* e lo step pubblica in produzione, o viceversa, la *pipeline* si ferma.

Il listato 6.4 mostra il primo dei due controlli; la funzione completa è nel listato A.2.

```
if [ "${expected_env:-${ENVIRONMENT:-}}" = "production" ]; then
    case "${BITBUCKET_TAG:-}" in
        *-rc*)
            log_error "RC tag '${BITBUCKET_TAG}' cannot be published to
    ↪ production"
            log_info  "  a release candidate belongs in staging; check the
    ↪ order of the"
            log_info  "  tag globs in bitbucket-pipelines.yml (the rc pattern
    ↪  must come first)"
            exit 1
            ;;
    esac
fi
```

Listato 6.4: Il primo controllo di `deployment_env_guard` (estratto).

A differenza di una richiesta di tag manuale (sezione 5.6), qui la *pipeline* fallisce di proposito, perché un rilascio nell'ambiente sbagliato è esattamente l'errore che non

deve passare inosservato. Il messaggio indica la causa probabile e dove intervenire. Gli step che pubblicano i pacchetti `.deb` non dichiarano un *deployment*, quindi il secondo controllo non si può applicare; il primo vale comunque, perché è l'ambiente calcolato a scegliere il repository apt su cui il pacchetto viene pubblicato.

La guardia è stata aggiunta a tutti i 18 repository con un solo rilascio della libreria e una riga nel file YAML di ciascuno. È quanto richiesto dal requisito **R6**.

6.6 Quanto si risparmia

Il risparmio del *gate* si può misurare sulla cronologia delle esecuzioni conservata da Bitbucket. Per questa tesi sono state raccolte, in sola lettura, tutte le *pipeline* dei 18 repository e della libreria dall'inizio del 2026: 1.464 *pipeline* e 6.359 step, con durata, esito e, per gli step di verifica, l'esito del *gate* letto dal *log*. Nei 16 repository che usano il *gate*, la verifica è stata saltata 260 volte perché il *commit* non modificava sorgenti e 31 volte perché lo stesso *commit* era già stato verificato da un'altra *pipeline*, per un totale di 291 verifiche evitate, in linea con la stima di circa 350 ricavata dalla sola cronologia Git, che contava i *commit* e non i *push*.

Tabella 6.2: Durata mediana dello step di verifica nei repository Rust, con verifica completa e saltata, e minuti di esecuzione evitati.

Repository	Completa	Saltata	Evitate	Minuti evitati
servizio di identificazione	12m 44s	36s	26	314
gestore dei flussi di test	16m 10s	46s	13	190
tunnel M2M	4m 26s	31s	41	157
disponibilità dei dispositivi	11m 54s	25s	12	131
catalogo dei test	12m 04s	51s	10	112
gestore dei SUT	9m 00s	43s	11	93
gateway MQTT	5m 29s	30s	16	79
servizio di *scheduling*	6m 26s	27s	8	48
Totale Rust			**137**	**1.122**
altri 8 repository			154	79

La tabella 6.2 riporta il calcolo per i repository Rust. Per ciascuno, ogni verifica evitata vale la differenza fra la durata mediana di una verifica completa riuscita e la durata effettiva dello step saltato, che non è zero, perché anche quando salta i controlli, lo step prepara il container, clona la libreria e interroga le API, e dura in genere fra 25 e 50 secondi invece che fra 5 e 16 minuti. In totale gli step evitati valgono circa 1.200 minuti di esecuzione, quasi tutti nei repository Rust, perché negli altri la verifica completa dura meno di due minuti, e saltarla cambia poco.

Il costo per l'azienda è però maggiore della durata, perché Bitbucket conteggia i minuti moltiplicandoli per la taglia dello step, e uno step `2x` consuma due minuti per ogni minuto di esecuzione [1]. Applicando la taglia di ciascuno step, le verifiche evitate corrispondono a circa 3.300 minuti di *build*. Nello stesso periodo gli step di verifica ne hanno consumati circa 5.700; senza il *gate* sarebbero stati circa 9.000, quindi il *gate* ha ridotto il consumo della verifica di circa il 37%.

Il numero va letto con due cautele. Assume che ogni verifica evitata sarebbe durata quanto la mediana delle verifiche complete del suo repository, e non considera le verifiche fallite, che durano in media meno di una completa. È quindi una stima, costruita però su durate misurate.

6.7 Esito

Al termine del lavoro 16 repository su 18 verificano il codice in modo automatico, contro i 2 iniziali. I 2 rimanenti sono quelli che caricano ancora la libreria con il meccanismo delle prime versioni (sezione 4.8) e non richiamano il *gate*. Uno contiene soltanto la configurazione di un'immagine Docker, l'altro è un progetto Python, e per quest'ultimo la migrazione resta da completare.

La verifica parte soltanto quando può portare informazione nuova, e in caso di dubbio parte comunque. Nel periodo osservato ha evitato 291 esecuzioni complete, circa 1.200 minuti di esecuzione e 3.300 minuti di *build* conteggiati da Bitbucket, il 37% del consumo che la verifica avrebbe avuto senza il *gate*. Nessun repository può più pubblicare una *release candidate* in produzione, indipendentemente dall'ordine in cui Bitbucket valuta i *pattern*. I requisiti **R4**, **R5** e **R6** vengono ripresi nel capitolo 8.

Capitolo 7

Ambienti di sviluppo riproducibili

I capitoli precedenti riguardano ciò che accade al codice dopo un *commit*. Questo riguarda ciò che accade prima, sulla macchina dello sviluppatore, ed è l'ultima criticità individuata nel capitolo 2 (sezione 2.2.5). Il lavoro descritto è più circoscritto di quello sulle *pipeline* e riguarda due template di Dev Container, uno per Rust e uno per Python, realizzati all'inizio del tirocinio e poi ripresi e ampliati da altri membri del team.

7.1 Il problema

Nel team era frequente passare da un progetto all'altro, e quindi da un linguaggio all'altro, per esempio da un servizio Rust a uno strumento Python o a un componente Go. Ogni passaggio richiedeva di configurare a mano l'ambiente: la versione del compilatore, gli strumenti di formattazione e analisi, le librerie di sistema necessarie alle dipendenze, le estensioni dell'editor. A parte il tempo, il risultato era una serie di ambienti diversi da sviluppatore a sviluppatore, e diversi da quello in cui girava la *pipeline*. Un problema di formattazione o un avviso di CLIPPY emergeva così soltanto in CI, dopo il *push*, invece che nell'editor.

L'obiettivo era quindi un ambiente per linguaggio, pronto all'uso, uguale per tutti e allineato agli strumenti della *pipeline*, come chiede il requisito **R7**.

7.2 Alternative considerate

Le strade possibili erano tre. La più semplice è una guida d'installazione, eventualmente accompagnata da uno script, che ciascuno esegue sulla propria macchina. Costa poco da scrivere, ma non garantisce che gli ambienti restino uguali nel tempo, perché ogni macchina parte da uno stato diverso e si aggiorna per conto suo. La

più completa è una macchina virtuale condivisa, che isola tutto ma è pesante da distribuire e da aggiornare. La terza è un Dev Container: un'immagine Docker che contiene l'ambiente, descritta da un file `devcontainer.json` che l'editor legge per avviarla e collegarsi ad essa [6].

La scelta è caduta sui Dev Container per ragioni di contesto più che di principio. Il team usava già Visual Studio Code, che supporta la specifica nativamente, e Docker, che serviva comunque per i test e per le immagini dei servizi. L'ambiente diventa un file versionato nel repository, quindi chi lo modifica lo modifica per tutti, e la modifica si rivede come qualsiasi altro codice.

7.3 Template per Rust

Il template Rust è nato dall'ambiente costruito a marzo per il servizio di *scheduling*, e ne è stato ricavato separando ciò che era specifico di quel progetto da ciò che serviva a qualsiasi progetto Rust del team.

7.3.1 Immagine di base e strumenti

L'immagine parte da quella pubblicata da Microsoft per i Dev Container Rust (`devcontainers/rust`, su Debian *bookworm*), già usata in azienda e mantenuta aggiornata dal suo autore, e contiene il compilatore, CARGO e l'utente non privilegiato con cui l'editor lavora. Sopra di essa vengono installati gli strumenti del progetto, divisi in due gruppi (listato 7.1):

```
install_required(){ cmd="$1"; name="$2";
    if ! command -v "$cmd" >/dev/null 2>&1; then cargo install "$name" --
    ↪ locked; fi; }
install_optional(){ cmd="$1"; name="$2";
    if ! command -v "$cmd" >/dev/null 2>&1; then
        cargo binstall -y "$name" || cargo install "$name" --locked \
            || echo "Skipping optional tool: $name"
    fi; }
install_required cargo-chef cargo-chef
install_optional cargo-nextest cargo-nextest
install_optional cargo-deb cargo-deb
install_optional cargo-audit cargo-audit
```

Listato 7.1: Strumenti obbligatori e opzionali nel Dockerfile del template Rust (estratto; il file completo è nel listato A.6).

Uno strumento obbligatorio che non si installa fa fallire la costruzione dell'immagine, perché senza di esso l'immagine non funzionerebbe: è il caso di CARGO-CHEF, usato subito dopo nel Dockerfile. Uno strumento opzionale viene prima cercato come binario già compilato con CARGO-BINSTALL, che impiega pochi secondi; se non è disponibile viene compilato dai sorgenti, e se anche questo fallisce viene saltato con un messaggio. In questo modo un singolo strumento di comodità, per esempio uno che non compila con la versione corrente del compilatore, non impedisce a tutto il team di avere un ambiente.

Prima degli strumenti viene aggiunto CLIPPY, come componente del compilatore; fra gli opzionali ci sono poi gli stessi strumenti usati dalla *pipeline*: CARGO-NEXTEST per i test, CARGO-DEB per i pacchetti e CARGO-AUDIT per le vulnerabilità (sezione 6.3). Ciò che passa in locale passa quindi, a parità di codice, anche nel *gate* di CI.

7.3.2 Dipendenze precompilate con cargo-chef

Un progetto Rust di medie dimensioni ha centinaia di dipendenze, e la prima compilazione dopo l'apertura del container può richiedere diversi minuti. Per ridurla, il template prepara le dipendenze già durante la costruzione dell'immagine, con CARGO-CHEF (listato 7.2):

```
ENV CARGO_TARGET_DIR=/usr/local/cargo/target
WORKDIR /workspaces/job-scheduler

COPY Cargo.toml ./
COPY src ./src
RUN set -e; cargo chef prepare --recipe-path recipe.json
RUN set -e; cargo chef cook --release --recipe-path recipe.json
```

Listato 7.2: Preparazione delle dipendenze nel Dockerfile del template Rust.

`cargo chef prepare` ricava dal progetto una "ricetta" che descrive soltanto le dipendenze, e `cargo chef cook` le scarica e le compila. Rispetto a un semplice `cargo fetch`, che si limita a scaricare i sorgenti, CARGO-CHEF produce anche i risultati della compilazione, e lo fa in un *layer* Docker che dipende solo dalla ricetta, quindi finché le dipendenze non cambiano, Docker riusa il *layer* dalla sua cache anche se il codice del progetto è cambiato.

La variabile `CARGO_TARGET_DIR` sposta la cartella dei risultati di compilazione fuori dallo spazio di lavoro. Lo spazio di lavoro è la cartella del progetto montata dalla macchina dello sviluppatore. Tenendo lì i risultati, quelli prodotti dentro il container

si mescolerebbero con quelli prodotti fuori, per un sistema operativo diverso, e i due ambienti si invaliderebbero la cache a vicenda.

Questa versione ha due limiti, emersi nell'uso. Il primo è che il template resta legato a un progetto: il Dockerfile copia `Cargo.toml` e i sorgenti di un repository preciso, e va adattato per ciascun progetto in cui viene copiato. Il secondo è più sottile. Le dipendenze vengono preparate con il profilo `--release`, mentre durante lo sviluppo si compila con il profilo predefinito, che CARGO tiene in una cartella separata: in sviluppo le dipendenze vanno quindi ricompilate, e il vantaggio si riduce ai sorgenti già scaricati e alle *build* di *release*. Entrambi i limiti sono stati superati nella versione successiva (sezione 7.6).

7.4 Template per Python

Il template Python segue lo stesso schema su un'altra immagine di base Microsoft, con Python 3.12. Aggiunge le librerie di sistema necessarie a compilare il client MariaDB usato dai servizi Django, e installa gli strumenti di sviluppo (formattazione, ordinamento degli *import*, analisi statica, test) con PIPX, che dà a ciascuno un ambiente separato, così gli strumenti non entrano in conflitto fra loro né con le dipendenze dei progetti. Le dipendenze più comuni dei progetti Python dell'azienda, dal *framework* web agli SDK di AWS, sono già installate nell'immagine, così che un progetto nuovo si apra con la maggior parte di ciò che gli serve.

7.5 Esperienza dello sviluppatore

Il file `devcontainer.json` descrive come l'editor usa l'immagine: quale cartella aprire, quali componenti aggiuntivi installare (Node.js, per esempio, per un generatore di client OpenAPI) e quali estensioni di Visual Studio Code attivare. Le estensioni sono 33 nel template Rust e 26 in quello Python, e la scelta non è stata fatta da una sola persona. Gli sviluppatori, che lavorano nello stesso *open space*, sono stati interpellati a voce uno per uno su quali estensioni usassero e trovassero utili, per linguaggio e per funzionalità. Alle loro preferenze sono state aggiunte altre estensioni individuate durante la preparazione dei template. L'accoglienza è stata favorevole e non sono emerse preferenze in conflitto.

Nel template Rust, dopo la creazione del container, un comando di inizializzazione (`postCreateCommand`) scarica le dipendenze e compila il progetto. Se la compilazione fallisce, per esempio perché il codice sul branch corrente non compila, il comando

lo segnala ma non fa fallire la creazione del container, perché lo sviluppatore deve poter entrare nell'ambiente proprio per correggere il codice.

7.6 Evoluzione successiva

I template sono stati il punto di partenza di un lavoro proseguito dal team. Il repository, creato da un collega, conteneva già il template Go, scritto da un altro membro del team che ha poi aggiunto anche quello TypeScript. Chi aveva creato il repository ha in seguito rielaborato il template Rust, risolvendo i due limiti descritti nella sezione 7.3.2: gli strumenti sono stati spostati in un'immagine separata, costruita una volta e riusata, e i template sono diventati indipendenti dal singolo progetto, che viene montato nel container invece di esservi copiato. A questo si è aggiunto un piccolo insieme di comandi per aprire e ricostruire i container dal terminale.

7.7 Esito

Al termine del tirocinio il team dispone di un template per ciascuno dei quattro linguaggi in uso, versionati in un repository comune. Quelli per Rust e Python, descritti in questo capitolo, hanno fissato lo schema seguito anche dagli altri: un'immagine di base ufficiale, strumenti obbligatori e opzionali, gli stessi strumenti della *pipeline*, estensioni scelte insieme al team. Il requisito **R7** è soddisfatto nel senso che un ambiente di sviluppo completo si ottiene aprendo un template, invece che seguendo una procedura manuale; non è stato misurato, invece, il tempo risparmiato per sviluppatore, un limite ripreso nel capitolo 8.

Capitolo 8

Risultati e valutazione

Questo capitolo misura i risultati del lavoro. Riprende i requisiti ricavati nel capitolo 2 e, per ciascuno, verifica se e in che misura il lavoro descritto nei capitoli 4–7 lo soddisfa. I confronti "prima" e "dopo" si riferiscono, per ogni repository, alla versione del file `bitbucket-pipelines.yml` immediatamente precedente alla migrazione alla libreria e a quella presente al termine del lavoro.

8.1 Verifica dei requisiti

Tabella 8.1: Esito rispetto ai requisiti individuati nel capitolo 2.

Req.	Esito	Evidenza
R1	soddisfatto	18 repository su 18 usano la libreria
R2	parziale	16 su 18 agganciati all'ultimo tag stabile; 2 caricano ancora da `master`
R3	soddisfatto	versione e tag calcolati dalla *pipeline* in 18 repository su 18 (erano 4); i casi ambigui tornano allo sviluppatore
R4	soddisfatto	Rust, Go, Python e TypeScript riconosciuti dal contenuto del repository
R5	soddisfatto	verifica automatica in 16 repository su 18 (erano 2); 291 verifiche evitate, circa 3.300 minuti di *build* in meno (−37%)
R6	soddisfatto	guardia sul *deployment* in 18 repository su 18 (erano 0)
R7	soddisfatto	template per Rust e Python, poi estesi dal team a Go e TypeScript; tempo risparmiato non misurato

La tabella 8.1 riassume l'esito; le sezioni seguenti ne riportano le evidenze. Due requisiti meritano subito un commento. **R2** è soddisfatto solo in parte, perché due repository, migrati in autonomia da un collega con il meccanismo delle prime versioni, caricano ancora i moduli uno per uno dal branch `master` della libreria, e una modifica sbagliata li raggiungerebbe senza passare da un rilascio. La modifica che li porta su `bootstrap.sh` è pronta, ma non è stata applicata per mancanza di permessi su quei

repository. **R7** è soddisfatto nel senso qualitativo descritto nel capitolo 7, cioè un ambiente completo si ottiene aprendo un template, ma non sono disponibili misure del tempo risparmiato.

8.2 Copertura dei repository

Il risultato più evidente riguarda che cosa le *pipeline* fanno. La tabella 8.2 confronta, per ciascuna funzionalità, il numero di repository che la implementavano prima e dopo l'intervento.

Tabella 8.2: Funzionalità delle *pipeline* prima e dopo l'intervento, su 18 repository.

Funzionalità	**Prima**	**Dopo**
pipeline avviata da un *push* o da un tag	8	18
test o analisi statica del codice	2	16
versione e tag calcolati dalla *pipeline*	4	18
guardia contro il rilascio di una *release candidate*	0	18
immagine Docker	13	17
pacchetto `.deb`	5	6

Le ultime due righe, che riguardano gli artefatti, cambiano poco, perché le *pipeline* sapevano già produrre ciò che serviva e l'intervento ha cambiato solo il modo in cui lo fanno. Le prime quattro, invece, misurano ciò che prima mancava. Dieci repository non avevano una *pipeline* che partisse da sola, e lo stesso numero si limitava, quando veniva avviata, a costruire un'immagine; al termine del lavoro tutti i 18 repository hanno una *pipeline* per branch e per tag, 16 verificano il codice e tutti calcolano la versione e controllano l'ambiente di destinazione. La tabella 8.3 riporta il dettaglio per repository.

8.3 Semplicità delle pipeline

Il numero di righe dei file YAML, da solo, è una metrica ingannevole. Nei 4 repository pilota, dove la *pipeline* esisteva già completa, le righe scendono da 1.974 a 1.212 (-39%) mentre le funzionalità aumentano. Negli altri repository le righe crescono, da 43–152 a 186–631, perché dove prima c'era un solo step che costruiva un'immagine ora c'è una *pipeline* intera. Confrontare i totali significherebbe confrontare cose diverse.

Più indicativo è che cosa contengono quelle righe. La tabella 8.4 misura, sui 4 repository pilota, tre aspetti che determinano quanto una *pipeline* sia facile da

Tabella 8.3: Funzionalità per repository prima e dopo l'intervento (nomi dei repository fittizi). T = test o analisi statica, V = versione e tag calcolati, D = pacchetto `.deb`, I = immagine Docker, G = guardia sul rilascio.

Repository	Prima	Dopo
auth-provider	I	V I G
core-backend	I	T V I G
device-availability	I	T V I G
image-service	D	T V D I G
job-scheduler	—	T V I G
lambda-launcher	I	T V I G
licensing-agent	I	V I G
m2m-link	I	T V D I G
mqtt-bridge	T V D I	T V D I G
ops-utilities	I	T V I G
result-manager	D	T V D I G
run-manager	D	T V D I G
sut-controller	I	T V I G
test-catalog	V I	T V I G
test-flow-engine	T V D	T V D G
test-id-service	V I	T V I G
web-frontend	I	T V I G
workflow-engine	I	T V I G

leggere e da modificare: quanta logica Bash è scritta direttamente nel file YAML (contando le righe con strutture di controllo come `if`, `for` e `case`), quante righe si ripetono identiche nello stesso file e quante sono copiate da un repository all'altro.

Tabella 8.4: Logica e duplicazione nelle *pipeline* dei 4 repository pilota, prima e dopo la migrazione.

	Prima	Dopo	Variazione
righe totali	1.974	1.212	−39%
righe con logica Bash (`if`, `for`, …)	289	26	−91%
righe ripetute nello stesso file	179	49	−73%
righe identiche in almeno 2 dei 4 file	163	55	−66%

La logica Bash nel YAML si riduce di nove decimi, perché le condizioni, i cicli e i calcoli sono passati nella libreria, dove sono scritti una volta, con nomi che ne spiegano lo scopo, e provati dal *self-test*. Nel file del repository resta la struttura della *pipeline* e una sequenza di chiamate leggibile anche da chi non conosce Bash. Le ripetizioni all'interno dello stesso file scendono di quasi tre quarti, grazie alle ancore YAML e alla scomparsa dei blocchi quasi identici per `develop` e `master` (sezione 2.2.1). Le righe che restano identiche fra repository sono soprattutto il blocco che carica la

libreria (listato 4.1) e le chiamate alle sue funzioni, cioè una duplicazione voluta che non contiene logica.

La stessa misura sui repository le cui *pipeline* sono state scritte da zero mostra il limite residuo. Dove un repository pubblica in più regioni, per esempio Europa e Cina, gli step di pubblicazione sono ancora ripetuti per ciascuna, con poche differenze. È la stessa forma di duplicazione che la libreria ha eliminato per la logica, e il candidato naturale per un intervento successivo.

8.4 Tempi e costi della CI

I risultati sul costo della verifica sono stati presentati nella sezione 6.6. In sintesi, nei 16 repository che usano il *gate* sono state evitate 291 verifiche complete, 260 perché il *commit* non modificava i sorgenti e 31 perché lo stesso *commit* era già stato verificato. Una verifica completa dura nei repository Rust fra 4 e 16 minuti, una saltata meno di un minuto; le verifiche evitate valgono circa 1.200 minuti di esecuzione e, tenendo conto della taglia degli step, circa 3.300 minuti di *build* conteggiati da Bitbucket, il 37% in meno di quanto la verifica avrebbe consumato senza il *gate*.

La stessa raccolta di dati, 1.464 *pipeline* e 6.359 step, mostra altri due aspetti che non erano obiettivi del lavoro ma ne completano il quadro. Il primo è che il 39% delle *pipeline* (569) resta in pausa su uno step manuale che nessuno avvia. È l'effetto atteso della scelta di lasciare manuali la creazione del tag e la pubblicazione (sezione 4.5), e non un consumo, perché una *pipeline* in pausa non usa minuti. Il secondo è che in 6 repository la *cache* delle dipendenze non viene mai ritrovata, e ogni esecuzione riparte da zero, un margine di risparmio ulteriore.

8.5 Manutenibilità

Il vantaggio di avere la logica in un solo posto si vede meglio nei casi concreti. Due modifiche descritte nei capitoli precedenti lo mostrano da lati opposti.

La guardia sul *deployment* (sezione 6.5) è stata scritta una volta nella libreria e pubblicata con un solo rilascio. Per attivarla nei 18 repository è bastato aggiungere una riga, la chiamata alla funzione, nello step di pubblicazione di ciascuno. L'intero intervento si è concluso nel giro di pochi giorni, e una correzione futura della guardia raggiungerà tutti i repository senza toccarne nessuno.

La correzione delle versioni Debian (sezione 5.7) mostra il caso opposto. La conversione del tag nella versione Debian è nella libreria, ma il comando che costruisce il pacchetto è scritto nel file YAML di ciascun repository, e ogni repository lo usava

a modo suo. I repository Go passavano la versione a uno script di pacchettizzazione, quelli Rust passavano a CARGO-DEB soltanto la revisione. La libreria calcola già la versione corretta e la espone nella variabile `DEB_VERSION`, ma perché abbia effetto ciascuno dei 6 repository che producono pacchetti deve usarla nel proprio comando di *build*. Al termine del lavoro la modifica è pronta su un branch per i 3 repository Go e ancora da scrivere per i 3 repository Rust, e le *release candidate* di questi pacchetti vengono quindi ancora pubblicate con la versione nella forma che DPKG ordina in modo sbagliato. Il problema sopravvive proprio nel punto in cui la logica non è centralizzata.

8.6 Limiti del lavoro svolto

Il lavoro ha alcuni limiti. Due repository caricano ancora la libreria da `master` e non usano il *gate* di CI, e i 6 repository che producono pacchetti Debian non usano ancora la versione corretta calcolata dalla libreria. Il versionamento richiede veri merge *commit* (sezione 5.5.4), il *gate* confronta solo l'ultimo *commit* di un *push*, e il *self-test* della libreria non confronta automaticamente il risultato con quello atteso. Infine, il modello V4 è in uso da poco tempo e per i Dev Container mancano misure del tempo di allestimento prima e dopo.

Capitolo 9

Conclusioni

9.1 Sintesi del lavoro svolto

Il tirocinio è partito da un problema di organizzazione. Le *pipeline* dei 18 repository applicativi producevano gli artefatti che servivano, ma ciascuna lo faceva a modo suo. La stessa logica esisteva in molte copie divergenti, la versione veniva decisa a mano o con regole diverse da repository a repository, e il codice arrivava in un artefatto senza essere verificato. Solo 2 repository su 18 eseguivano test, solo 4 calcolavano la versione, nessuno impediva a una *release candidate* di arrivare in produzione.

La risposta è stata spostare la logica comune in un unico punto, cioè in una libreria Bash di otto moduli, caricata da tutte le *pipeline* e agganciata alla sua ultima versione stabile, così che una modifica raggiunga i repository solo quando viene pubblicata. Sopra questa base sono state costruite le tre funzioni che mancavano. Il calcolo della versione deduce il numero dalla provenienza dei merge e, nei casi che le regole non coprono, restituisce la decisione allo sviluppatore invece di indovinare. Il *gate* di CI verifica il codice in tutti i linguaggi in uso, ma solo quando la verifica può portare informazione nuova. La guardia sul *deployment* rende impossibile pubblicare una *release candidate* in produzione, qualunque sia l'ordine in cui Bitbucket valuta i *pattern* dei tag. A margine delle *pipeline*, i template di Dev Container hanno portato sulla macchina dello sviluppatore gli stessi strumenti usati in CI.

Il valore per l'azienda si misura su ciò che le *pipeline* fanno oggi. I repository che verificano il codice sono passati da 2 a 16, quelli con versione e tag calcolati da 4 a 18, quelli protetti contro il rilascio di una *release candidate* da 0 a 18. Nelle quattro *pipeline* più complete la logica Bash scritta direttamente nei file YAML si è ridotta del 91%. Il *gate* ha evitato 291 verifiche inutili, circa 3.300 minuti di *build* conteggiati da Bitbucket, il 37% del consumo che la verifica avrebbe avuto senza di esso. C'è poi un risultato più difficile da misurare, cioè che una correzione alla logica

comune si scrive una volta e raggiunge tutti i repository con la pubblicazione di un tag, invece di richiedere 18 modifiche separate.

9.2 Sviluppi futuri

Gli sviluppi più immediati riguardano il completamento dell'adozione e nascono direttamente dai limiti descritti nella sezione 8.6. I due repository che caricano ancora la libreria da `master` vanno agganciati all'ultima versione stabile e collegati al *gate* di CI, completando anche la migrazione del progetto Python. Per i 6 repository che producono pacchetti Debian si tratta di unire la modifica già pronta per i 3 repository Go e di scrivere quella per i 3 repository Rust, così che il comando di *build* usi la variabile `DEB_VERSION` già calcolata dalla libreria. A quel punto la versione delle *release candidate* verrebbe ordinata correttamente da DPKG in tutti i pacchetti. Più in generale, il caso suggerisce di portare nella libreria anche i comandi di pacchettizzazione, che oggi restano scritti nei singoli file YAML.

Un secondo gruppo di interventi riguarda la robustezza della libreria stessa. Il *gate* può essere reso più preciso considerando tutti i *commit* di un *push* e non solo l'ultimo, e il versionamento può essere esteso ai casi in cui un branch viene integrato senza un vero merge *commit*, oggi esclusi dalle regole.

Infine, alcune scelte andranno rivalutate con il tempo. Il modello V4 di versionamento è in uso da poco, e solo un periodo di utilizzo più lungo mostrerà se le sue regole coprono tutti i casi reali o se serve un'ulteriore revisione. Per i Dev Container, raccogliere misure del tempo di allestimento dell'ambiente prima e dopo la loro introduzione permetterebbe di quantificare un beneficio che oggi è stato solo osservato.

9.3 Considerazioni personali

Il lavoro descritto in questa tesi non era previsto. Sono arrivato in azienda per conoscere le architetture usate ogni giorno, e il primo argomento che mi è stato presentato sono state le *pipeline*. Mi sono accorto quasi subito che ogni modifica andava ripetuta identica in tutti i repository, e ho proposto di cambiare le cose senza avere il permesso di modificarne nessuno. Questo vincolo, che all'inizio sembrava un ostacolo, è stato la parte più formativa del tirocinio, perché mi ha costretto a costruire un ambiente in cui provare tutto in locale e a dimostrare che la soluzione funzionava prima di chiedere di usarla. L'accesso è arrivato un repository alla volta,

e con esso ho imparato che in un'azienda la fiducia si guadagna mostrando risultati che gli altri possono verificare.

La seconda cosa che ho imparato riguarda il codice condiviso. Ho visto presto che cambiare una funzione da cui dipendono 18 repository è molto più difficile che scriverla. La rinomina di una sola variabile, o un difetto nel *gate* che ha bloccato le *pipeline* per ore, mi hanno mostrato che una libreria usata da altri è un'interfaccia pubblica, e che la compatibilità all'indietro e la possibilità di scegliere quando aggiornarsi valgono quanto le funzioni stesse. Allo stesso modo, le tre riscritture del modello di versionamento mi hanno insegnato a ripercorrere i casi d'uso reali prima di scrivere le regole.

Infine, il lavoro non è stato solitario. Le scelte che riguardavano tutti, dalle regole sui merge alle estensioni dei Dev Container, sono state discusse con il team, e diverse parti della libreria e dei template sono state estese dai colleghi dopo di me. Vedere altri adottare in autonomia qualcosa che ho costruito, e migliorarlo, è stato il riscontro più concreto che il lavoro fosse utile.

Appendice A

Codice sorgente

Questa appendice riporta il codice citato nei capitoli precedenti. Nomi dei repository, indirizzi e credenziali sono stati sostituiti con valori fittizi; il resto è riportato senza modifiche, commenti in inglese compresi.

A.1 Libreria condivisa

I file provengono dalla versione `1.15.2` della libreria, la prima che contiene il modello di versionamento V4. Il punto d'ingresso (sezione 4.3.1) carica gli altri moduli nell'ordine corretto.

```
# Single entry point for the shared pipeline library.
#
# Instead of sourcing every lib file from each consumer's bitbucket-pipelines.yml
# (~6 lines duplicated across 18 repos), a pipeline sources just this file:
#
#     git clone --filter=blob:none --no-checkout https://x-token-auth:${PIPE_SCRIPT_TOKEN}@.../pipeline-shared-lib.git /tmp/pipeline-utils
#     PIPE_REF="$(git -C /tmp/pipeline-utils tag --list --sort=-v:refname | grep -E '^[0-9]+\.[0-9]+\.[0-9]+$' | head -1)"
#     git -C /tmp/pipeline-utils checkout --quiet "${PIPE_REF:-master}"
#     source /tmp/pipeline-utils/reusable-bash-code/lib/bootstrap.sh
#
# Consumers pin to the highest stable X.Y.Z tag, not to a branch -- publishing a
# tag is what rolls a change out to them; merging to master alone changes nothing.
#
```

```
# Adding a new lib module then only requires editing this file, not all 18 repos.
# The git clone itself must stay in the YAML: you cannot call a function from a
# repo you haven't cloned yet (chicken-and-egg), so bootstrap only aggregates the
# sourcing, not the clone.

_BOOTSTRAP_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

# Order matters: log-utils first (everything logs through it), then modules that
# depend on lower-level helpers (semver before git-tag, etc.).
source "${_BOOTSTRAP_DIR}/log-utils.sh"
source "${_BOOTSTRAP_DIR}/semver-utils.sh"
source "${_BOOTSTRAP_DIR}/git-tag-utils.sh"
source "${_BOOTSTRAP_DIR}/package-utils.sh"
source "${_BOOTSTRAP_DIR}/version-resolver.sh"
source "${_BOOTSTRAP_DIR}/pipeline-guards.sh"
source "${_BOOTSTRAP_DIR}/ci-gate.sh"
```

Listato A.1: `bootstrap.sh`.

Le guardie che decidono se uno step può proseguire (sezioni 4.5 e 6.5).

```
_LIB_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "${_LIB_DIR}/log-utils.sh"

skip_if_tag_just_created() {
    log_step "skip current operation when tag was just created"

    local operation="${1:-operation}"
    if [ "${TAG_JUST_CREATED:-0}" = "1" ]; then
        log_skip "Tag just created; tag pipeline handles ${operation}"
        exit 0
    fi

    log_ok "Tag not just created; proceeding with ${operation}."
}

init_tag_guard() {
    log_step "load tag guard state from generated artifact file"

    TAG_JUST_CREATED=0
    local source_file="${1:-.tag_ready}"
```

```
21      if [ -f "$source_file" ]; then
22          source "$source_file"
23      fi
24      export TAG_JUST_CREATED
25  }
26
27  tag_wall(){
28      log_step "enforce tag-based safety checks before manual publish"
29
30      if [ "${TAG_READY:-0}" != "1" ]; then
31          log_skip "Commit not tagged"
32          exit 0
33      fi
34
35      # Wall: if the tag was just created in Step 2, let the tag pipeline do
        ↪ the builds
36      if [ "${TAG_JUST_CREATED:-0}" = "1" ]; then
37          log_skip "Tag was just created in the previous step"
38          exit 0
39      fi
40
41      # Resolve tag preference: TAG_VERSION -> HEAD tag
42      if [ -n "${TAG_VERSION:-}" ]; then
43          export LATEST_TAG="${TAG_VERSION}"
44      else
45          export LATEST_TAG="$(git tag --points-at HEAD | head -n1 || true)"
46      fi
47
48      if [[ -z "${LATEST_TAG}" ]] || [[ ! "${LATEST_TAG}" =~ ^([0-9]{1,3})
        ↪ \.([0-9]{1,3})\.([0-9]{1,3})(-rc[0-9]{1,3}([.-][a-zA-Z0-9-]+)?)?$ ]];
        ↪ then
49          log_skip "No valid semver tag available"
50          exit 0
51      fi
52
53      # Ensure tag points to this commit (prevent stale manual builds)
54      if [[ "$(git rev-list -n1 "${LATEST_TAG}" 2>/dev/null || true)" != "${
        ↪ BITBUCKET_COMMIT}" ]]; then
55          log_skip "Tag ${LATEST_TAG} is not on this commit"
56          exit 0
57      fi
58
59      # Resolve Environment
60      if [ "${BITBUCKET_BRANCH:-}" = "master" ] || [ "${BITBUCKET_BRANCH:-}" =
        ↪ "main" ]; then
```

```
61          export ENVIRONMENT="production"
62      else
63          export ENVIRONMENT="staging"
64      fi
65
66      log_ok "Tag ${LATEST_TAG} is valid and points to this commit; proceeding
    ↪ with ${ENVIRONMENT} build."
67  }
68
69  cargo_exists(){
70      log_step "detect Cargo.toml and read current package version"
71
72      if [ -f Cargo.toml ]; then
73          CURRENT_CARGO=$(grep '^version' Cargo.toml | head -n1 | cut -d'"' -f2
    ↪ )
74          log_ok "Cargo.toml version: $CURRENT_CARGO"
75      else
76          CURRENT_CARGO=""
77          log_skip "Cargo.toml not found"
78      fi
79  }
80
81  # Last line of defence before a build is published to an environment.
82  #
83  # Two independent things can go wrong, and this checks both:
84  #
85  #   1. A release candidate reaching production. Bitbucket runs only the FIRST
86  #      tag pipeline whose glob matches, and '*.*.*' happily swallows
87  #      '1.2.3-rc1' -- so an rc tag can land on the stable pipeline purely
88  #      through glob ordering. Ordering is easy to get wrong in review and
89  #      invisible at runtime; this check is not.
90  #
91  #   2. ENVIRONMENT disagreeing with the deployment the step actually runs in.
92  #      ENVIRONMENT is derived (from the tag, the branch, or version.env)
    ↪ while
93  #      BITBUCKET_DEPLOYMENT_ENVIRONMENT comes from the step definition. When
94  #      the two drift apart the build is pushed somewhere nobody intended.
95  #
96  # Steps without a 'deployment:' (the .deb build/publish steps) leave
97  # BITBUCKET_DEPLOYMENT_ENVIRONMENT empty. There is nothing to cross-check
98  # there, so check (2) is skipped -- but check (1) still applies, because
99  # ENVIRONMENT is what selects the apt bucket.
100 #
101 # Call it after ENVIRONMENT has been resolved, immediately before build/push.
102 deployment_env_guard(){
```

```
103     log_step "verify the build is allowed in this deployment environment"
104
105     local deployment="${BITBUCKET_DEPLOYMENT_ENVIRONMENT:-}"
106     local expected_env=""
107
108     if [ -n "$deployment" ]; then
109         case "$deployment" in
110             production-*) expected_env="production" ;;
111             staging-*)    expected_env="staging" ;;
112             *)
113                 log_error "unrecognized deployment '${deployment}'"
114                 log_info  "  expected a name starting with 'production-' or '
    ↪ staging-'"
115                 exit 1
116                 ;;
117         esac
118     fi
119
120     # (1) An rc tag must never be published as production, however it got
    ↪ here.
121     if [ "${expected_env:-${ENVIRONMENT:-}}" = "production" ]; then
122         case "${BITBUCKET_TAG:-}" in
123             *-rc*)
124                 log_error "RC tag '${BITBUCKET_TAG}' cannot be published to
    ↪ production"
125                 log_info  "  a release candidate belongs in staging; check
    ↪ the order of the"
126                 log_info  "  tag globs in bitbucket-pipelines.yml (the rc
    ↪ pattern must come first)"
127                 exit 1
128                 ;;
129         esac
130     fi
131
132     if [ -z "$deployment" ]; then
133         log_skip "step has no deployment; cross-check skipped (ENVIRONMENT=${
    ↪ ENVIRONMENT:-<unset>})"
134         return 0
135     fi
136
137     # (2) The derived environment must agree with the step's deployment.
138     if [ "${ENVIRONMENT:-}" != "$expected_env" ]; then
139         log_error "ENVIRONMENT does not match the deployment this step runs
    ↪ in"
140         log_info  "  ENVIRONMENT: ${ENVIRONMENT:-<unset>}"
```

```
        log_info  "  deployment:  ${deployment} (expects '${expected_env}')"
        exit 1
    fi

    log_ok "publishing to ${deployment} (ENVIRONMENT=${ENVIRONMENT})"
}
```

Listato A.2: `pipeline-guards.sh`.

La conversione del tag nella versione dei pacchetti Debian (sezione 5.7) e nella versione PEP 440 dei progetti Python.

```
_LIB_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "${_LIB_DIR}/log-utils.sh"

# Map a git tag to the version string that goes into debian/changelog.
#
# The tag is the source of truth, but a git tag and a Debian version are not
# the same grammar: dpkg reads everything after the last '-' as the Debian
# revision, and an absent revision sorts BEFORE any revision. So the natural
# spelling '1.3.4-rc5' makes the release candidate sort *newer* than the
    ↪ final
# 1.3.4 and a box on an rc would never move onto the stable release:
#
#   dpkg --compare-versions 1.3.4-rc5 gt 1.3.4   -> true   (wrong)
#   dpkg --compare-versions 1.3.4~rc5 lt 1.3.4   -> true   (what we want)
#
# '~' is the only character dpkg sorts before the empty string, which is
# exactly what a prerelease needs. Stable tags get an explicit '-1' revision
    ↪ so
# a packaging-only rebuild can ship as '-2' without burning a patch number.
#
#   tag 1.3.4-rc5  ->  deb 1.3.4~rc5   (staging)
#   tag 1.3.4      ->  deb 1.3.4-1     (production)
#
# V4: also handles branch work tags. The git tag separates the branch suffix
# with '-' (never '+': the tag is also a Docker image tag), the Debian
    ↪ spelling keeps '.'
# so nothing after the last '-' is read as a Debian revision:
#   tag 1.3.4-rc5-auth  ->  deb 1.3.4~rc5.auth   (staging/branch)
deb_version_from_tag(){
    local tag="$1"

    if [[ "$tag" =~ ^(.+)-rc([0-9]+)[.-](.+)$ ]]; then
        # suffixed: 1.3.4-rc5-auth -> 1.3.4~rc5.auth
```

```
        printf '%s~rc%s.%s' "${BASH_REMATCH[1]}" "${BASH_REMATCH[2]}" "${
    ↪ BASH_REMATCH[3]}"
    elif [[ "$tag" =~ ^(.+)-rc([0-9]+)$ ]]; then
        printf '%s~rc%s' "${BASH_REMATCH[1]}" "${BASH_REMATCH[2]}"
    else
        printf '%s-1' "$tag"
    fi
}

# Map a git tag to a PEP 440 version for pyproject.toml / wheels.
#
# The tag cannot go into pyproject.toml verbatim: PEP 440 has no slot for a
# label after the pre-release, so uv/pip reject '0.5.2-rc0-auth' (and the
# legacy '0.5.2-rc0.auth') outright. The branch name goes into the local
# version label instead, which PEP 440 spells with '+' and '.'-separated
# lowercase alphanumerics:
#
#   tag 1.3.4                  ->  1.3.4                  (production)
#   tag 1.3.4-rc5              ->  1.3.4rc5               (staging)
#   tag 1.3.4-rc5-auth         ->  1.3.4rc5+auth          (staging/branch)
#   tag 1.3.4-rc5-JIRA-1-Login ->  1.3.4rc5+jira.1.login
#
# '+' is fine here even though it is banned in the tag: this string never
# becomes a Docker image tag. Note public indexes (PyPI) refuse local
    ↪ versions;
# private ones accept them.
pep440_version_from_tag(){
    local tag="$1"

    if [[ "$tag" =~ ^(.+)-rc([0-9]+)[.-](.+)$ ]]; then
        local label="${BASH_REMATCH[3],,}"
        label="${label//[-_]/.}"
        printf '%src%s+%s' "${BASH_REMATCH[1]}" "${BASH_REMATCH[2]}" "$label"
    elif [[ "$tag" =~ ^(.+)-rc([0-9]+)$ ]]; then
        printf '%src%s' "${BASH_REMATCH[1]}" "${BASH_REMATCH[2]}"
    else
        printf '%s' "$tag"
    fi
}

parse_deb_revision(){
    log_step "split version into deb version, revision, and environment"

    # V4: match suffixed branch tags first (X.Y.Z-rcN-suffix, legacy .suffix)
    if [[ "$VERSION" =~ ^(.+)-rc([0-9]+)[.-](.+)$ ]]; then
```

```
        PKG_VER="${BASH_REMATCH[1]}"          # X.Y.Z      (upstream part only )
        DEB_REVISION="rc${BASH_REMATCH[2]}.${BASH_REMATCH[3]}"  # rcN.suffix
        ENVIRONMENT="staging"
    elif [[ "$VERSION" =~ ^(.+)-rc([0-9]+)$ ]]; then
        PKG_VER="${BASH_REMATCH[1]}"          # X.Y.Z      (upstream part only )
        DEB_REVISION="rc${BASH_REMATCH[2]}"   # rcN
        ENVIRONMENT="staging"
    else
        PKG_VER="$VERSION"                    # X.Y.Z
        DEB_REVISION="1"                      # stable = revision 1
        ENVIRONMENT="production"
    fi

    # The version the package actually declares, and therefore what `apt list `
    # shows on the box. PKG_VER is the upstream part alone and must never be
    # used for this: stamping it is what made tag 1.3.4-rc5 install as "1.3.4".
    DEB_VERSION="$(deb_version_from_tag "$VERSION")"

    # The tag verbatim, for human-facing output (Go ldflags, --version, logs)
    # where the tilde spelling would only get in the way.
    RAW_VERSION="$VERSION"

    export PKG_VER DEB_REVISION ENVIRONMENT DEB_VERSION RAW_VERSION

    # BACKWARD COMPATIBILITY for downstream repos
    export CARGO_VER="${PKG_VER}"

    log_ok "tag=${RAW_VERSION} deb=${DEB_VERSION} env=${ENVIRONMENT}"
}
```

Listato A.3: `package-utils.sh`.

Di `semver-utils.sh`, che conta 672 righe, sono riportate la tabella delle regole in testa al file e le funzioni citate nel capitolo 5; le parti omesse sono indicate con `[...]`.

```
# ============================================================================
# V4 versioning
#
# Three disjoint families of tags live in every repo:
#
```

```
#   stable     X.Y.Z              -- master/main releases
#   develop    X.Y.Z-rcN          -- develop pre-releases
#   branch     X.Y.Z-rcN-suffix   -- feature/bugfix/hotfix work tags
#
# What changed from V3: the next version no longer depends on the current
# branch alone. On master and develop it also depends on WHERE the commit
# came from, which is read from the merge commit. A plain commit and a merge
# are different events with different rules, and anything the rules do not
# cover is handed back to the developer as a manual tag request instead of
# being guessed.
#
# Rule table:
#
#   A work branch numbers itself off master alone and ignores every other
#   branch. Two features open at the same time therefore both build
#   X.(Y+1).0 and only their suffix tells them apart, which is enough: the
#   suffix is part of the tag, so they can never collide. That number is a
#   placeholder. The real one is decided when the branch lands on develop.
#
#   feature/* every commit         -> MINOR+1 from the latest stable, RC+1,
#                                     suffixed with the branch name
#   bugfix/*  every commit         -> PATCH+1 from the latest stable, RC+1,
#                                     suffixed
#   hotfix/*  every commit         -> PATCH+1 from the latest stable, RC+1,
#                                     suffixed. Not a placeholder: a hotfix
#                                     goes straight to master with this
    ↪ number
#
#   develop  first branch of the cycle -> the candidate's X.Y.Z, at rc0
#            merge from feature    -> MINOR+1 on the base develop is on, rc0
#            merge from bugfix     -> PATCH+1 on the base develop is on, rc0
#            merge of a branch already in this cycle -> RC+1, no new number
#            plain commit          -> RC+1 on the base develop is on
#            merge from anything   -> manual tag
#
#            develop is a staging branch: it opens no cycle of its own, and
#            with no rc tag on it there is nothing to refine, so a plain
#            commit there asks for a manual tag until the first branch lands.
#            It also never moves backwards -- each merge counts up from where
#            develop already is, never from the number the branch carried.
#
#   master   merge from develop    -> the candidate on develop's tip, without
#                                     its rc
#            merge from hotfix     -> the candidate on the hotfix tip,
    ↪ without
```

```
#                                       rc or suffix
#            plain commit          -> manual tag
#            merge from anything   -> manual tag
#
#   any other branch               -> manual tag
#
# PATCH never rolls over into MINOR. When it climbs past _PATCH_WARN_AT the
# pipeline keeps tagging but tells the developer that a manual MINOR bump is
# probably due.
# ============================================================================

# [...]

# Extract the short name after the first slash, sanitize, and limit to max 30
    ↪ chars:
# - replaces slashes and underscores with hyphens
# - removes any non-alphanumeric or non-hyphen characters
# - collapses consecutive hyphens and trims leading/trailing hyphens
# - truncates to 30 characters
_branch_suffix(){
    local raw="${1#*/}"
    local clean
    clean="$(echo "$raw" | tr '/_' '--' | tr -cd 'a-zA-Z0-9-')"
    clean="$(echo "$clean" | tr -s '-')"
    clean="${clean#-}"
    clean="${clean%-}"
    clean="${clean:0:30}"
    clean="${clean%-}"
    if [ -z "$clean" ]; then
        clean="work"
    fi
    echo "$clean"
}

# [...]

# True when HEAD is a merge commit (has a second parent). The whole V4 model
# rests on this: the team merges with real merge commits, so the second
    ↪ parent
# and the merge subject still carry the provenance. A squash or fast-forward
# merge erases both, and the rules fall through to a manual tag request.
_is_merge_commit(){
    git rev-parse --verify --quiet "HEAD^2" >/dev/null 2>&1
}

```

```
92  # [...]
93
94  # Source branch of the merge sitting on HEAD, or "" when unreadable.
95  _merge_source_branch(){
96      local subject
97      subject="$(git log -1 --format=%s HEAD 2>/dev/null || true)"
98
99      # Bitbucket pull request: "Merged in feature/x (pull request #12)"
100     if [[ "$subject" =~ ^Merged\ in\ ([^[:space:]]+)\ \(pull\ request\ #
        ↪ [0-9]+\)$ ]]; then
101         echo "${BASH_REMATCH[1]}"
102         return 0
103     fi
104     # Bitbucket manual merge: "Merged in feature/x"
105     if [[ "$subject" =~ ^Merged\ in\ ([^[:space:]]+) ]]; then
106         echo "${BASH_REMATCH[1]}"
107         return 0
108     fi
109     # git default: "Merge branch 'feature/x' into develop"
110     if [[ "$subject" =~ ^Merge\ (remote-tracking\ )?branch\ \'([^\']+)\' ]];
        ↪ then
111         local b="${BASH_REMATCH[2]}"
112         echo "${b#origin/}"
113         return 0
114     fi
115     # GitHub style, still seen on mirrors: "Merge pull request #5 from org/
        ↪ feature/x"
116     if [[ "$subject" =~ ^Merge\ pull\ request\ #[0-9]+\ from\ [^/]+/(.+)$ ]];
        ↪  then
117         echo "${BASH_REMATCH[1]}"
118         return 0
119     fi
120     # Hand-written: "Merge feature/x: some text"
121     if [[ "$subject" =~ ^Merge\ ([A-Za-z0-9._-]+/[A-Za-z0-9._/-]+) ]]; then
122         echo "${BASH_REMATCH[1]%%:*}"
123         return 0
124     fi
125
126     echo ""
127 }
128
129 # [...]
130
131 # Highest tag of a given family that the merged-in branch carries.
132 # Restricted to the second parent, so tags already on master/develop before
```

```
# the merge are not mistaken for the incoming branch's own tags.
_merged_branch_tag(){
    local pattern="$1"
    git tag --merged "HEAD^2" 2>/dev/null \
        | grep -E "$pattern" \
        | sort -V | tail -n1 || true
}

# [...]

# The candidate the merged branch was actually ON: the tag of that family
# sitting on its tip commit. _merged_branch_tag takes the highest tag in the
# whole history behind the branch instead, and the two differ as soon as a
# branch carries more than one line -- which is how master ended up
    ↪ publishing
# a number develop had already moved away from. The tip is the answer; the
# highest is only a fallback for a tip the branch pipeline left untagged.
_merged_tip_tag(){
    local pattern="$1"
    git tag --points-at "HEAD^2" 2>/dev/null \
        | grep -E "$pattern" \
        | sort -V | tail -n1 || true
}

# [...]

# Candidate carried by the merge: the tip tag, or the highest one behind it
# when the tip has none. Echoes nothing when the branch carries neither.
_incoming_candidate(){
    local pattern="$1" tag
    tag="$(_merged_tip_tag "$pattern")"
    if [ -n "$tag" ]; then
        echo "$tag"
        return 0
    fi
    tag="$(_merged_branch_tag "$pattern")"
    if [ -n "$tag" ]; then
        log_warn "The merged branch has no tag on its tip; falling back to
    ↪ the highest one it carries (${tag})."
        echo "$tag"
    fi
}

# [...]

```

```
176 parseSemver_calcVer(){
177     log_step "compute next semantic version from branch, merge origin and
    ↪ tags (v4)"
178 
179     MANUAL_TAG_REQUIRED=0
180     export MANUAL_TAG_REQUIRED
181 
182     local branch="${BITBUCKET_BRANCH:-}"
183     local btype
184     btype="$(_branch_type "$branch")"
185 
186     local is_merge="no" src_branch="" src_type="none"
187     if _is_merge_commit; then
188         is_merge="yes"
189         src_branch="$(_merge_source_branch)"
190         if [ -n "$src_branch" ]; then
191             src_type="$(_branch_type "$src_branch")"
192         else
193             src_type="unreadable"
194         fi
195     fi
196 
197     log_info "  branch=${branch}  type=${btype}  merge=${is_merge}  from=${
    ↪ src_branch:-<none>} (${src_type})"
198 
199     case "$btype" in
200         master)  _calc_master  "$is_merge" "$src_branch" "$src_type" ;;
201         develop) _calc_develop "$is_merge" "$src_branch" "$src_type" ;;
202         feature) _calc_work_branch "$branch" "minor" ;;
203         bugfix)  _calc_work_branch "$branch" "patch" ;;
204         hotfix)  _calc_work_branch "$branch" "patch" ;;
205         unknown)
206             _require_manual_tag \
207                 "Branch '${branch}' is outside the versioning rules (only
    ↪ master, develop, feature/*, bugfix/* and hotfix/* are calculated)." \
208                 "These branches are used rarely enough that the version is a
    ↪ human decision: tag this commit by hand if it needs one."
209             ;;
210     esac
211 
212     if [ "${MANUAL_TAG_REQUIRED}" = "1" ]; then
213         return 0
214     fi
215 
216     log_ok "Tag-derived next version: ${VERSION}"
```

```
}

# [...]

# Release from develop: the release IS the develop candidate without its rc.
# 1.3.0-rc4 becomes 1.3.0. Nothing is recomputed, so what master publishes is
# exactly the number the work branches converged on.
_calc_master_from_develop(){
    local src_branch="$1"
    local dev_tag ms

    dev_tag="$(_incoming_candidate '^[0-9]+\.[0-9]+\.[0-9]+-rc[0-9]+$')"
    if [ -z "$dev_tag" ]; then
        _require_manual_tag \
            "Merge on master from '${src_branch}' but the merged branch
    ↪ carries no develop tag (X.Y.Z-rcN) to release." \
            "Tag the release by hand, then let the next work branch compute
    ↪ from it."
        return 0
    fi

    VERSION="$(_version_core "$dev_tag")"
    ms="$(_latest_stable)"
    log_info "  releasing develop candidate ${dev_tag} -> ${VERSION} (latest
    ↪ stable: ${ms:-<none>})"

    if [ -n "$ms" ] && ! _v_gt "$VERSION" "$ms"; then
        _require_manual_tag \
            "Releasing ${dev_tag} would publish ${VERSION}, which is not
    ↪ ahead of the latest stable ${ms}." \
            "develop is behind what master already released. Rebuild the
    ↪ candidate on top of ${ms}, or tag this release by hand."
        return 0
    fi

    _warn_high_patch "$VERSION"
    _guard_version_is_new || return 0
}

# [...]

# -- feature / bugfix / hotfix work branches ---------------------------------
#
# Every commit produces a suffixed RC tag. The branch numbers itself off the
# latest release alone: MINOR+1 for a feature, PATCH+1 for a bugfix or a
```

```
257 # hotfix. It deliberately ignores every other open branch, so two branches
258 # started from the same release build the same X.Y.Z and are told apart by
259 # their suffix -- which is part of the tag, so nothing can collide.
260 #
261 # For feature and bugfix that number is a placeholder: develop recomputes it
262 # when the branch lands. For a hotfix it is the real one, because a hotfix
263 # goes straight to master.
264 #
265 # The base is computed once, on the branch's first tag, and reused afterwards
    ↪ ,
266 # so a release landing on master mid-branch does not move the branch. The RC
267 # counter is scoped to the pair (base, suffix).
268 _calc_work_branch(){
269     local branch="$1" kind="$2"
270     local suffix base ms
271
272     suffix="$(_branch_suffix "$branch")"
273     base="$(_branch_open_base "$suffix")"
274
275     if [ -n "$base" ]; then
276         log_info "  continuing the line already open on ${base}"
277     else
278         ms="$(_latest_stable)"
279         if [ -z "$ms" ]; then
280             _require_manual_tag \
281                 "Branch '${branch}' has no stable tag on master to number
    ↪ itself from." \
282                 "Create the first stable tag (X.Y.Z) on master by hand."
283             return 0
284         fi
285
286         base="$(_bump_version "$ms" "$kind")"
287         if [ -z "$base" ]; then
288             _require_manual_tag \
289                 "Latest stable tag '${ms}' is not valid semver, so the
    ↪ version for '${branch}' cannot be derived." \
290                 "Fix the stable tag, or tag this branch by hand."
291             return 0
292         fi
293         log_info "  new line: ${kind^^}+1 from the latest stable ${ms} -> ${
    ↪ base}"
294     fi
295
296     [ "$kind" = "patch" ] && _warn_high_patch "$base"
297     _branch_rc_bump "$base" "$suffix"
```

```
}

# [...]

# Last check before handing VERSION over: never propose a tag that already
# exists somewhere in the repo. Returns 1 (and switches to manual) when it
    ↪ does.
_guard_version_is_new(){
    if [ -z "${VERSION:-}" ]; then
        return 0
    fi
    if git rev-parse --verify --quiet "refs/tags/${VERSION}" >/dev/null 2>&1;
    ↪  then
        _require_manual_tag \
            "The computed version ${VERSION} is already an existing tag in
    ↪ this repository." \
            "Pick the next free version and tag this commit by hand."
        return 1
    fi
    return 0
}
```

Listato A.4: `semver-utils.sh` (estratto).

A.2 Esempio di pipeline di un repository

La *pipeline* completa del servizio di *scheduling*, che prima dell'intervento non aveva alcuna *pipeline* (sezione 2.2.2). Contiene lo step di verifica (listato 6.1), il calcolo della versione, la creazione manuale del tag, la pubblicazione dell'immagine con la guardia sull'ambiente e le due *pipeline* per tag (listato 6.3).

```
image: rust:latest

options:
  size: 4x

definitions:
  services:
    docker:
      memory: 7168

  caches:
    cargo: /usr/local/cargo
```

```
13      cargo-bin: ~/.cargo/bin
14      cargo-target: target
15      rustup: ~/.rustup
16
17    scripts:
18      - &utils |
19        git clone --filter=blob:none --no-checkout https://x-token-auth:${
      ↪ PIPE_SCRIPT_TOKEN}@bitbucket.org/esempio/pipeline-shared-lib.git /tmp/
      ↪ pipeline-utils
20        PIPE_REF="$(git -C /tmp/pipeline-utils tag --list --sort=-v:refname |
      ↪ grep -E '^[0-9]+\.[0-9]+\.[0-9]+$' | head -1)"
21        git -C /tmp/pipeline-utils checkout --quiet "${PIPE_REF:-master}"
22        source /tmp/pipeline-utils/reusable-bash-code/lib/bootstrap.sh
23
24    steps:
25      - step: &ci-gate
26          name: "CI: fmt + clippy + test"
27          services:
28            - docker
29          caches:
30            - cargo
31            - cargo-bin
32            - cargo-target
33            - rustup
34          script:
35            - *utils
36            - |
37              check_ci_gate
38            - |
39              # E2E tier -- the #[ignore]d, testcontainers-gated tests that `
      ↪ check_ci_gate`
40              # (cargo test --all-targets) skips. Docker required (DooD); each
      ↪ test spins up
41              # its own throwaway Redis, serialized by serial_test's #[serial]
      ↪ (and the
42              # docker-serial group in .config/nextest.toml). No Redis service
      ↪ needed beyond
43              # `docker` above. Scoped to the e2e_api binary since the hermetic
      ↪  tests already
44              # ran in check_ci_gate.
45              which cargo-nextest 2>/dev/null || cargo install cargo-nextest --
      ↪ locked
46              cargo nextest run --features testcontainers --run-ignored all -E
      ↪ 'binary(e2e_api)'
47
```

```
48      #
        ↪ ------------------------------------------------------------------------
        ↪
49      # Reusable script fragments (anchors)
50      #
        ↪ ------------------------------------------------------------------------
        ↪
51      - step:
52          name: "[anchors] reusable fragments"
53
54          script:
55            - *utils
56            - &ecr_build_and_tag |
57              set -euo pipefail
58              mkdir -p application-folder
59              cd application-folder
60              export DOCKER_BUILDKIT=0
61
62              FULL_IMAGE="service/${IMAGE_NAME}"
63              echo "Building ${FULL_IMAGE}:${ENVIRONMENT}-${LATEST_TAG}"
64
65              docker build -t "${FULL_IMAGE}:${ENVIRONMENT}-latest" ../
66              docker tag "${FULL_IMAGE}:${ENVIRONMENT}-latest" "${FULL_IMAGE}:$
        ↪ {ENVIRONMENT}-${LATEST_TAG}"
67
68            - &ecr_push_pipe
69              pipe: atlassian/aws-ecr-push-image:2.6.0
70              variables:
71                AWS_ACCESS_KEY_ID: $AWS_ACCESS_KEY_ID
72                AWS_SECRET_ACCESS_KEY: $AWS_SECRET_ACCESS_KEY
73                AWS_DEFAULT_REGION: $AWS_ECR_REGION
74                IMAGE_NAME: "service/${IMAGE_NAME}"
75                TAGS: "${ENVIRONMENT}-latest ${ENVIRONMENT}-${LATEST_TAG}"
76
77            - &resolve_tag_env |
78              set -euo pipefail
79
80              export IMAGE_NAME="job-scheduler"
81
82              # ====================================================
83              # PATH 1: Tag pipeline (BITBUCKET_TAG is set)
84              # ====================================================
85              if [ -n "${BITBUCKET_TAG:-}" ]; then
86                echo "Tag pipeline detected: BITBUCKET_TAG=${BITBUCKET_TAG}"
87
```

```
88              resolve_environment
89
90            # ======================================================
91            # PATH 2: Branch pipeline (use artifacts)
92            # ======================================================
93            else
94              echo "Branch pipeline detected"
95
96              init_tag_guard
97
98              tag_wall
99
100             echo "Resolved: LATEST_TAG=${LATEST_TAG}, ENVIRONMENT=${
    ↪ ENVIRONMENT}, IMAGE_NAME=${IMAGE_NAME}"
101           fi
102
103           # Refuse to publish when the resolved ENVIRONMENT and the step's
104           # deployment disagree, and block any rc tag from reaching
    ↪ production
105           # however it got routed here. Shared implementation:
106           # pipeline-shared-lib/reusable-bash-code/lib/pipeline-guards.sh
107           deployment_env_guard
108
109   #
    ↪ ------------------------------------------------------------------------
    ↪
110   # Step 1 (automatic): Version calculation
111   #
    ↪ ------------------------------------------------------------------------
    ↪
112   - step: &version_calc
113       name: "Compute next version and state"
114       script:
115         - set -euo pipefail
116         - git fetch --tags --force
117         - *utils
118         - |
119           # version_calc (shared lib): version_manager + cargo_exists +
120           # Cargo.toml reconcile (check_cargo) + writes .version/.tag_ready
    ↪ .
121           # Safe in repos with or without Cargo.toml.
122           version_calc
123       artifacts:
124         - .version
125         - .tag_ready
```

```
126
127     #
        ↪ ------------------------------------------------------------------------
        ↪
128     # Step 2 (manual): Create tag with safety checks
129     #
        ↪ ------------------------------------------------------------------------
        ↪
130     - step: &manual_tag_only
131         name: "Create tag with safety checks"
132         trigger: manual
133         script:
134           - set -euo pipefail
135           - *utils
136           - |
137             tag_manager
138
139         artifacts:
140           - .tag_ready
141
142     #
        ↪ ------------------------------------------------------------------------
        ↪
143     # Step 3: Build and Push (uses deployment variables)
144     #
        ↪ ------------------------------------------------------------------------
        ↪
145     - step: &build-and-push
146         name: "Build and Push"
147         services:
148           - docker
149         caches:
150           - docker
151         script:
152           - *utils
153           - *resolve_tag_env
154           - *ecr_build_and_tag
155           - *ecr_push_pipe
156
157 pipelines:
158   default:
159     - step: *ci-gate
160     - step:
161         script:
162           - echo "Nothing to do, but everything's done!"
```

```yaml

  branches:
    master:
      - step: *ci-gate
      - step: *version_calc
      - step: *manual_tag_only
      - parallel:
          - step:
              <<: *build-and-push
              name: "Build and Push to Production (Europe)"
              trigger: manual

    develop: &staging-branches
      - step: *ci-gate
      - step: *version_calc
      - step: *manual_tag_only
      - parallel:
          - step:
              <<: *build-and-push
              name: "Build and Push to Staging (Europe)"
              trigger: manual

    "feature/*": *staging-branches

    "bugfix/*": *staging-branches

    "hotfix/*": *staging-branches

  # ------------------------------------------------------------------------
  # Tag-based pipeline: only runs when a tag is pushed
  # Uses same &build-and-push (resolve_tag_env detects BITBUCKET_TAG
    ↪ automatically)
  # ------------------------------------------------------------------------
  tags:
  # ------------------------------------------------------------------
  # ORDER MATTERS: Bitbucket runs the FIRST matching tag pipeline
  # only. The RC glob is listed before the stable glob because
  # `1.0.0-rc1` matches both (stable's trailing `*` swallows
  # `-rc1`). Listing RC first routes prereleases to staging.
  # Both globs are digit-anchored so junk tags can't slip in.
  # ------------------------------------------------------------------

    "[0-9]*.[0-9]*.[0-9]*-rc[0-9]*":
      - step: *ci-gate
      - parallel:
```

```
          - step:
              <<: *build-and-push
              name: "Build and Push to Staging (Europe)"
              trigger: automatic

    "[0-9]*.[0-9]*.[0-9]*":
      - step: *ci-gate
      - parallel:
          - step:
              <<: *build-and-push
              name: "Build and Push to Production (Europe)"
              trigger: automatic
```

Listato A.5: `bitbucket-pipelines.yml` del servizio di *scheduling*.

A.3 Template dei Dev Container

Il template Rust nella prima versione, descritta nel capitolo 7.

```
FROM mcr.microsoft.com/devcontainers/rust:1-1-bookworm

# Update rustup and install cargo tools
ENV PATH=/home/vscode/.cargo/bin:/root/.cargo/bin:/usr/local/cargo/bin:$PATH

# Update rustup and add clippy
RUN set -e; \
    rustup update && \
    rustup component add clippy

# Install required cargo tooling strictly; optional developer tools are best-effort.
RUN set -eux; \
        install_required(){ cmd="$1"; name="$2"; if ! command -v "$cmd" >/dev/null 2>&1; then cargo install "$name" --locked; fi; }; \
        install_optional(){ cmd="$1"; name="$2"; \
            if ! command -v "$cmd" >/dev/null 2>&1; then \
                cargo binstall -y "$name" || cargo install "$name" --locked || echo "Skipping optional tool: $name"; \
            fi; \
        }; \
        install_required cargo-chef cargo-chef; \
        install_optional cargo-binstall cargo-binstall; \
        install_optional cargo-audit cargo-audit; \
```

```
        install_optional cargo-clean-all cargo-clean-all; \
        install_optional cargo-clean-recursive cargo-clean-recursive; \
        install_optional cargo-deb cargo-deb; \
        install_optional cargo-edit cargo-edit; \
        install_optional cargo-expand cargo-expand; \
        install_optional cargo-machete cargo-machete; \
        install_optional cargo-outdated cargo-outdated; \
        install_optional cargo-watch cargo-watch; \
        install_optional cargo-nextest cargo-nextest; \
        install_optional cargo-tarpaulin cargo-tarpaulin

ENV CARGO_TARGET_DIR=/usr/local/cargo/target
WORKDIR /workspaces/job-scheduler

# Pre-fetch and pre-build dependencies from Cargo.toml at image build time.
COPY Cargo.toml ./
COPY src ./src
RUN set -e; cargo chef prepare --recipe-path recipe.json

RUN set -e; cargo chef cook --release --recipe-path recipe.json

# Fix permissions for postCreateCommand (runs as vscode)
RUN set -e; \
    mkdir -p /usr/local/cargo/registry /usr/local/cargo/git /usr/local/cargo/
    ↪ target && \
    chown -R vscode:vscode /usr/local/cargo
```

Listato A.6: `Dockerfile` del template Rust.

```
{
  "name": "job-scheduler-dev",
  //"dockerComposeFile": "docker-compose.yml",
  //"service": "dev",
  "build": {
    "dockerfile": "Dockerfile",
    "context": ".."
  },
  "workspaceFolder": "/workspaces/job-scheduler",
  "features": {
    "ghcr.io/devcontainers/features/node:1": {
      "version": "20"
    },
    "ghcr.io/devcontainers/features/java:1": {
      "version": "17"
```

```
    }
  },
  /*"runServices": [
    "redis"
  ],*/
  //"shutdownAction": "stopCompose",
  "shutdownAction": "stopContainer",
  /*"forwardPorts": [
    3000,
    6379
  ],*/
  "postCreateCommand": "sh -lc 'set -e; npm install -g @openapitools/openapi-
    ↪ generator-cli; cargo fetch; cargo build --all-targets || echo \"[
    ↪ postCreate] cargo build failed, container remains usable\"'",
  "customizations": {
    "vscode": {
      "extensions": [
        "rust-lang.rust-analyzer",
        "vadimcn.vscode-lldb",
        "fill-labs.dependi",
        "tamasfe.even-better-toml",
        "usernamehw.errorlens",
        "eamodio.gitlens",
        "shd101wyy.markdown-preview-enhanced",
        "keyshout.sqlite-db-viewer",
        "gruntfuggly.todo-tree",
        "christian-kohler.path-intellisense",
        "johnpapa.vscode-peacock",
        "solomonkinard.bookmark",
        "miguelsolorio.fluent-icons",
        "oderwat.indent-rainbow",
        "mhutchie.git-graph",
        "arjun.swagger-viewer",
        "a2crunch.vscode-openapi",
        "cordx56.rustowl-vscode",
        "hbenl.vscode-test-explorer",
        "hwence.rust-test-explorer",
        "jstearcy.rust-doc-viewer",
        "panicbit.cargo",
        "polymeilex.rust-targets",
        "swellaby.vscode-rust-test-adapter",
        "zerotaskx.rust-extension-pack",
        "zhangyue.rust-mod-generator",
        "anthropic.claude-code",
        "clemenspeters.format-json",
```

```
59          "ms-azuretools.vscode-containers",
60          "ms-azuretools.vscode-docker",
61          "ms-vscode.anycode-rust",
62          "ms-vscode.test-adapter-converter",
63          "redhat.vscode-yaml"
64        ]
65      }
66    }
67  }
```

Listato A.7: `devcontainer.json` del template Rust.

Bibliografia

[1] Atlassian. How build minutes/seconds are calculated in bitbucket cloud. https://support.atlassian.com/bitbucket-cloud/kb/how-build-minutes-seconds-are-calculated-in-bitbucket-cloud/, . Consultato tra marzo e ottobre 2026.

[2] Atlassian. Bitbucket pipelines documentation. https://support.atlassian.com/bitbucket-cloud/docs/get-started-with-bitbucket-pipelines/, . Consultato tra marzo e ottobre 2026.

[3] cargo-deb contributors. cargo-deb: Debian packages from cargo projects. https://github.com/kornelski/cargo-deb. Consultato tra marzo e ottobre 2026.

[4] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014. ISBN 978-1484200773.

[5] Debian Project. Debian policy manual. https://www.debian.org/doc/debian-policy/. Consultato tra marzo e ottobre 2026.

[6] Development Containers. Development containers specification. https://containers.dev/. Consultato tra marzo e ottobre 2026.

[7] Docker Inc. Docker documentation. https://docs.docker.com/. Consultato tra marzo e ottobre 2026.

[8] Vincent Driessen. A successful Git branching model. https://nvie.com/posts/a-successful-git-branching-model/, 2010. Consultato tra settembre e ottobre 2026.

[9] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018. ISBN 978-1942788331.

[10] Martin Fowler. Continuous integration. https://martinfowler.com/articles/continuousIntegration.html, 2006. Consultato tra settembre e ottobre 2026.

[11] Free Software Foundation. GNU bash reference manual. https://www.gnu.org/software/bash/manual/. Consultato tra marzo e ottobre 2026.

[12] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. ISBN 978-0321601919.

[13] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, 2nd edition, 2023. ISBN 978-1718503106.

[14] nextest contributors. cargo-nextest: a next-generation test runner for rust. https://nexte.st/. Consultato tra marzo e ottobre 2026.

[15] Luca Palmieri. cargo-chef: cache the dependencies of a rust project. https://github.com/LukeMathWalker/cargo-chef. Consultato tra marzo e ottobre 2026.

[16] Tom Preston-Werner. Semantic versioning 2.0.0. https://semver.org/. Consultato tra marzo e ottobre 2026.

[17] The Rust Project. The cargo book. https://doc.rust-lang.org/cargo/. Consultato tra marzo e ottobre 2026.

Ringraziamenti

Arrivato alla fine di questo percorso, so di non aver raggiunto questo traguardo da solo. Nonostante le difficoltà incontrate, porterò sempre con me un bellissimo ricordo di questi anni, soprattutto grazie alle persone con cui li ho condivisi.

Devo cominciare ringraziando quelle persone che, sera dopo sera, ci sono sempre state, riuscendo a farmi dimenticare anche le giornate peggiori. Abbiamo giocato praticamente a tutto, fatto quiz improbabili e inventato qualsiasi cosa pur di trascorrere del tempo insieme. E alla fine non importava nemmeno cosa facessimo, perché bastava esserci. Grazie di cuore a **Mauro**, **Senia**, **Stella** e **Vecchio**, che ovviamente merita di essere nominato. Non sapete quante volte siete riusciti a migliorarmi una giornata senza neanche accorgervene.

E poi c'è **Andrea**, l'ultima novità, se non la più bella. Siamo così simili e allo stesso tempo così diversi. Tu sei scemo, io lo sono altrettanto, e quanto è bello esserlo insieme. Non credo ci sia qualcuno con cui mi trovi più in sintonia. Ci conosciamo da relativamente poco, eppure abbiamo legato fin da subito, come se ci conoscessimo da una vita. Siamo due esseri strani in questo mondo di persone con i fiocchi. Non chiedermi cosa voglia dire, ma sento che sia la definizione perfetta per noi.

Con te posso scherzare e insultarti quanto voglio, sapendo che non te la prenderai mai, proprio come io non me la prenderei con te. E nonostante tutte le stupidaggini che diciamo, sei una persona buonissima, e nessuno potrebbe mai dire il contrario.

Certo, riesci anche a farmi perdere la pazienza come pochi altri. Ti distrai per qualsiasi cosa, ti incanti a fissare il nulla e quando torno a parlarti scopro che non hai ascoltato nemmeno una parola. E puntualmente, anche quando vorrei soltanto insultarti ancora di più, finisci per farmi ridere.

Nonostante tu sia arrivato da poco nella mia vita, sei già diventato una delle

persone a cui tengo di più, e spero che questa nostra assurda amicizia continui ancora per tantissimo tempo.

Spero soltanto che, almeno questa volta, tu sia riuscito a prestare attenzione fino alla fine.

Ti voglio bene, Andrea.

Parlando di amici, è impossibile non pensare a chi mi accompagna ormai dai tempi delle superiori.

Stefano, non sei mai stato a casa con me, eppure casa tua è sempre stata uno di quei posti in cui mi sono sentito a casa. Tra noi non c'è mai stato bisogno di pensarci due volte prima di chiedere aiuto, e me lo hai dimostrato ancora una volta quando mi hai aiutato con lo stage che mi ha portato fino a questa laurea. Ma voglio ringraziarti soprattutto per tutte le passioni condivise, per le passeggiate senza una destinazione, fatte soltanto per esplorare qualcosa, parlare del più e del meno e sparare le nostre solite idiozie al vento. Grazie davvero, Stefano.

E come non nominare **Leo**, una di quelle persone a cui basta scrivere per sapere che ci sarà. Con te trascorrere del tempo ha sempre un sapore diverso, non importa dove siamo o cosa facciamo: so già che ci divertiremo. Fin dai tempi delle superiori siamo sempre stati quelli dell'«accollo subito», pronti a qualsiasi cosa pur di organizzare una serata insieme. Spero che continueremo a trovare il modo di lasciarci alle spalle tutto quello che ci pesa, magari andando a farci una sciata e dimenticandoci per qualche ora del resto del mondo. Non vedo l'ora di provare ancora quell'adrenalina che riesco a sentire soltanto quando facciamo qualcosa di nostro. Grazie per tutte le serate e per quelle che ancora ci aspettano.

A proposito di sciare… qualcuno ha detto sciare? Siete pazzi? Sono sicuro che **Angelo** abbia già spalancato gli occhi e stia facendo qualsiasi gesto possibile per far capire che vuole venire anche lui.

Angelo, ti voglio un bene immenso. Sei un fratello splendido e non ti scambierei con nessun altro al mondo. Non vedo l'ora di continuare a viaggiare insieme a te, scoprire posti nuovi e imparare a guardare il mondo anche attraverso i tuoi occhi. Ogni volta che siamo insieme riusciamo a divertirci, e persino i nostri litigi finiscono

per diventare qualcosa su cui ridere.

In questi anni la distanza non è sempre stata facile, e proprio per questo ogni volta che entravi in chiamata era un piccolo momento di felicità. Mi bastava vederti, sapere come stavi e scoprire quanto fossi cresciuto dall'ultima volta. Vederti diventare grande mi rende felice, ma mi fa anche capire quanto sia bello essere tuo fratello. Spero che, per quanto cresceremo e cambieranno le nostre vite, continueremo sempre a trovare il tempo per fare tutte le nostre stupidaggini. Abbiamo ancora tantissime montagne da scendere insieme, Angelo, e voglio credere che continueremo a sciare fianco a fianco per tutta la vita.

E non posso parlare di Angelo senza parlare di **Aurora**.

Lo sappiamo entrambi, io e te siamo diversi. Non sei una persona di molte parole con me e a volte sembra quasi che viviamo in due mondi differenti. Ma ho imparato ad apprezzare tutti quei piccoli gesti con cui, a modo tuo, mi fai sentire quanto ci tieni. Come quando torno da Torino e sei la prima a fare un video per annunciare che tuo fratello è tornato, oppure quando devi assolutamente dimostrarmi che, mentre io non ci sono e Angelo passa il tempo a dormire, sei tu quella che fa tutto in casa.

Sono piccole cose, ma per me significano tanto. So che sei una bellissima persona e sono felice di vederti crescere, anche se a volte la distanza mi fa sentire come se mi perdessi dei pezzi della tua vita. Non vedo l'ora di ritrovarci tutti e tre insieme, a fare i pazzi come abbiamo sempre fatto fin da piccoli. E spero che, per quanto cambieranno le nostre vite, ci sarà sempre qualcosa capace di farci tornare quei tre bambini che ridevano per qualsiasi stupidaggine.

Voglio ringraziare anche **Nonno**, perché per tutta la nostra famiglia sei sempre stato una persona su cui poter contare. Ogni volta che qualcuno ha avuto bisogno di qualcosa, hai sempre cercato di esserci e di dare una mano, senza tirarti indietro. Hai fatto tanto per tutti noi, spesso senza chiedere nulla in cambio, e so che una parte di ciò che siamo riusciti a costruire oggi è anche merito tuo. Grazie per tutto quello che hai fatto per permetterci di arrivare fin qui.

Zio Giovanni, c'è una frase che mi hai detto e che credo porterò con me per sempre: «So cosa vuol dire vivere da solo».

Me l'hai detta mentre mi porgevi cento euro e io cercavo in tutti i modi di rifiutarli. Alla fine ti ho risposto soltanto: «Grazie, zio». Ma quello che forse non sai è quanto mi abbiano reso felice quelle parole.

Non erano i soldi. Era sapere che qualcuno capiva davvero cosa significasse vivere da solo, dover pensare a tutto e cercare di cavarsela senza pesare sulle persone che ti stanno aiutando. Era sapere che, se qualcosa fosse andato storto, ci sarebbe stato qualcuno capace di comprendermi senza bisogno di troppe spiegazioni.

E forse mi ha fatto ancora più piacere perché quelle parole arrivavano proprio da te. Ho preso tantissimo dal tuo carattere, dal tuo umorismo e dal tuo modo di affrontare le cose. Non siamo due persone da grandi discorsi, e probabilmente nemmeno questa volta riuscirei a dirti tutto questo guardandoti negli occhi. Ma voglio che tu sappia che sono orgoglioso di quello che ho preso da te, e che quel giorno, con una frase così semplice, mi hai fatto sentire molto meno solo. Grazie, zio.

Devo ringraziare anche **Zio Rino**, che non mi ha mai fatto mancare un posto in cui stare, un appoggio o una porta aperta quando ne avevo bisogno. E insieme a lui voglio ringraziare **Emanuel**, **Chiara** e **Nicole**, che sono sempre stati pronti ad accoglierci e a farci sentire a casa. Grazie a voi ho potuto trascorrere momenti lontano dall'università e dalle sue preoccupazioni, ritrovando un po' della leggerezza di cui ogni tanto avevo bisogno. Spero di poter sempre contare su di voi, proprio come voi potrete sempre contare su di me.

E a festeggiare questo traguardo insieme a me c'è anche la persona che, più di chiunque altro, mi è stata accanto durante questi anni.

Laura, tu hai visto tutto. Mi hai visto festeggiare dopo un esame superato e piangere quando le cose non andavano come speravo. Hai conosciuto le mie giornate migliori, ma anche quelle in cui ero stanco, scoraggiato e tutto sembrava più difficile del dovuto. E nonostante questo, sei sempre stata lì.

Ci siamo fatti da spalla a vicenda, ci siamo sostenuti e siamo cresciuti insieme, attraversando un periodo della nostra vita che, senza di te, sarebbe stato sicuramente molto più triste. E non posso dimenticare tutti quei bacini che mi mandavi attraverso lo schermo durante le infinite sessioni di studio. Magari per te erano gesti da niente, ma riuscivano sempre a strapparmi un sorriso, anche quando avrei voluto soltanto chiudere i libri e lasciare perdere tutto.

Voglio ringraziarti per tutte quelle volte in cui, pur di vedermi felice, hai fatto

più di quanto avresti mai pensato di fare. Per essere venuta a sciare con me, per aver imparato ad appassionarti alle cose che amo e per aver condiviso esperienze che, altrimenti, non sarebbero state le stesse. Ma soprattutto voglio ringraziarti perché, in mezzo a tutto questo, sei stata anche la mia migliore amica.

Abbiamo ancora tantissimi posti da vedere e tantissime cose da fare insieme. I viaggi che abbiamo fatto finora mi hanno soltanto fatto venire ancora più voglia di partire con te, scoprire il mondo e riempirci di nuovi ricordi. Non so dove ci porterà la vita, ma spero che continueremo a esplorarla insieme, con la stessa voglia di condividere tutto che ci ha accompagnati fino a oggi.

Grazie di tutto, Laura.

Ci sono poi due persone senza le quali non sarei nemmeno arrivato al punto di poter scrivere queste parole.

Mamma e **Papà**, grazie.

Grazie perché, prima ancora di aiutarmi a raggiungere questo traguardo, mi avete dato la possibilità di provarci. Siete due persone tanto diverse tra loro, eppure insieme siete tutto ciò di cui avrei mai potuto avere bisogno. Non vi cambierei con nessuno al mondo.

Crescendo con voi ho imparato a non avere paura di ciò che non conosco, a mettermi in gioco e a provare anche quando non ero sicuro di riuscire. Non mi avete mai tenuto chiuso in una bolla, ma mi avete lasciato libero di scoprire il mondo, di sbagliare e di capire da solo quale strada volessi prendere.

C'è una cosa che forse non vi ho mai detto abbastanza: ogni volta che nella mia vita succede qualcosa, mi ritrovo a pensare a ciò che mi avete insegnato. A volte sono frasi che mi avete detto anni fa, altre volte semplicemente gesti che vi ho visto fare per tutta la vita. Magari in quel momento non ci facevo nemmeno caso, ma oggi quelle parole e quei gesti continuano a guidarmi nelle mie scelte.

Se oggi riesco ad affrontare il mondo con un po' più di sicurezza, se sento di avere quel passo in più che a volte mi permette di andare avanti, so che non è soltanto merito mio. È soprattutto il risultato di tutto ciò che voi avete costruito dentro di me.

Non so se riuscirò mai a restituirvi tutto ciò che avete fatto per me, ma spero almeno di riuscire a rendervi orgogliosi della persona che sto diventando. Perché questo traguardo porta il mio nome, ma senza di voi non sarebbe mai esistito.

Vi voglio un mondo di bene, Mamma e Papà. Grazie davvero per tutto.

Ma c'è una persona a cui voglio dedicare un ringraziamento speciale. Una persona che è presente nella mia vita ogni singolo giorno, senza mai farmi pesare nulla, senza mai chiedere niente in cambio, ma essendo sempre pronta a fare qualsiasi cosa per me.

Nonna, io davvero non so come ringraziarti.

Da piccolo, ogni giorno dopo scuola venivo a piedi a casa tua per mangiare. Mi preparavi sempre le cose più buone del mondo e poi mi lasciavi riposare, lontano da tutto, come se non ci fosse un posto migliore in cui poter stare. Mi chiedevi com'era andata a scuola, ascoltavi quello che avevo da raccontarti e ti interessavi a ogni piccola cosa della mia giornata.

E alla fine, ripensandoci oggi, mi viene da sorridere perché non è cambiato praticamente niente.

Sono cresciuto, la scuola è diventata università e le giornate sono diventate più complicate, ma tu hai continuato a chiedermi come fosse andata, a preoccuparti per me e a farmi sentire sempre nello stesso posto sicuro in cui mi rifugiavo da bambino. Perché casa tua è sempre stata, e continua a essere, uno dei posti più belli in cui possa trovarmi.

Anche adesso che siamo lontani, ogni mattina mi sveglio con il tuo buongiorno e ogni sera mi addormento con la tua buonanotte. Sono gesti semplicissimi, eppure sono diventati una parte della mia vita a cui non saprei rinunciare.

Quando superavo un esame, a volte sembrava quasi che tu lo sapessi ancora prima che riuscissi a dirtelo. Arrivavano subito i tuoi messaggi, i tuoi mille auguri e tutta quella felicità che riuscivi a provare per qualcosa che avevo fatto io. E quando invece un esame andava male, eri sempre pronta a dirmi che non era giusto, che erano i professori a non capire e che sicuramente avrei meritato di più.

Forse non eri sempre la persona più imparziale del mondo, ma eri sempre, senza alcun dubbio, la mia più grande tifosa.

Voglio ringraziarti soprattutto per aver creduto in me anche quando io facevo fatica a farlo, per non avermi mai fatto sentire solo, nemmeno quando eravamo lontani. Perché, nonostante tutti i cambiamenti di questi anni, tu sei sempre rimasta lì, felice per ogni mio successo e pronta a consolarmi dopo ogni sconfitta.

E forse la cosa più bella è proprio questa: per quanto possa crescere, per quanti traguardi possa raggiungere e per quanto lontano possa andare, con te potrò sempre essere quel bambino che tornava da scuola e ti faceva i disegnini per dirti quanto eri speciale.

Oggi non ho un disegno da regalarti. Ho soltanto queste parole, scritte alla fine di un percorso che anche tu hai vissuto insieme a me, un buongiorno, una buonanotte e

un esame alla volta.

Sei la nonna migliore del mondo, e continuerò a dirtelo proprio come facevo quando ero piccolo.

Grazie di tutto, Nonna. Ti voglio un bene che queste parole non riusciranno mai a spiegare.

E adesso, dopo aver ringraziato tutte le persone che hanno reso questo percorso così speciale, credo che mi manchi soltanto una cosa: uno specchio.

Perché, alla fine, voglio ringraziare anche **me stesso**.

Per tutte le volte in cui sarebbe stato più facile fermarmi e invece ho continuato. Per gli ostacoli che ho superato, per gli errori che ho commesso e per non essermi mai arreso davanti alle difficoltà.

Sono arrivato fin qui grazie a tante persone, ma sono stato io a dover affrontare ogni esame, ogni paura e ogni momento di sconforto.

E adesso, caro specchio, mi aspetto grandi cose da te.